



Facultad de
**Ciencias Sociales y
Tecnologías de la Inf.**
Talavera de la Reina. UCLM

UNIVERSIDAD DE CASTILLA-LA MANCHA
Tecnologías y Sistemas de Información

GRADO EN INGENIERÍA INFORMÁTICA
SISTEMAS DE INFORMACIÓN

Trabajo Fin de Grado

**Creación de una Plataforma/API para la
Evaluación de Calidad de Datos
en Proyectos de IA**

Autoría: Alejandro Manuel Saiz García

Tutoría: Fernando Gualo Cejudo y Ricardo Pérez del Castillo

Junio, 2026

Alejandro Manuel Saiz García

Talavera de la Reina – España

© 2026 Alejandro Manuel Saiz García

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Se permite la copia, distribución y/o modificación de este documento bajo los términos de la Licencia de Documentación Libre GNU, versión 1.3 o cualquier versión posterior publicada por la *Free Software Foundation*; sin secciones invariantes. Una copia de esta licencia esta incluida en el apéndice titulado «GNU Free Documentation License».

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando la persona autora haya sido informada de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

TRIBUNAL:

Presidencia:

Vocal:

Secretaría:

FECHA DE DEFENSA:

CALIFICACIÓN:

PRESIDENCIA

VOCAL

SECRETARÍA

Fdo.:

Fdo.:

Fdo.:

Resumen

La inteligencia artificial se ha consolidado como una tecnología transformadora en sectores como la sanidad, las finanzas, la industria o la administración pública, y la calidad de los datos que alimentan estos sistemas constituye un factor determinante en su fiabilidad y en la equidad de sus resultados. Sin embargo, las herramientas disponibles para la evaluación de dicha calidad presentan limitaciones significativas: las bibliotecas basadas en código imponen una barrera técnica elevada, mientras que los *scripts* ad hoc carecen de estandarización, historización y mecanismos de decisión automatizados.

Este Trabajo Fin de Grado presenta DATAQUAL, una plataforma web *full-stack* para la evaluación automatizada de la calidad de datos. La plataforma implementa siete métricas de calidad alineadas con los estándares ISO/IEC 5259 (calidad de datos para Inteligencia Artificial (IA)) e ISO/IEC 25012 (modelo general de calidad de datos), un motor modular y extensible, un sistema de *fingerprinting* SHA-256 para la trazabilidad de incidencias entre evaluaciones y *Quality Gates* configurables inspirados en SonarQube. El sistema se despliega como ocho servicios Docker orquestados con Docker Compose, combinando una interfaz web interactiva (Next.js, React) con una API RESTful (Flask, Python) y procesamiento asíncrono (Celery, Redis).

La plataforma ha sido validada con conjuntos de datos ampliamente utilizados en el ámbito de la IA y la ciencia de datos, confirmando la detección efectiva de problemas de calidad, la trazabilidad de incidencias entre evaluaciones sucesivas y la correcta clasificación de datasets mediante *Quality Gates* con estados PASSED, WARNING y FAILED.

DATAQUAL contribuye a democratizar el acceso a la evaluación estandarizada de calidad de datos, eliminando la barrera técnica que suponen las herramientas basadas en código. Su alineación con el Reglamento Europeo de Inteligencia Artificial facilita el cumplimiento normativo en un contexto regulatorio cada vez más exigente, contribuyendo al desarrollo de sistemas de IA más fiables, equitativos y auditables.

Abstract

Artificial intelligence has become a transformative technology in sectors such as health-care, finance, industry and public administration, and the quality of the data that feed these systems is a determining factor in their reliability and the fairness of their outcomes. However, the tools available for data quality assessment have significant limitations: code-based libraries impose a high technical barrier, while ad hoc scripts lack standardisation, historisation and automated decision mechanisms.

This undergraduate thesis presents DATAQUAL, a full-stack web platform for automated data quality evaluation. The platform implements seven quality metrics aligned with the ISO/IEC 5259 (data quality for AI) and ISO/IEC 25012 (general data quality model) standards, a modular and extensible engine, a SHA-256 fingerprinting system for issue traceability across evaluations, and configurable Quality Gates inspired by SonarQube. The system is deployed as eight Docker services orchestrated with Docker Compose, combining an interactive web interface (Next.js, React) with a RESTful API (Flask, Python) and asynchronous processing (Celery, Redis).

The platform has been validated with widely used datasets in the field of AI and data science, confirming the effective detection of quality issues, issue traceability across successive evaluations, and correct dataset classification through Quality Gates with PASSED, WARNING and FAILED states.

DATAQUAL contributes to democratising access to standardised data quality evaluation by removing the technical barrier imposed by code-based tools. Its alignment with the European Artificial Intelligence Act supports regulatory compliance in an increasingly demanding landscape, fostering the development of more reliable, fair and auditable AI systems.

Agradecimientos

A mis padres, Teresa y Pedro, por apoyarme en todo, aconsejarme cuando hacía falta y confiar siempre en mí para tomar la última decisión. Permitirme equivocarme con respaldo es una de las mejores formas de crecer.

A Carmen, por haberse comido mis bloqueos y mis momentos de debilidad, y por tener siempre la capacidad de levantarme cuando más lo necesitaba.

A Rocío y a Jaime, veros luchar por lo que os proponéis es el mejor regalo que puede tener vuestro hermano.

A mis amigos, por darme los mejores momentos durante mi etapa universitaria, por ser familia cuando la mía estaba lejos y por ser mi apoyo durante estos años. Porque aunque los caminos ahora se separan, no nos falta una excusa para volver a vernos.

A mis tutores, Fernando Gualo Cejudo y Ricardo Pérez del Castillo, por la paciencia que han tenido conmigo y por exigirme cuando era necesario.

Gracias.

Alejandro

Índice general

Resumen	III
Abstract	IV
Agradecimientos	V
Índice general	VI
Índice de cuadros	XII
Índice de figuras	XIV
Listado de acrónimos	XVI
1. Introducción	1
1.1. Motivación	1
1.2. Presentación de la solución	2
1.3. Justificación de la solución	3
1.4. Público objetivo	4
1.5. Alcance del proyecto	4
1.6. Estructura del documento	4
2. Antecedentes y estado del arte	6
2.1. Importancia de la calidad de datos en IA	6
2.1.1. Marcos conceptuales fundacionales	6
2.1.2. Impacto económico de la mala calidad de datos	6
2.1.3. El paradigma data-centric AI	7
2.2. Marco normativo: estándares ISO	8

ÍNDICE GENERAL

2.2.1.	ISO/IEC 25012: Modelo de calidad de datos	8
2.2.2.	ISO/IEC 5259: Calidad de datos para analítica e IA	8
2.2.3.	ISO/IEC 25024: Medición de la calidad de datos	10
2.3.	Alcance metodológico	10
2.3.1.	Taxonomía de características de calidad según ISO/IEC 5259-2 . .	10
2.3.2.	Características evaluables algorítmicamente	11
2.3.3.	Características fuera del alcance del proyecto	13
2.3.4.	Priorización de características para el MVP	14
2.4.	Estado del arte: herramientas existentes	15
2.4.1.	Great Expectations	15
2.4.2.	Apache Deequ	15
2.4.3.	DQOps	16
2.4.4.	Soda Core	16
2.4.5.	Otras herramientas relevantes	17
2.4.6.	Análisis comparativo	17
2.5.	Diferenciación de DATAQUAL	18
2.6.	Justificación académica	19
3.	Objetivos	21
3.1.	Objetivo general	21
3.2.	Objetivos específicos	21
3.2.1.	OE1. Módulo de ingesta y almacenamiento	21
3.2.2.	OE2. Motor de evaluación con métricas parametrizables	22
3.2.3.	OE3. API RESTful con especificación formal	23
3.2.4.	OE4. Aplicación web con <i>dashboards</i> interactivos	24
3.2.5.	OE5. Mecanismos de identificación de problemas	25
3.3.	Objetivos transversales	26
3.4.	Alcance y limitaciones	27
4.	Metodología y herramientas	29
4.1.	Gestión del proyecto	29

ÍNDICE GENERAL

4.1.1.	Metodología ágil adoptada: Scrum adaptado	29
4.1.2.	Adaptación de Scrum al TFG individual	30
4.1.3.	Herramientas de gestión	31
4.2.	Metodología de desarrollo	31
4.2.1.	Desarrollo iterativo	31
4.2.2.	Prototipado	32
4.3.	Pila tecnológica	32
4.3.1.	<i>Frontend</i>	33
4.3.2.	<i>Backend</i>	33
4.3.3.	Base de datos y almacenamiento	33
4.3.4.	Procesamiento asíncrono y orquestación	35
4.4.	Entorno de desarrollo	35
4.5.	Planificación temporal	36
4.6.	Backlog inicial e historias de usuario	39
4.7.	Estimación de esfuerzo	41
4.7.1.	Técnica de estimación	41
4.7.2.	Estimación inicial vs. real	41
4.8.	Estimación de costes	42
4.9.	Gestión de riesgos	42
5.	Desarrollo iterativo por sprints	45
5.1.	Requisitos globales del sistema	45
5.1.1.	Historias de usuario	45
5.1.2.	Casos de uso principales	45
5.1.3.	Burndown global del proyecto	46
5.2.	Sprint 1: Análisis, arquitectura y setup inicial	46
5.2.1.	Planificación del sprint	46
5.2.2.	Trabajo realizado	47
5.2.3.	Cómo se ha hecho	47
5.2.4.	Retrospectiva	51
5.3.	Sprint 2: <i>Backend</i> principal, datasets y API REST	51

ÍNDICE GENERAL

5.3.1.	Planificación del sprint	51
5.3.2.	Trabajo realizado	52
5.3.3.	Cómo se ha hecho	52
5.3.4.	Retrospectiva	55
5.4.	Sprint 3: Motor de métricas de calidad	57
5.4.1.	Planificación del sprint	57
5.4.2.	Trabajo realizado	57
5.4.3.	Cómo se ha hecho	57
5.4.4.	Retrospectiva	60
5.5.	Sprint 4: <i>Frontend</i> e interfaz de usuario	61
5.5.1.	Planificación del sprint	61
5.5.2.	Trabajo realizado	61
5.5.3.	Cómo se ha hecho	62
5.5.4.	Retrospectiva	62
5.6.	Sprint 5: Funcionalidades avanzadas	67
5.6.1.	Planificación del sprint	67
5.6.2.	Trabajo realizado	68
5.6.3.	Cómo se ha hecho	68
5.6.4.	Retrospectiva	70
5.7.	Sprint 6: Pruebas, refinamiento y documentación	71
5.7.1.	Planificación del sprint	71
5.7.2.	Trabajo realizado	72
5.7.3.	Cómo se ha hecho	72
5.7.4.	Retrospectiva	74
6.	Resultados y discusión	75
6.1.	Resultados alcanzados	75
6.1.1.	Resultados cuantitativos	75
6.1.2.	Esfuerzo y desviaciones	76
6.1.3.	Verificación del cumplimiento de objetivos	76
6.2.	Fortalezas de la plataforma	77

ÍNDICE GENERAL

6.3. Limitaciones	78
6.4. Comparación con herramientas existentes	79
7. Conclusiones y trabajo futuro	81
7.1. Revisión de objetivos	81
7.2. Conclusiones	82
7.3. Lecciones aprendidas	83
7.4. Líneas de trabajo futuro	84
7.5. Aportación del proyecto	85
7.6. Competencias adquiridas	86
7.7. Valoración personal	87
A. Especificación de la API RESTful	89
A.1. Módulo de autenticación (/api/auth)	89
A.2. Módulo de proyectos (/api/projects)	89
A.3. Módulo de conjuntos de datos (/api/datasets)	90
A.4. Módulo de métricas (/api/metrics)	90
A.5. Módulo de evaluaciones (/api/evaluations)	90
A.6. Módulo de administración (/api/admin)	91
B. Catálogo de métricas de calidad	92
B.1. Completitud (completeness)	93
B.2. Unicidad (uniqueness)	93
B.3. Exactitud sintáctica (syntactic_accuracy)	94
B.4. Consistencia lógica (logical_consistency)	95
B.5. Equilibrio de clases (class_balance)	96
B.6. Actualidad (currentness)	97
B.7. Diversidad (diversity)	97
B.8. Valores atípicos (outliers)	98
B.9. Modelo de severidad dinámica	98
C. Backlog completo del proyecto	100

D. Análisis retrospectivo de riesgos	103
E. Estimación de costes y viabilidad económica	105
F. Plan de pruebas integral	107
F.1. Objetivos y alcance	107
F.2. Estrategia y niveles de prueba	108
F.3. Tipos de pruebas aplicadas	108
F.4. Suites de prueba (resumen por capa)	108
F.4.1. Capa de <i>backend</i> (Flask + Celery)	109
F.4.2. Capa de dominio (<i>Sonar-Lite</i>)	109
F.4.3. Capa de <i>frontend</i> (Next.js)	109
F.4.4. Pruebas no funcionales	109
F.5. Entornos y datos de prueba	109
F.6. Criterios de entrada, salida y aceptación	109
F.7. Métricas y KPI! del plan	112
F.8. Riesgos del plan de pruebas	112
F.9. Roles y responsabilidades	113
F.10. Trazabilidad y herramientas	113
F.11. Estado actual y trabajo futuro	113
G. Manual de usuario	114
H. Diagramas UML del sistema	124
Referencias	128

Índice de cuadros

2.1. Correspondencia entre las dimensiones de calidad de International Organization for Standardization (ISO)/IEC 25012 e ISO/IEC 5259-2 y las métricas implementadas en DATAQUAL	9
2.2. Cobertura de DATAQUAL respecto al modelo de calidad de ISO/IEC 5259-2	12
2.3. Comparativa de DATAQUAL con herramientas de calidad de datos existentes	18
4.1. Tecnologías del <i>frontend</i>	33
4.2. Tecnologías del <i>backend</i>	34
4.3. Tecnologías de base de datos y almacenamiento	35
4.4. Tecnologías de procesamiento asíncrono y orquestación	36
4.5. Backlog inicial — historias de usuario principales	40
4.6. Estimación inicial vs. real por sprint	41
4.7. Estimación de costes del proyecto	42
4.8. Plan de gestión de riesgos	43
5.1. Sprint 1: Historias de usuario y estimación	47
5.2. Servicios Docker de DATAQUAL	48
5.3. Protocolos de comunicación entre componentes	48
5.4. Sprint 2: Historias de usuario y estimación	52
5.5. Sprint 3: Historias de usuario y estimación	57
5.6. Sprint 4: Historias de usuario y estimación	61
5.7. Sprint 5: Historias de usuario y estimación	67
5.8. Inputs al <i>hash</i> por tipo de <i>issue</i>	69
5.9. Sprint 6: Actividades y estimación	71
6.1. Resultados cuantitativos de la implementación	75

ÍNDICE DE CUADROS

6.2. Esfuerzo estimado vs. real por sprint	76
6.3. Verificación del cumplimiento de los objetivos específicos	77
6.4. Comparación de DATAQUAL con herramientas existentes (perspectiva post- implementación)	80
B.1. Criterios de severidad dinámica por tipo de métrica.	99
C.1. Backlog Sprint 1	100
C.2. Backlog Sprint 2	100
C.3. Backlog Sprint 3	101
C.4. Backlog Sprint 4	101
C.5. Backlog Sprint 5	101
C.6. Backlog Sprint 6	102
D.1. Reconstrucción retrospectiva de riesgos con ocurrencia y acciones aplicadas	104
E.1. Horas reales de desarrollo por sprint y categoría	105
E.2. Desglose completo de costes del proyecto	106
E.3. Estimación de costes de producción en AWS (por mes)	106
F.1. Niveles de prueba y distribución prevista	108
F.2. Suites de <i>backend</i>	110
F.3. Suites de dominio <i>Sonar-Lite</i>	111
F.4. Suites de <i>frontend</i>	111
F.5. Suites no funcionales	112
F.6. Indicadores de calidad del proceso de pruebas	112

Índice de figuras

4.1. Roadmap del proyecto: hitos entregados por sprint	37
4.2. Diagrama de Gantt del proyecto (curso académico 2025–2026)	38
4.3. Distribución de esfuerzo por disciplinas a lo largo de los sprints	39
5.1. Burndown global del proyecto: <i>story points</i> restantes por sprint (línea ideal vs. progreso real)	46
5.2. Arquitectura lógica por capas de DATAQUAL en su estado final (cierre del Sprint 5), mostrando las dependencias <i>depends-on</i> (flecha discontinua de punta abierta). El directorio <code>services/metrics/</code> (★) constituye el núcleo extensible del motor de evaluación, alineado con el principio abierto–cerrado (Gamma et al., 1994)	49
5.3. Pestaña <i>Versions</i> de un <i>dataset</i> : lista cronológica de versiones con etiquetas de rama, estado de calidad y puntuación por versión	54
5.4. Canvas de árbol de versiones (<i>Visual version tree</i>): grafo interactivo con nodos coloreados por estado de <i>Quality Gate</i> y diferencial de puntuación entre versiones padre-hijo	55
5.5. Comparación entre versiones de un <i>dataset</i> : <i>issues</i> resueltos, nuevos y persistentes entre dos versiones seleccionadas, con detalle de descripción y columna afectada	56
5.6. Cuadro de mando principal de DATAQUAL: tarjetas resumen de proyectos, distribución de estados de <i>Quality Gate</i> y alertas de proyectos con problemas detectados	63
5.7. Historial de análisis de un proyecto: evolución temporal de la puntuación de calidad, tendencia por <i>dataset</i> y tabla de ejecuciones con estado de <i>Quality Gate</i>	64
5.8. Página de resultados de evaluación: puntuación de calidad global, resumen de métricas por dimensión, listado de <i>issues</i> con severidad y columna afectada, y cálculo detallado de la puntuación final	65

ÍNDICE DE FIGURAS

5.9. Módulo de exploración de datos (EDA): resumen estadístico, diagrama de caja (<i>boxplot</i>), distribuciones por columna, matriz de correlación Pearson/Spearman y diagrama de dispersión	66
5.10. <i>Quality Gate</i> de DATAQUAL: estado global con código de color (PASSED/WARNING/FAILED) desglose de condiciones configuradas y <i>badges</i> de trazabilidad de <i>issues</i> por <i>fingerprinting</i>	70
G.1. Pantalla unificada de autenticación con pestañas de inicio de sesión y registro	116
G.2. Asistente de creación de un nuevo proyecto en tres pasos	116
G.3. Asistente de carga de <i>dataset</i> CSV con marcado opcional de columnas sensibles (Personally Identifiable Information (PII))	117
G.4. Configurador de métricas con catálogo, parametrización detallada y plantillas reutilizables	118
G.5. Lanzamiento de un análisis con seguimiento del progreso asíncrono mediante <i>polling</i>	119
G.6. Detalle de un análisis: <i>Quality Score</i> , métricas e <i>issues</i> con <i>badges</i> de trazabilidad entre ejecuciones	120
G.7. Módulo de perfilado interactivo (Exploratory Data Analysis (EDA)): histogramas, diagramas de caja, correlaciones y detección de <i>outliers</i>	121
G.8. Configuración del <i>Quality Gate</i> con preajustes de rigor y umbrales manuales (PASSED, WARNING, FAILED)	122
H.1. Diagrama Unified Modeling Language (UML) de casos de uso de DATAQUAL	125

Listado de acrónimos

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DMBOK	Data Management Body of Knowledge
EDA	Exploratory Data Analysis
IA	Inteligencia Artificial
IQR	Interquartile Range
IDOR	Insecure Direct Object Reference
ISO	International Organization for Standardization
JWT	JSON Web Token
ML	Machine Learning
MLOps	Machine Learning Operations
MVC	Model-View-Controller
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
PBKDF2	Password-Based Key Derivation Function 2
PII	Personally Identifiable Information
QA	Quality Assurance
REST	Representational State Transfer
RGPD	Reglamento General de Protección de Datos
SAAS	Software as a Service
SHA	Secure Hash Algorithm
SLA	Service Level Agreement
SODACL	Soda Checks Language
SQL	Structured Query Language

LISTADO DE ACRÓNIMOS

SQUARE	Systems and Software Quality Requirements and Evaluation
TFG	Trabajo Fin de Grado
UI	User Interface
UML	Unified Modeling Language
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

Capítulo 1

Introducción

Un modelo de IA es, en última instancia, tan fiable como los datos con los que fue entrenado. Esta dependencia, evidente en teoría, tiene consecuencias prácticas graves y documentadas: sistemas que discriminan, predicciones que fallan en producción, auditorías imposibles de realizar porque nadie registró el estado de los datos en el momento del entrenamiento. El presente Trabajo Fin de Grado (TFG) responde a esta realidad con el diseño e implementación de DATAQUAL, una plataforma web para la evaluación automatizada de la calidad de datos en proyectos de IA.

Este capítulo introductorio presenta el problema que motiva el desarrollo de la plataforma, describe la solución propuesta y sus capacidades principales, justifica su necesidad frente a los enfoques existentes, identifica el público al que se dirige y describe la estructura del documento.

1.1 Motivación

El principio conocido como *garbage in, garbage out* lleva décadas resumiendo una verdad incómoda para la industria del dato: la calidad de las salidas de un sistema está limitada por la calidad de sus entradas. En IA y Machine Learning (ML), esta dependencia es especialmente directa porque los modelos aprenden patrones de los propios datos de entrenamiento (Wang and Strong, 1996); si esos datos contienen errores, sesgos o valores ausentes, el modelo los hereda y, con frecuencia, los amplifica (Redman, 1998).

Los estudios más citados sobre el tema confirman que los problemas de calidad de datos superan a los errores algorítmicos como causa de fallos en producción. Sambasivan et al. (2021), a partir de entrevistas con 53 profesionales de IA en múltiples organizaciones, identificaron el etiquetado incorrecto, los datos desactualizados y las muestras no representativas como las causas predominantes de fracaso en despliegues de ML. Ng (2021) demostró que mejorar la calidad del dataset de entrenamiento produce ganancias de precisión que ningún ajuste arquitectónico puede igualar cuando los datos son defectuosos.

El coste económico es proporcional: Redman (2001) estimó que corregir un error se vuelve entre diez y cien veces más caro conforme avanza por las etapas de un proyecto. Encuestas recientes al sector cifran en torno al 60–80 % el tiempo que los profesionales de datos dedican

a limpieza y preparación (Kaggle, 2022), tiempo que no se invierte en construir ni mejorar modelos.

La calidad de los datos no es, además, únicamente una cuestión técnica. Los datos desequilibrados o no representativos son una fuente frecuente de sesgo con consecuencias éticas y sociales: el algoritmo de selección de personal de Amazon, descartado en 2018 por discriminar sistemáticamente a las candidatas femeninas (Dastin, 2018), y los sistemas de reconocimiento facial con tasas de error desproporcionadas para personas de piel oscura documentados por Buolamwini y Gebru (2018) ilustran cómo las deficiencias en los datos de entrenamiento se trasladan al comportamiento del modelo. El marco regulatorio refuerza esta dimensión: el Reglamento Europeo de IA (Parlamento Europeo y Consejo de la Unión Europea, 2024) establece requisitos explícitos de calidad, trazabilidad y documentación de los datos en sistemas de alto riesgo, y el Reglamento General de Protección de Datos (RGPD) (Parlamento Europeo y Consejo de la Unión Europea, 2016) exige que los datos personales sean exactos y estén actualizados, convirtiendo la evaluación formal de la calidad en una obligación legal, no solo una buena práctica.

Este trabajo nace del interés personal del autor por el tratamiento y la gestión de los datos como disciplina, y de su voluntad de profundizar en el ámbito de la inteligencia artificial y la ciencia de datos, campos en los que la calidad del dato de entrada resulta, precisamente, el factor más determinante.

1.2 Presentación de la solución

DATAQUAL es una plataforma web *full-stack* para la evaluación automatizada de la calidad de datos en proyectos de IA, concebida para que cualquier equipo pueda auditar sus datos sin necesidad de escribir código. Unifica en un mismo entorno la carga de datos, la configuración de métricas, la ejecución de evaluaciones y la consulta de resultados, y expone además una Application Programming Interface (API) Representational State Transfer (REST)ful (Fielding, 2000) para integrarse con herramientas externas. El despliegue se realiza mediante ocho servicios Docker orquestados con Docker Compose, lo que permite ejecutar la plataforma completa en cualquier máquina con un único comando y sin depender de servicios en la nube. Sus capacidades principales son:

- **Carga de conjuntos de datos** en formato Comma-Separated Values (CSV), con almacenamiento seguro en MinIO y versionado gestionado mediante una jerarquía de versiones con etiquetas personalizables.
- **Perfilado exploratorio del dataset**: análisis estadístico automático por columna que incluye distribuciones, valores nulos, tipos de datos, estadísticos descriptivos e histogramas, para facilitar la comprensión del dato antes de configurar las métricas formales.

- **Configuración de métricas de calidad** adaptadas al tipo de proyecto, con parámetros y umbrales personalizables alineados con los estándares ISO/IEC 25012 (ISO/IEC, 2008), ISO/IEC 25024 (ISO/IEC, 2015) e ISO/IEC 5259 (ISO/IEC, 2024).
- **Ejecución asíncrona de evaluaciones**, que analiza los conjuntos de datos frente a las métricas configuradas sin bloquear la interfaz de usuario.
- **Visualización interactiva de resultados** mediante cuadros de mando que incluyen gráficos de evolución, tablas de métricas por columna, histogramas, diagramas de caja y matrices de correlación.
- **Seguimiento de la evolución de la calidad** a lo largo del tiempo, gracias a un sistema de *fingerprinting* que permite comparar automáticamente los resultados con líneas base anteriores.
- **Quality Gates** (puertas de calidad): umbrales mínimos configurables que determinan si un conjunto de datos es apto para su uso, con estados PASSED, WARNING y FAILED, inspirados en el concepto homónimo de SonarQube (SonarSource, 2024).
- **Protección de datos sensibles**: el usuario marca las columnas que contienen información sensible al cargar el dataset, y la plataforma enmascara automáticamente sus valores en todos los resultados y muestras de la evaluación, evitando su exposición en los informes.
- **Interfaz multiidioma**: la plataforma soporta español e inglés, con cambio de idioma en tiempo real sin necesidad de recargar la aplicación.

1.3 Justificación de la solución

Evaluar la calidad de un dataset con *scripts* de Python o consultas Structured Query Language (SQL) ad hoc es viable para una exploración puntual, pero se vuelve insostenible cuando el número de proyectos crece o el equipo incorpora perfiles no técnicos. Cada equipo termina definiendo sus propias métricas con sus propios umbrales, sin posibilidad de comparar resultados entre proyectos ni de detectar si la calidad se degrada de una ejecución a la siguiente. DATAQUAL aborda ambas carencias con un catálogo de métricas alineado con ISO/IEC 25012 (ISO/IEC, 2008) e ISO/IEC 5259 (ISO/IEC, 2024) y un sistema de seguimiento entre ejecuciones basado en *fingerprinting*.

Herramientas más maduras como Great Expectations (Great Expectations, 2024) o Deequ de Amazon resuelven el problema de la estandarización, pero exigen que quien las configure sepa programar, lo que excluye a analistas y responsables de negocio. DATAQUAL invierte esta restricción: la interfaz web permite configurar y lanzar evaluaciones sin instalar nada ni escribir una línea de código, mientras que la API REST queda disponible para los perfiles técnicos que prefieran integrar las evaluaciones en sus propios flujos.

El tercer problema, compartido por todos los enfoques anteriores, es la ausencia de un veredicto claro: un *script* puede detectar que el 12 % de una columna es nulo, pero no indica si eso es aceptable para el proyecto concreto. DATAQUAL incorpora *Quality Gates* configurables (puntuación mínima, límite de issues críticos y límite de issues nuevos) que producen un estado PASSED, WARNING o FAILED, al modo en que SonarQube (SonarSource, 2024) lo hace para el código fuente.

1.4 Público objetivo

DATAQUAL está pensada para equipos multidisciplinares en los que conviven perfiles con distinto nivel técnico. Los ingenieros de datos pueden invocar las evaluaciones desde sus propios *scripts* o flujos mediante la API REST; los científicos de datos y los ingenieros de ML disponen de un mecanismo sistemático para verificar la calidad del dataset antes de arrancar el entrenamiento; y los analistas sin perfil técnico pueden explorar los resultados directamente desde el navegador sin instalar nada. Los responsables de calidad y cumplimiento normativo encuentran en los informes de evaluación evidencias auditables que facilitan la justificación ante los requisitos del RGPD (Parlamento Europeo y Consejo de la Unión Europea, 2016) y el Reglamento Europeo de IA (Parlamento Europeo y Consejo de la Unión Europea, 2024).

En todos los casos, la plataforma expone la misma información subyacente a través de dos canales complementarios: la interfaz web para exploración visual y la API REST para automatización, sin que el usuario deba elegir entre accesibilidad y capacidad de integración.

1.5 Alcance del proyecto

DATAQUAL es una plataforma *full-stack* orientada a proyectos de IA y ML, con soporte para conjuntos de datos tabulares en formato CSV. Cubre el ciclo completo de evaluación: carga y versionado de datasets, perfilado exploratorio, ejecución de siete métricas de calidad alineadas con ISO/IEC 25012 (ISO/IEC, 2008), ISO/IEC 25024 (ISO/IEC, 2015) e ISO/IEC 5259 (ISO/IEC, 2024), seguimiento de incidencias entre evaluaciones, *Quality Gates* con veredicto automatizado y protección de datos personales. El sistema se despliega en entorno local mediante Docker Compose y expone una API REST con autenticación JSON Web Token (JWT) para su integración en flujos de trabajo externos.

Quedan **fuera del alcance** del presente trabajo: el procesamiento de conjuntos de datos que excedan la memoria disponible en la máquina de despliegue, la corrección automática de errores y la integración nativa con plataformas *cloud* propietarias (AWS, Azure, GCP).

1.6 Estructura del documento

El presente documento se organiza en siete capítulos y siete anexos. A continuación se describe brevemente el contenido de cada uno de los capítulos restantes:

Capítulo 2: Antecedentes y estado del arte

Revisa los fundamentos teóricos de la calidad de datos, los principales estándares internacionales (ISO/IEC 25012, ISO/IEC 25024 e ISO/IEC 5259), las dimensiones de calidad relevantes y las herramientas existentes en el mercado, identificando las carencias que justifican el desarrollo de DATAQUAL.

Capítulo 3: Objetivos

Define el objetivo general y los objetivos específicos del proyecto, estableciendo el alcance detallado y las restricciones del trabajo.

Capítulo 4: Metodología y herramientas

Describe la metodología de gestión y desarrollo adoptada, la pila tecnológica, el backlog inicial, la planificación (Gantt), la estimación de esfuerzo, los costes y la gestión de riesgos.

Capítulo 5: Desarrollo iterativo por sprints

Desarrolla el proceso de implementación iterativo por sprints, incluyendo para cada sprint la planificación, el trabajo realizado, las decisiones técnicas adoptadas y la retrospectiva.

Capítulo 6: Resultados y discusión

Presenta los resultados globales obtenidos, analiza el cumplimiento de los objetivos planteados y compara DATAQUAL con las herramientas existentes.

Capítulo 7: Conclusiones y trabajo futuro

Recoge las conclusiones, la revisión de objetivos, la aportación del proyecto, las competencias adquiridas, la valoración personal y las líneas de trabajo futuro.

El documento incluye siete anexos: especificación completa de la API REST (Anexo A), catálogo de métricas de calidad (Anexo B), backlog completo (Anexo C), plan de riesgos detallado (Anexo D), estimación de costes y viabilidad (Anexo E), manual de usuario (Anexo F) y colección de diagramas UML (Anexo G).

Capítulo 2

Antecedentes y estado del arte

Saber que los datos condicionan la calidad de un modelo de IA no basta si no se dispone de una forma sistemática de medirlo. Durante las últimas décadas, la investigación ha producido dos tipos de respuesta: marcos conceptuales y estándares internacionales que definen qué es la calidad de datos, y herramientas que permiten medirla de forma automatizada. Este capítulo revisa ambos frentes para situar DATAQUAL en su contexto: qué se sabe sobre la calidad de datos en IA, qué normas la regulan, qué herramientas ya existen y por qué ninguna cubre exactamente el espacio que DATAQUAL ocupa.

2.1 La importancia de la calidad de datos en proyectos de IA

2.1.1 Marcos conceptuales fundacionales

Los trabajos seminales de Wang y Strong (1996) establecieron un marco conceptual para la calidad de datos basado en cuatro categorías fundamentales:

- **Calidad intrínseca:** precisión, objetividad, credibilidad y reputación de los datos.
- **Calidad contextual:** relevancia, valor añadido, completitud, cantidad apropiada y oportunidad temporal de los datos respecto a la tarea en la que se utilizan.
- **Calidad representacional:** interpretabilidad, facilidad de comprensión, representación concisa y representación consistente.
- **Accesibilidad:** disponibilidad de los datos y seguridad de acceso a los mismos.

El estándar ISO/IEC 25012 recoge el vocabulario conceptual de este marco (completitud, exactitud, consistencia y accesibilidad) rearticulándolo en un modelo orientado a sistemas informáticos que distingue entre características inherentes al dato y características dependientes del entorno tecnológico que lo aloja.

2.1.2 Impacto económico de la mala calidad de datos

Redman (1998) estimó que los errores en datos consumen entre el 8 % y el 12 % de los ingresos de una organización, pudiendo alcanzar entre el 40 % y el 60 % de los gastos operativos en empresas de servicios; investigaciones posteriores del mismo autor (2001) elevaron ese rango hasta el 25 % en sectores con mayor dependencia de datos. Estudios

posteriores del MIT Information Quality Program y del Data Management Body of Knowledge (DMBOK) (DAMA International, 2017) han reproducido estas cifras con resultados similares.

En proyectos de IA, el coste se multiplica: un modelo entrenado con datos defectuosos no solo produce resultados incorrectos, sino que las decisiones tomadas a partir de esas predicciones pueden tener consecuencias financieras, legales y reputacionales difíciles de revertir. Sambasivan et al. (2021) demostraron, en un estudio con 53 equipos de IA de países en desarrollo realizado en Google Research, que el comportamiento de los sistemas de IA está determinado críticamente por los datos, en mayor medida que por el código. Su contribución teórica central es el concepto de *Data Cascades* (cascadas de datos): eventos compuestos originados por problemas de calidad que se propagan y amplifican a lo largo del *pipeline*, causando fallos en etapas posteriores difíciles de trazar hasta su origen. El estudio reveló que el 92 % de los profesionales encuestados había experimentado al menos una cascada de datos. Su trabajo, titulado «Everyone wants to do the model work, not the data work», evidenció la paradoja central del campo: los datos son el factor más determinante del rendimiento y, al mismo tiempo, el aspecto menos valorado del proceso.

Según el *Data Science Report* de CrowdFlower (CrowdFlower, 2016), los científicos de datos dedicaban en 2016 hasta el 60 % de su tiempo a limpiar y organizar datos y un 19 % adicional a recopilarlos, sumando cerca del 80 % en preparación. Encuestas más recientes (Anaconda, Inc., 2022) muestran que ese porcentaje ha descendido hasta aproximadamente el 38 %, pero la preparación y limpieza sigue siendo con diferencia la tarea que más tiempo consume, por encima de la visualización (13 %) y el entrenamiento de modelos (9 %). La tendencia revela un problema estructural que, aunque ha mejorado, sigue siendo el principal cuello de botella del trabajo con datos y refuerza la necesidad de herramientas automatizadas de diagnóstico de calidad.

2.1.3 El paradigma data-centric AI

Ng (2021) acuñó el término *data-centric AI* para describir un giro metodológico respecto al enfoque tradicional *model-centric*: en lugar de mantener los datos fijos y mejorar el algoritmo, se mantiene el modelo fijo y se mejora sistemáticamente el dataset. La premisa es contundente: en la mayoría de proyectos reales de IA, invertir esfuerzo en los datos produce ganancias de rendimiento mayores que afinar la arquitectura del modelo.

Diversas iniciativas han consolidado este paradigma en la comunidad:

- La *Data-Centric AI Competition*, organizada por Andrew Ng y Landing AI, en la que los participantes compiten mejorando un dataset fijo en lugar de un modelo fijo.
- *DataPerf*, una iniciativa de MLCommons que proporciona *benchmarks* centrados en la calidad de datos para sistemas de ML.

- El área de investigación *Data Quality for AI*, que explora métricas de calidad específicas para conjuntos de datos destinados al entrenamiento de modelos.

El paradigma data-centric cobra especial relevancia cuando se considera que los propios datos pueden ser fuente de sesgo sistemático. Los datos desequilibrados o no representativos son la raíz de muchos casos documentados de sesgo en sistemas de IA: desde algoritmos de contratación que discriminan por género (Dastin, 2018) hasta sistemas de reconocimiento facial con tasas de error desproporcionadas para minorías étnicas (Buolamwini and Gebru, 2018). El equilibrio de clases y la representatividad de los datos son factores determinantes en la equidad de los sistemas inteligentes.

2.2 Marco normativo: estándares ISO

La evaluación de la calidad de datos se enmarca dentro de un conjunto de estándares internacionales publicados por la ISO. DATAQUAL toma como referencia conceptual los estándares ISO/IEC 25012 e ISO/IEC 25024, que definen respectivamente el modelo de calidad de datos y sus métricas de medición, y se alinea de forma explícita con ISO/IEC 5259, el estándar específico para calidad de datos en analítica e IA, cuya taxonomía de características estructura las decisiones de alcance del proyecto.

2.2.1 ISO/IEC 25012: Modelo de calidad de datos

El estándar ISO/IEC 25012 (ISO/IEC, 2008) forma parte de la familia Systems and Software Quality Requirements and Evaluation (SQUARE) y define un modelo de calidad de datos que comprende 15 características distribuidas en tres agrupaciones. La **calidad inherente** recoge cinco propiedades intrínsecas del dato (exactitud, completitud, consistencia, credibilidad y actualidad, *currentness*) evaluables con independencia del sistema que los almacena. La **calidad dependiente del sistema** comprende tres propiedades (disponibilidad, portabilidad y recuperabilidad) determinadas por el contexto tecnológico. Una tercera agrupación engloba siete características a la vez inherentes y dependientes del sistema (accesibilidad, conformidad, confidencialidad, eficiencia, precisión, trazabilidad y comprensibilidad), cuya evaluación requiere tanto el análisis del dato como el conocimiento del entorno que lo gestiona.

DATAQUAL implementa métricas que cubren las principales características inherentes del estándar, añadiendo además métricas específicas para proyectos de IA que no están contempladas en la norma. El Cuadro 2.1 muestra la correspondencia entre las características normativas y las métricas implementadas en DATAQUAL.

2.2.2 ISO/IEC 5259: Calidad de datos para analítica e IA

El estándar ISO/IEC 5259 (ISO/IEC, 2024) aborda específicamente la calidad de datos en contextos de analítica e inteligencia artificial y aprendizaje automático. La serie consta

Cuadro 2.1: Correspondencia entre las dimensiones de calidad de ISO/IEC 25012 e ISO/IEC 5259-2 y las métricas implementadas en DATAQUAL

Característica	Norma	Métrica DATAQUAL	Implementación
Complejidad	ISO 25012 + 25024	completeness	Ratio de valores no nulos por columna
Exactitud	ISO 25012 + 25024	syntactic_accuracy	Conformidad con patrones de formato
Consistencia ^a	ISO 5259-2	logical_consistency	Reglas IF-THEN entre columnas
Consistencia ^a	ISO 5259-2 + 25024	uniqueness	Detección de registros duplicados
Actualidad	ISO 5259-2	currentness	Antigüedad del dato más reciente
<i>Características adicionales para IA (ISO/IEC 5259-2)</i>			
Equilibrio (Balance)	ISO 5259-2	class_balance	Entropía de Shannon
Diversidad	ISO 5259-2	diversity	Cobertura de valores esperados por columna

^a La consistencia se operacionaliza en dos métricas: coherencia lógica entre columnas y ausencia de duplicados, siguiendo las sub-dimensiones Con-ML-4 y Con-ML-1 de ISO/IEC 5259-2 respectivamente.

de cinco partes, de las cuales la Parte 2 (medidas de calidad de datos) fue publicada en noviembre de 2024 (ISO/IEC, 2024):

1. **Parte 1. Visión general:** establece los conceptos fundamentales y el vocabulario común.
2. **Parte 2. Métricas de calidad de datos:** define métricas cuantitativas estandarizadas para evaluar la calidad de datos destinados a sistemas de IA.
3. **Parte 3. Gestión de datos:** cubre la gestión de conjuntos de datos, incluyendo versionado y trazabilidad.
4. **Parte 4. Marco de procesos:** describe el flujo de evaluación integrado en el ciclo de vida de un proyecto de IA.
5. **Parte 5. Verificación y validación:** establece los requisitos para verificar y validar los procesos y resultados de evaluación de calidad de datos.

DATAQUAL se alinea con este estándar en varios aspectos: proporciona métricas cuantitativas estandarizadas (Parte 2), implementa gestión de datasets con versionado (Parte 3) y ofrece un flujo de evaluación integrado en el ciclo de vida del proyecto (Parte 4).

Desde el punto de vista del alcance, la contribución más relevante de ISO/IEC 5259-2 es la organización de las características de calidad en cuatro grupos según su naturaleza y su dependencia del contexto tecnológico: *características inherentes*, *características mixtas* (inherentes y dependientes del sistema), *características dependientes del sistema* y *características adicionales* específicas para IA/ML. Esta taxonomía, analizada en detalle en la

Sección 2.3, constituye el marco conceptual sobre el que se delimita el alcance técnico de DATAQUAL.

2.2.3 ISO/IEC 25024: Medición de la calidad de datos

El estándar ISO/IEC 25024 (ISO/IEC, 2015) complementa al 25012 proporcionando métricas concretas y fórmulas de medición para cada característica de calidad. De este modo, se pasa de un modelo conceptual abstracto a un conjunto de indicadores cuantificables.

DATAQUAL implementa directamente tres de las métricas definidas en este estándar: el **ratio de completitud** (proporción de valores no nulos sobre el total de valores esperados en una columna o dataset), el **ratio de unicidad** (proporción de valores únicos respecto al total de registros, utilizada para detectar duplicados) y el **ratio de conformidad sintáctica** (proporción de valores que se ajustan a un patrón o formato predefinido mediante expresiones regulares, rangos numéricos o conjuntos de valores permitidos).

La adopción de estas fórmulas estandarizadas garantiza que los resultados de DATAQUAL sean comparables con los obtenidos por otras herramientas que implementen el mismo estándar, facilitando la comparabilidad entre herramientas y la auditoría de resultados.

El Reglamento Europeo de Inteligencia Artificial (Parlamento Europeo y Consejo de la Unión Europea, 2024) refuerza la relevancia del conjunto de estándares descrito en esta sección: exige que las organizaciones puedan demostrar la calidad y trazabilidad de los datos utilizados en sus modelos, lo que convierte la evaluación estandarizada de calidad de datos en una necesidad no solo técnica, sino también de cumplimiento normativo.

2.3 Alcance metodológico: selección y priorización de características en el Minimum Viable Product (MVP)

El marco normativo descrito en la sección anterior permite estructurar las decisiones de alcance de DATAQUAL en dos niveles. El primero distingue qué características son evaluables sobre el fichero de datos y cuáles requieren acceso a infraestructura o metadatos externos, criterio derivado directamente de la taxonomía de ISO/IEC 5259-2 (ISO/IEC, 2024). El segundo nivel responde a decisiones de priorización del MVP: algunas características técnicamente evaluables no fueron incluidas en esta versión por razones de alcance del proyecto, quedando como trabajo futuro. La presente sección detalla ambos criterios.

2.3.1 Taxonomía de características de calidad según ISO/IEC 5259-2

ISO/IEC 5259-2 organiza las características de calidad de datos para analítica e IA en cuatro grupos según su naturaleza y su grado de dependencia del contexto tecnológico:

- **Características inherentes (5):** exactitud (*accuracy*), completitud (*completeness*), consistencia (*consistency*), credibilidad (*credibility*) y actualidad (*currentness*). Son propiedades intrínsecas del dato, evaluables a partir del análisis de sus valores sin necesi-

dad de conocer el sistema que lo aloja.

- **Características mixtas (7):** accesibilidad (*accessibility*), conformidad (*compliance*), eficiencia (*efficiency*), precisión (*precision*), trazabilidad (*traceability*), confidencialidad (*confidentiality*) y comprensibilidad (*understandability*). Dependen tanto de las propiedades del dato como del contexto del sistema que lo gestiona.
- **Características dependientes del sistema (3):** disponibilidad (*availability*), portabilidad (*portability*) y recuperabilidad (*recoverability*). Vienen determinadas exclusivamente por la infraestructura tecnológica de la organización.
- **Características adicionales para IA/ML (9):** equilibrio (*balance*), diversidad (*diversity*), efectividad (*effectiveness*), identificabilidad (*identifiability*), relevancia (*relevance*), representatividad (*representativeness*), similitud (*similarity*), puntualidad (*timeliness*) y auditabilidad (*auditability*). Abordan necesidades específicas del aprendizaje automático que los marcos generalistas de calidad de datos no contemplan.

El Cuadro 2.2 sintetiza la cobertura de DATAQUAL respecto a las cuatro categorías anteriores.

2.3.2 Características evaluables algorítmicamente

Las características inherentes de ISO/IEC 5259-2 son evaluables algorítmicamente porque sus métricas se calculan a partir de los propios valores del dataset, sin necesidad de acceder a sistemas externos ni de conocer el dominio de negocio. La completitud se mide como la proporción de valores no nulos; la exactitud sintáctica, como la conformidad con patrones de formato definidos; la consistencia, como la ausencia de contradicciones lógicas entre columnas (sub-dimensión Con-ML-4 de ISO/IEC 5259-2) y como la ausencia de registros duplicados (sub-dimensión Con-ML-1); y la actualidad, como la antigüedad del valor más reciente en columnas temporales. Del grupo de características adicionales, el equilibrio de clases (*balance*) y la diversidad (*diversity*) son también evaluables algorítmicamente: el primero se cuantifica mediante la entropía de Shannon sobre los valores de la columna objetivo, y el segundo mide la cobertura de los valores y rangos esperados definidos por el usuario.

Esta propiedad de *auto-contención* es lo que hace posible desarrollar una herramienta de análisis y evaluación que opera exclusivamente sobre el fichero de datos sin requerir integración con los sistemas de la organización. El motor de métricas de DATAQUAL explota esta característica al procesar cada AnalysisRun de forma asíncrona y aislada, leyendo el dataset almacenado en MinIO y calculando las métricas configuradas sin acceso a los sistemas propios de la organización propietaria del dataset.

Cuadro 2.2: Cobertura de DATAQUAL respecto al modelo de calidad de ISO/IEC 5259-2

Categoría	Características	DATAQUAL	Motivo
Inherentes	Exactitud, Completitud, Consistencia, Actualidad	Implementadas	Extraíbles del dataset sin contexto externo
	Credibilidad	Fuera de alcance	No priorizada en el MVP; candidata para versiones futuras
Mixtas	Accesibilidad, Conformidad, Eficiencia, Precisión, Trazabilidad, Confidencialidad, Comprensibilidad	Fuera de alcance	Requieren información del sistema que aloja el dato
Dep. del sistema	Disponibilidad, Portabilidad, Recuperabilidad	Fuera de alcance	Determinadas por la infraestructura TI de la organización
Adicionales (impl.)	Equilibrio (<i>class_balance</i>), Diversidad (<i>diversity</i>)	Implementadas	Impacto directo en fiabilidad y cumplimiento normativo de modelos de IA
Adicionales (téc.)	Efectividad, Relevancia, Representatividad, Similitud	Fuera de alcance	Requieren metadatos organizativos externos al fichero de datos
Adicionales (futuro)	Identificabilidad, Auditabilidad, Puntualidad	Fuera de alcance	No priorizadas en el MVP; candidatas para versiones futuras

2.3.3 Características fuera del alcance del proyecto

Las características no implementadas en DATAQUAL responden a dos motivos distintos: imposibilidad técnica estructural y decisiones de priorización del MVP. Los tres primeros párrafos describen características técnicamente inviables sin acceso a infraestructura o metadatos externos; el último recoge las que quedaron fuera por decisión de alcance del proyecto.

Dependencia de infraestructura organizativa. Las características dependientes del sistema (disponibilidad, portabilidad y recuperabilidad) no describen propiedades del dato en sí, sino de la infraestructura que lo almacena y sirve. La *disponibilidad* mide si el dataset puede ser recuperado por usuarios autorizados dentro de un plazo acordado, lo que remite a acuerdos de nivel de servicio (Service Level Agreement (SLA)) y arquitecturas de almacenamiento. La *portabilidad* evalúa la capacidad de mover el dato entre sistemas preservando su calidad, lo que requiere acceso a los conectores y formatos de exportación propietarios de la organización. La *recuperabilidad* depende de las políticas de copia de seguridad y los mecanismos de restauración de datos. Ninguna de estas características puede evaluarse analizando el contenido de un fichero: exigen acceso al entorno operativo de la organización, del que una herramienta de análisis externa no dispone por definición.

Características mixtas con dependencia del sistema. Las siete características mixtas (accesibilidad, conformidad, eficiencia, precisión, trazabilidad, confidencialidad y comprensibilidad) dependen tanto de las propiedades del dato como del entorno tecnológico que lo gestiona. Su evaluación requiere información sobre el sistema que aloja el dato: los controles de acceso para la accesibilidad, los estándares aplicables para la conformidad, o las políticas de uso para la comprensibilidad. Sin acceso a esa capa de contexto organizativo, su medición no puede realizarse sobre el fichero de datos de forma aislada.

Dependencia de metadatos organizativos y contexto de negocio. Un subconjunto de las características adicionales requiere información que trasciende el fichero de datos. La *representatividad* necesita la definición de la población objetivo del modelo, que es un requisito de negocio no inscrito en el dato. La *relevancia* y la *efectividad* requieren un diccionario de datos y políticas de gobernanza privadas para determinar si el dataset es adecuado para la tarea prevista. La *similitud* presupone un conocimiento del espacio de características del dominio que no resulta accesible sin la colaboración directa de la organización propietaria.

Decisiones de priorización del MVP. Algunas características técnicamente evaluables no fueron incluidas en esta versión por razones de alcance del proyecto. La *credibilidad*

requiere la verificación de la procedencia de los datos, lo que, aunque factible, no se consideró prioritario para el caso de uso objetivo. La *identificabilidad*, la *auditabilidad* y la *puntualidad* son candidatas a implementarse en versiones futuras de DATAQUAL.

2.3.4 Priorización de características para el MVP

Entre las características algorítmicamente evaluables, el desarrollo del MVP de DATAQUAL ha priorizado aquellas con mayor impacto en la fiabilidad de los modelos de IA y en el cumplimiento del marco normativo aplicable:

1. **Las características inherentes de ISO/IEC 25012** (completitud, exactitud, consistencia y actualidad) implementadas a través de cinco métricas: completeness, syntactic_accuracy, logical_consistency, currentness y uniqueness, siendo esta última la sub-dimensión Con-ML-1 de consistencia definida en ISO/IEC 5259-2. Esto proporciona cobertura directa de los requisitos de ISO/IEC 5259-2 (ISO/IEC, 2024) e ISO/IEC 25024 (ISO/IEC, 2015).
2. **Equilibrio de clases** (class_balance), medido mediante la entropía de Shannon. El desequilibrio de clases es uno de los factores documentados de sesgo en sistemas de IA (Buolamwini and Gebru, 2018; Dastin, 2018) y su corrección tiene un impacto directo en la equidad y fiabilidad de los modelos de clasificación.
3. **Diversidad del dataset** (diversity): evalúa, de forma *opt-in* con expectativas definidas por el usuario, si el dataset cubre los valores y rangos esperados por columna, detectando valores ausentes, categorías sub-representadas e intervalos numéricos sin datos, lo que puede comprometer la capacidad de generalización de los modelos entrenados con ese conjunto (ISO/IEC, 2024).

Adicionalmente, los valores atípicos (*outliers*) se han incorporado como herramienta de diagnóstico en el módulo de EDA, sin ser computados como parte de la puntuación de calidad global. Esta decisión sigue explícitamente la recomendación de ISO/IEC 5259-2 (ISO/IEC, 2024), que no considera los *outliers* una dimensión de calidad inherente, sino un indicador diagnóstico cuya relevancia depende del dominio de aplicación.

El resultado es una delimitación deliberada del alcance del proyecto: DATAQUAL proporciona una evaluación algorítmica y estadística de las características de calidad priorizadas para proyectos de IA, aquellas medibles directamente sobre el fichero de datos con mayor impacto en la fiabilidad de los modelos. Las características que requieren infraestructura externa, metadatos organizativos o que no fueron priorizadas en el MVP quedan fuera de su ámbito actual, configurándose como líneas de trabajo futuro.

2.4 Estado del arte: herramientas existentes

Para contextualizar la contribución de DATAQUAL es necesario conocer qué existe ya. Las herramientas descritas a continuación representan el estado del arte en evaluación de calidad de datos; sus fortalezas y sus límites definen el espacio en el que DATAQUAL opera.

2.4.1 Great Expectations

Great Expectations (Great Expectations, 2024) es una biblioteca de código abierto escrita en Python para la definición y validación de «expectativas» (*expectations*) sobre conjuntos de datos. Las expectativas son aserciones declarativas que describen las propiedades que se esperan de los datos, como «esta columna no debe contener valores nulos» o «los valores de esta columna deben estar comprendidos entre 0 y 100».

Fortalezas. Great Expectations cuenta con un ecosistema maduro y una comunidad activa. Soporta múltiples *backends* de ejecución (Pandas, Spark, SQL), lo que permite evaluar datasets de distintos tamaños y procedencias. Además, genera automáticamente documentación visual de los resultados de validación (*Data Docs*).

Limitaciones. Se trata de una herramienta *code-first* que requiere conocimientos de Python para su configuración y uso. La versión de código abierto no dispone de interfaz gráfica integrada; GX Cloud ofrece una interfaz web gestionada pero como servicio externo (*SaaS*), sin posibilidad de autoalojamiento. No ofrece Quality Gates nativos con estado global configurable, no implementa un sistema de rastreo de issues entre ejecuciones sucesivas y presenta una curva de aprendizaje pronunciada para equipos sin experiencia en programación.

2.4.2 Apache Deequ

Apache Deequ (Schelter et al., 2018) es una biblioteca de verificación de calidad de datos desarrollada por Amazon y construida sobre Apache Spark. Está diseñada para operar a gran escala sobre datasets masivos en entornos distribuidos.

Fortalezas. Su principal ventaja es la escalabilidad masiva que hereda de Spark, lo que permite procesar datasets de millones de registros de forma distribuida. Ofrece integración nativa con los servicios de Amazon Web Services, verificación declarativa de restricciones y detección de anomalías basada en series temporales.

Limitaciones. Requiere un entorno JVM y un clúster de Spark para su ejecución, lo que supone una barrera de entrada considerable. No dispone de interfaz web, resulta desproporcionado para datasets de tamaño medio y no implementa métricas específicas para proyectos de IA como el equilibrio de clases.

2.4.3 DQOps

DQOps (DQOps, 2024) es una herramienta de calidad de datos de código abierto inspirada en la filosofía de SonarQube (SonarSource, 2024). Proporciona un enfoque sistemático a la calidad de datos con monitorización continua y alertas automatizadas.

Fortalezas. DQOps comparte una filosofía similar a la de DATAQUAL en cuanto al tratamiento de la calidad de datos como un proceso continuo. Soporta múltiples fuentes de datos (BigQuery, Snowflake, PostgreSQL), ofrece una interfaz web para la configuración y visualización de resultados, y permite definir alertas automatizadas ante degradaciones de calidad.

Limitaciones. Su configuración es compleja dado el elevado número de checks disponibles. Aunque incorpora soporte para ficheros planos (CSV, Parquet) a través de DuckDB, su enfoque principal es la monitorización continua de fuentes de datos conectadas. No ha sido diseñado específicamente para proyectos de IA y carece de métricas específicas como el equilibrio de clases o la diversidad de los datos de entrenamiento.

2.4.4 Soda Core

Soda Core (Soda Data, Inc., 2024) es un *framework* de código abierto (licencia Apache 2.0) para la validación y monitorización continua de la calidad de datos. Los controles de calidad se expresan en Soda Checks Language (SODACL), un lenguaje declarativo basado en YAML Ain't Markup Language (YAML) de sintaxis legible sin necesidad de programar en Python. Cuenta con respaldo comercial de Soda Data, Inc. y se integra de forma nativa con orquestadores habituales como Apache Airflow y dbt.

Fortalezas. Soda Core soporta un amplio abanico de fuentes de datos: bases de datos relacionales (PostgreSQL, BigQuery, Snowflake) y ficheros tabulares mediante integración con pandas (CSV, Parquet). Su lenguaje SODACL reduce la barrera técnica respecto a herramientas *code-first*. La versión comercial Soda Cloud añade una interfaz web de visualización, comparación histórica de resultados y *Quality Gates* con notificaciones automatizadas.

Limitaciones. La interfaz web está disponible únicamente en el nivel comercial (Soda Cloud), por lo que el uso puramente *open source* se limita a la línea de comandos. Soda Core carece de métricas específicas para IA (equilibrio de clases, detección de valores atípicos orientada a modelos) y no implementa *fingerprinting* de issues entre ejecuciones ni ocultación nativa de datos sensibles (PII).

2.4.5 Otras herramientas relevantes

Herramientas comerciales. Existen soluciones comerciales orientadas a la preparación de datos, como Alteryx (Alteryx, Inc., 2024) y Talend (Qlik Technologies, Inc., 2024). Estas herramientas ofrecen interfaces gráficas pulidas, capacidades de transformación (*data wrangling*) y soporte empresarial, pero presentan limitaciones significativas para el caso de uso de DATAQUAL: coste de licenciamiento elevado, dependencia del proveedor (*vendor lock-in*) y operación mayoritaria como Software as a Service (SAAS), lo que impide que la organización controle dónde residen sus datos. Además, su foco es la transformación de datos, no la evaluación de calidad para proyectos de IA.

Herramientas orientadas a ML. Evidently AI (Evidently AI, 2024) es una biblioteca *open source* específicamente diseñada para la monitorización de datos y modelos de ML: detecta *data drift*, evalúa el equilibrio de clases y genera informes visuales en HTML. Su enfoque está orientado a la comparación distribucional entre conjuntos de referencia y producción, más que a la auditoría estática de un fichero puntual. Pandera (Union.ai and Pandera Contributors, 2024) ofrece validación esquemática de *DataFrames* de pandas mediante anotaciones de tipo, útil en la etapa de pruebas del *pipeline* pero sin interfaz gráfica ni historial de ejecuciones. ydata-profiling (ydata-ai, 2024) genera informes exploratorios exhaustivos en HTML a partir de un fichero de datos, con estadísticas descriptivas, correlaciones y alertas básicas de calidad, pero sin motor de evaluación configurable ni Quality Gates.

2.4.6 Análisis comparativo

El Cuadro 2.3 presenta una comparación sistemática de DATAQUAL con las cuatro herramientas de código abierto analizadas. Se han seleccionado dieciséis características que cubren la funcionalidad, la accesibilidad, las capacidades específicas para IA y los aspectos operativos.

Como se puede observar, DATAQUAL ocupa un nicho diferenciado al combinar una interfaz web completa, de uso libre, con métricas específicas para IA, *fingerprinting* de issues, versionado de datasets e interfaz multiidioma (español e inglés), un conjunto de capacidades que ninguna de las herramientas analizadas ofrece en su totalidad. La cifra de métricas predefinidas no es directamente comparable entre herramientas: Great Expectations y DQOps contabilizan aserciones atómicas individuales (una por cada regla o patrón sobre una columna concreta), mientras que DATAQUAL implementa siete dimensiones de calidad de alto nivel alineadas con ISO/IEC 5259-2, cada una configurable para evaluar cualquier número de columnas y reglas. Este enfoque basado en dimensiones normalizadas produce resultados comparables entre proyectos y equipos; las aserciones atómicas son, por el contrario, específicas de cada dataset y requieren definición manual por parte del desarrollador. Además, Great Expectations presenta una naturaleza *code-first* que limita su accesibilidad a usuarios

Cuadro 2.3: Comparativa de DATAQUAL con herramientas de calidad de datos existentes

Característica	DATAQUAL	Gr. Exp.	Deequ	DQOps	Soda
Interfaz web	Sí	Parcial ^b	No	Sí	Parcial ^a
Interfaz multiidioma	Sí	—	—	No	No
Requiere programación	No	Sí	Sí	Parcial	No
Métricas predefinidas	7	300+	20+	260+	30+
Detección automática de tipos	Sí	No	No	Parcial	Parcial
Quality Gates	Sí	No	No	Sí	Sí
Fingerprinting de issues	Sí	No	No	No	No
Comparación entre ejecuciones	Sí	No	Sí	Sí	Sí
Equilibrio de clases	Sí	No	No	No	No
Detección de outliers	Sí	Parcial	Sí	Parcial	No
Enmascaramiento de PII	Sí	No	No	No	No
Versionado de datasets	Sí	No	No	No	No
Procesamiento asíncrono	Sí	No	Sí	Sí	Sí
Autoalojable	Sí	Sí	Sí	Sí	Sí
Específico para IA/ML	Sí	No	No	No	No
Coste	Libre	Freemium	Libre	Freemium	Freemium

^a La interfaz web de Soda (Soda Cloud) está disponible únicamente en el nivel comercial. ^b GX Cloud proporciona una interfaz web gestionada (*SaaS*) con nivel gratuito para desarrolladores y planes de pago para equipos.

no técnicos. Apache Deequ ofrece escalabilidad masiva, pero a costa de una complejidad operativa que lo hace inviable para conjuntos de datos de tamaño moderado. DQOps y Soda Core comparten la filosofía de calidad continua y disponen de *Quality Gates*, pero ambos orientan su análisis principalmente a fuentes de datos conectadas y carecen de métricas específicas para IA. Soda Core relega además su interfaz web al nivel comercial de pago, mientras que DQOps sí ofrece una interfaz web gratuita en local, aunque sin las capacidades de colaboración en equipo de su versión en la nube.

2.5 Diferenciación de DATAQUAL

El análisis anterior revela un espacio concreto que ninguna herramienta cubre en su totalidad. DATAQUAL lo ocupa a través de ocho capacidades que, tomadas en conjunto, no tienen equivalente directo en el ecosistema *open source*:

1. **Solución *full-stack* con interfaz integrada y gratuita.** Mientras que Great Expectations y Apache Deequ son herramientas *code-only* y Soda Core reserva su interfaz web al nivel comercial de pago, DATAQUAL proporciona una interfaz web completa construida con React (Meta Platforms, Inc., 2024) y Next.js (Vercel, 2024) disponible sin coste de licencia y autoalojable, que permite a usuarios no técnicos configurar métricas, lanzar evaluaciones y consultar resultados sin escribir código.
2. **Diseño centrado en IA.** DATAQUAL incorpora métricas específicas para proyectos de aprendizaje automático que no se encuentran en las herramientas generalistas: equili-

brio de clases (`class_balance`, entropía de Shannon) y diversidad del dataset (`diversity`), orientada a detectar subrepresentación de valores o rangos esperados por columna.

3. **Interfaz multiidioma.** La plataforma soporta español e inglés con cambio de idioma en tiempo real, lo que amplía su accesibilidad a equipos internacionales sin coste adicional de configuración.
4. **Sistema de fingerprinting SHA-256.** Inspirado en SonarQube (SonarSource, 2024), DATAQUAL genera un hash Secure Hash Algorithm (SHA)-256 para cada issue detectado, lo que permite rastrear su ciclo de vida a lo largo de múltiples ejecuciones: issues nuevos, recurrentes y corregidos.
5. **Quality Gates como puntos de decisión.** Cada evaluación produce un veredicto global (PASSED, WARNING o FAILED) basado en umbrales configurables, proporcionando una señal inequívoca sobre la aptitud del dataset para su uso.
6. **Protección de PII integrada.** La ocultación de los valores de columnas sensibles en todos los outputs de la plataforma es una funcionalidad nativa, no un complemento externo. Esto facilita el cumplimiento del RGPD y otras normativas de protección de datos.
7. **Arquitectura de plugins extensible.** El motor de métricas se basa en una clase abstracta `BaseMetric` y un registro global (`METRIC_REGISTRY`) que permiten añadir nuevas métricas sin modificar el código existente, siguiendo el principio abierto-cerrado de los patrones de diseño (Gamma et al., 1994).
8. **Autoalojamiento con soberanía de datos.** DATAQUAL se despliega mediante Docker Compose (Docker, Inc., 2024) sin dependencias de servicios en la nube. Todos los datos permanecen en la infraestructura del usuario, facilitando la soberanía del dato y el cumplimiento normativo.

2.6 Justificación académica

DATAQUAL no es un ejercicio de una sola disciplina. Su desarrollo ha requerido conocimientos de varias áreas del Grado en Ingeniería Informática:

Arquitectura de software y sistemas distribuidos. El proyecto despliega ocho servicios orquestados con Docker Compose (Docker, Inc., 2024), sin dependencias de infraestructura en la nube. El *backend* estructura la lógica según el patrón Model-View-Controller (MVC) sobre Flask (Ronacher, 2024) y aplica dos patrones del catálogo clásico (Gamma et al., 1994): Strategy en la clase abstracta `BaseMetric` y Registry en `METRIC_REGISTRY`, ambos bajo el principio abierto-cerrado. Un *middleware* propio, `EvaluationWatchdog`, ejecuta un hilo de fondo que detecta evaluaciones bloqueadas y las retoma sin intervención manual.

Capa de datos y procesamiento asíncrono. La capa de persistencia usa PostgreSQL (The PostgreSQL Global Development Group, 2024) con campos JSONB para configuraciones flexibles, SQLAlchemy (Bayer, 2024) como Object-Relational Mapping (ORM) con migraciones versionadas en Alembic, y MinIO (MinIO, Inc., 2024) como almacén de objetos compatible con S3. Las evaluaciones se procesan de forma asíncrona con Celery (Celery Project, 2024), usando Redis (Redis Ltd., 2024) como *broker*; Flower monitoriza el estado de los *workers* en tiempo real. Cuando Celery no está disponible, HybridEvaluationService ejecuta la evaluación en un hilo local como *fallback*.

Frontend y comunicación cliente-servidor. La interfaz se construye con React (Meta Platforms, Inc., 2024), Next.js (Vercel, 2024), TypeScript (Microsoft, 2024) y Material User Interface (UI) (MUI, 2024), y se comunica con el *backend* a través de una API REST protegida con JWT. Las visualizaciones interactivas —gráficos de líneas, barras, *doughnut* y *scatter*— se implementan con Chart.js (Chart.js Contributors, 2024) mediante su adaptador para React. La internacionalización (español e inglés con cambio en tiempo real) se gestiona con react-i18next.

Estadística aplicada y análisis exploratorio. El motor de métricas usa la entropía de Shannon para medir el equilibrio de clases: produce un valor entre 0 y 100 donde 100 indica distribución uniforme y los valores próximos a 0 señalan desequilibrio severo. El módulo de EDA detecta valores atípicos con el Interquartile Range (IQR) y ofrece histogramas, diagramas de caja, gráficos de barras y matrices de correlación (Pearson y Spearman).

Seguridad y cumplimiento normativo. La plataforma usa autenticación JWT y derivación de contraseñas con Password-Based Key Derivation Function 2 (PBKDF2), y limita la tasa de peticiones con Flask-Limiter, respaldado por Redis en el despliegue con Docker Compose. La validación de entradas en los puntos críticos se realiza con esquemas Marshmallow; las reglas de consistencia lógica se filtran mediante una lista de tokens prohibidos para evitar la ejecución de código arbitrario. Las columnas marcadas como PII se enmascaran en todos los *outputs* de evaluación, lo que facilita el cumplimiento del RGPD.

Cada una de estas áreas ha dejado huella directa en la arquitectura: las decisiones de una capa condicionan las de las demás, lo que convierte el proyecto en un ejercicio de integración real, no solo de aplicación aislada de conocimientos.

Capítulo 3

Objetivos

El análisis del capítulo 2 permite delimitar los objetivos que guían el desarrollo de DATAQUAL: el objetivo general, cinco objetivos específicos que cubren las principales dimensiones funcionales, los objetivos transversales de calidad del software y el alcance del proyecto con sus limitaciones.

3.1 Objetivo general

El objetivo general de este TFG consiste en **diseñar, implementar y validar una plataforma web interactiva y una API RESTful que permitan evaluar de forma automatizada la calidad de los datos empleados en proyectos de IA**, proporcionando métricas cuantitativas, visualizaciones interactivas y mecanismos de diagnóstico que faciliten la identificación y corrección de problemas de calidad en los conjuntos de datos.

Este objetivo responde a la necesidad, identificada en la sección 1.1, de disponer de herramientas que superen las limitaciones de los enfoques ad hoc (*scripts* puntuales, consultas SQL manuales o inspecciones en cuadernos Jupyter) y que, al mismo tiempo, no impongan la barrera técnica que presentan las bibliotecas basadas exclusivamente en código (sección 2.4). DATAQUAL ofrece una solución *full-stack*, alineada con los estándares ISO/IEC 25012 (ISO/IEC, 2008) e ISO/IEC 5259 (ISO/IEC, 2024), que hace accesible la evaluación de calidad de datos sin necesidad de escribir código.

3.2 Objetivos específicos

El objetivo general se descompone en los siguientes cinco objetivos específicos (**OE1–OE5**), cada uno de los cuales aborda una dimensión funcional diferenciada de la plataforma.

3.2.1 OE1. Módulo de ingesta y almacenamiento

Antes de evaluar la calidad de un dataset, la plataforma debe ser capaz de recibirlo, interpretarlo y custodiarlo de forma fiable. Este requisito, aparentemente sencillo, esconde una dificultad práctica recurrente: los archivos CSV del mundo real presentan combinaciones heterogéneas de codificaciones, delimitadores y convenciones de formato que los analiza-

dores estándar no resuelven sin configuración explícita. Ignorar este problema implica que una fracción significativa de los datasets reales sería rechazada en la entrada, invalidando la utilidad de cualquier análisis posterior.

El objetivo consiste en desarrollar un módulo de ingesta y almacenamiento que resuelva este problema de forma transparente para el usuario, sin requerir conocimiento técnico sobre el formato del fichero. Los requisitos que lo concretan son los siguientes:

- **Soporte CSV.** La plataforma acepta conjuntos de datos en formato CSV, el formato tabular más extendido en proyectos de IA y ciencia de datos.
- **Parseo robusto.** El módulo implementa una estrategia de *fallback* que combina múltiples codificaciones (UTF-8, UTF-8-BOM, CP1252, Latin-1), separadores (coma, punto y coma, tabulador, *pipe*) y motores de lectura, garantizando la ingesta correcta del mayor número posible de archivos reales sin intervención del usuario.
- **Almacenamiento seguro.** Los archivos originales se almacenan en un sistema de objetos compatible con S3 (MinIO) para garantizar su integridad y disponibilidad; los metadatos (esquema, estadísticas descriptivas y conteos) se persisten en PostgreSQL para permitir consultas eficientes.
- **Extracción automática de esquema.** El sistema infiere automáticamente los nombres de columna, los tipos de dato y las estadísticas descriptivas básicas de cada dataset cargado, eliminando la necesidad de configuración manual previa a la evaluación.
- **Versionado de conjuntos de datos.** El módulo soporta la carga de nuevas versiones de un mismo dataset, manteniendo la trazabilidad entre versiones mediante una relación padre-hijo y un indicador de versión activa.

3.2.2 OE2. Motor de evaluación con métricas parametrizables

El núcleo funcional de DATAQUAL es su motor de evaluación. Las herramientas generalistas de calidad de datos ofrecen métricas orientadas a bases de datos transaccionales (completitud, unicidad, conformidad de formato), pero carecen de las dimensiones específicas que condicionan el rendimiento de un modelo de IA: el desequilibrio entre clases introduce sesgo sistemático, y la falta de representatividad de ciertos valores o rangos compromete la capacidad de generalización. Este objetivo cubre ambos frentes: las dimensiones estándar definidas por los organismos normativos y las dimensiones propias del contexto de ML.

El motor debe cubrir las dimensiones de calidad de ISO/IEC 25012 (ISO/IEC, 2008), aplicando las fórmulas de medición definidas en ISO/IEC 25024 (ISO/IEC, 2015), y añadir las métricas específicas para IA de ISO/IEC 5259 (ISO/IEC, 2024). Las **siete métricas evaluables** que contribuyen al *Quality Score* son:

1. **Completitud** (completeness): ratio de valores no nulos por columna, alineada con la característica de completitud de ISO/IEC 25012.

2. **Exactitud sintáctica** (*syntactic_accuracy*): conformidad de los valores con patrones de formato definidos mediante expresiones regulares, correspondiente a la dimensión de exactitud.
3. **Consistencia lógica** (*logical_consistency*): verificación de reglas condicionales entre columnas (reglas IF - THEN), alineada con la dimensión de consistencia.
4. **Actualidad** (*currentness*): antigüedad del dato más reciente respecto a un instante de referencia, correspondiente a la dimensión de actualidad (*currentness*).
5. **Unicidad** (*uniqueness*): detección de registros duplicados, implementando la sub-dimensión Con-ML-1 de consistencia definida en ISO/IEC 5259-2.
6. **Equilibrio de clases** (*class_balance*): evaluación del balance de la variable objetivo mediante la entropía de Shannon, métrica específica para proyectos de IA que permite detectar desequilibrios que sesgan los modelos (Ng, 2021).
7. **Diversidad del dataset** (*diversity*): evaluación, de forma *opt-in* con expectativas definidas por el usuario, de si el dataset cubre los valores y rangos esperados por columna, detectando subrepresentación que puede comprometer la capacidad de generalización de los modelos (ISO/IEC, 2024).

Adicionalmente, el motor debe incluir una clase de **detección de valores atípicos** (*outliers*) basada en el IQR (Tukey, 1977). Siguiendo la recomendación de ISO/IEC 5259 (ISO/IEC, 2024) de no tratar los *outliers* como una dimensión de calidad per se (su relevancia depende del dominio de aplicación), esta clase **no participará en el *Quality Score*** sino que se utilizará exclusivamente como herramienta de diagnóstico en el módulo de perfilado de datos (EDA).

El motor debe adoptar una arquitectura de *plugins*, de modo que cada métrica se implemente como una clase independiente que herede de una clase base abstracta y se registre dinámicamente en un registro centralizado. Esta arquitectura garantiza la extensibilidad del sistema, permitiendo incorporar nuevas métricas sin modificar el núcleo del motor.

Adicionalmente, cada métrica debe ser parametrizable: umbrales de aceptación, columnas objetivo, métodos de detección y pesos relativos (en una escala de 0,0 a 1,0) que determinen su contribución a la puntuación global de calidad. El sistema debe soportar también plantillas de métricas que permitan reutilizar configuraciones entre proyectos.

3.2.3 OE3. API RESTful con especificación formal

Una plataforma de evaluación de calidad de datos no debería ser un sistema cerrado: su valor aumenta cuando puede integrarse en *pipelines* de Machine Learning Operations (MLOPS), en procesos de integración continua o en herramientas externas que consuman sus resultados de forma programática. Para ello es necesario que la API esté bien estructurada y cuente con una especificación explícita que permita a terceros integrarla sin necesidad

de leer el código fuente.

El objetivo consiste en diseñar e implementar una API RESTful que exponga *endpoints* para la carga de datos, configuración de evaluaciones, ejecución de análisis y consulta de resultados. La especificación completa de rutas, parámetros y formatos de respuesta se recoge en el Anexo A; este objetivo define el alcance funcional que dicha especificación debe cubrir.

La API se organiza en seis módulos funcionales cohesivos, siguiendo el patrón *Blueprint* de Flask:

auth Registro de usuarios, inicio de sesión, refresco de *tokens* JWT y consulta del perfil del usuario autenticado.

projects

Operaciones Create, Read, Update, Delete (CRUD) sobre proyectos, incluyendo la asignación de conjuntos de datos y la configuración de métricas.

datasets

Carga de conjuntos de datos, consulta de metadatos, gestión de versiones y descarga de archivos.

metrics

Definición y configuración de métricas de calidad, gestión de plantillas y consulta del catálogo de métricas disponibles.

evaluations

Lanzamiento de evaluaciones, consulta de progreso, recuperación de resultados e *issues* detectados.

admin

Utilidades de administración del sistema, incluyendo un *endpoint* de comprobación de salud (*health check*).

Todos los *endpoints* protegidos requieren autenticación mediante JWT. El formato de respuesta es uniforme y predecible, con la estructura {success, data, message} y soporte de paginación en los listados. La convención REST de rutas anidadas añade *blueprints* técnicos adicionales (project_metrics, project_datasets, dashboard, patterns) que no amplían el alcance funcional pero sí el número total de *blueprints* registrados.

3.2.4 OE4. Aplicación web con *dashboards* interactivos

Una API potente pero sin interfaz gráfica reproduce exactamente la barrera de acceso que DATAQUAL pretende eliminar: limita el uso a perfiles con conocimientos de programación. La interfaz web no es un complemento estético del sistema sino una decisión de diseño central: permite que analistas de datos, gestores de proyectos o auditores de calidad operen

la plataforma sin escribir una sola línea de código, configurando métricas, interpretando resultados y tomando decisiones directamente desde el navegador.

El objetivo consiste en desarrollar una aplicación web que cubra el ciclo completo de trabajo sobre un dataset: gestión de proyectos y versiones, configuración de evaluaciones, visualización de resultados y exploración exploratoria de los datos. Las capacidades concretas que debe ofrecer son:

- **Cuadro de mando principal** con resumen de proyectos, conjuntos de datos y distribución de *Quality Gates*.
- **Página de detalle de proyecto** con pestañas diferenciadas para conjuntos de datos, configuración de métricas, historial de evaluaciones y estado del *Quality Gate*.
- **Visualización de resultados de evaluación** con indicadores de puntuación de calidad (*gauge*), pestañas por métrica y listado de *issues* detectados.
- **Exploración interactiva de datos** (EDA): histogramas, diagramas de caja, gráficos de barras, gráficos de dispersión, matrices de correlación (Pearson y Spearman) y detección visual de valores atípicos con factor IQR configurable.
- **Comparación entre versiones de un dataset**: página dedicada que muestra las diferencias en número de filas, columnas, puntuación de calidad, columnas añadidas o eliminadas e *issues* nuevos, resueltos y comunes entre dos versiones.
- **Canvas de linaje de versiones**: árbol visual interactivo que representa la genealogía completa de versiones de un dataset, permitiendo navegar por la historia de cambios.
- **Gráficos de tendencia** que muestren la evolución de la calidad a lo largo de las sucesivas evaluaciones.
- **Indicadores visuales de *Quality Gate*** con códigos de color: verde (PASSED), amarillo (WARNING) y rojo (FAILED).
- **Diseño adaptable** (*responsive*) con soporte para dispositivos de escritorio, tableta y teléfono móvil.

3.2.5 OE5. Mecanismos de identificación de problemas

Detectar que un dataset tiene un 15 % de valores nulos en una columna es útil; saber si ese porcentaje ha empeorado respecto a la semana pasada, si el problema es recurrente o si ya se corrigió una vez, es lo que permite tomar decisiones informadas. La mayoría de las herramientas de calidad de datos producen un informe por ejecución pero no conservan memoria entre ejecuciones: cada análisis es un snapshot aislado sin conexión con el anterior.

Este objetivo resuelve ese problema introduciendo tres mecanismos que, combinados, convierten la evaluación puntual en un proceso de seguimiento continuo:

1. **Sistema de *issues*.** Cada problema de calidad detectado debe registrarse como un *issue* con cuatro niveles de severidad (*critical, high, medium, low*), calculada dinámicamente en función de la distancia al umbral configurado. Cada *issue* debe incluir muestras de los datos afectados (hasta cinco filas de ejemplo), con ocultación automática de los valores de columnas marcadas como PII.
2. ***Fingerprinting* determinista.** Cada *issue* debe recibir un identificador único generado mediante SHA-256 a partir de sus atributos característicos (métrica, columna, tipo de problema), permitiendo su identificación unívoca entre ejecuciones independientes. Este mecanismo, inspirado en el sistema de rastreo de *issues* de SonarQube (SonarSource, 2024), permite clasificar automáticamente cada incidencia como *nueva, recurrente* o *corregida* respecto a la evaluación anterior.
3. ***Quality Gates*.** Umbrales mínimos configurables que determinan si un conjunto de datos es apto para su uso, con tres estados posibles: PASSED (score por encima del umbral y sin exceso de *issues* críticos ni nuevos), WARNING (score dentro del margen de advertencia configurable por encima del umbral mínimo, o número de *issues* nuevos superior al límite configurado) y FAILED (score por debajo del umbral mínimo o *issues* críticos por encima del máximo permitido). Las *Quality Gates* proporcionan una señal clara y automatizable que puede integrarse en *pipelines* de MLOPS (Kreuzberger et al., 2023) o de integración continua para bloquear el uso de conjuntos de datos de calidad insuficiente.

3.3 Objetivos transversales

Además de los objetivos funcionales descritos, el desarrollo de DATAQUAL persigue un conjunto de objetivos transversales de calidad del software que condicionan las decisiones arquitectónicas y tecnológicas del proyecto:

Seguridad

Autenticación basada en JWT con *tokens* de acceso y refresco, *hashing* de contraseñas mediante PBKDF2, limitación de tasa de peticiones (*rate limiting*), validación de entradas en los puntos críticos mediante esquemas Marshmallow, protección contra inyección de código en las reglas de consistencia lógica y enmascaramiento automático de columnas marcadas como PII.

Escalabilidad

Procesamiento asíncrono de evaluaciones mediante Celery y Redis como *broker* de mensajes, lo que permite escalar horizontalmente el número de *workers* en función de la carga de trabajo, así como separación de responsabilidades entre servicios.

Mantenibilidad

Arquitectura modular basada en *blueprints*, servicios y modelos, con un sistema de

métricas extensible por *plugins* que permite incorporar nuevas métricas con un esfuerzo mínimo. Migraciones de base de datos versionadas y separación estricta entre *frontend* y *backend*.

Usabilidad

Interfaz de usuario basada en Material Design con formularios dotados de validación en tiempo real, notificaciones *toast*, migas de pan (*breadcrumbs*) de navegación y estados de carga y error claramente diferenciados. La plataforma soporta español e inglés con cambio de idioma en tiempo real, sin necesidad de recargar la aplicación.

Portabilidad

Despliegue del *stack* completo mediante Docker Compose, con configuración por variables de entorno y sin dependencias de servicios en la nube propietarios. Los ocho servicios que componen la plataforma (*frontend*, *backend*, PostgreSQL, MinIO, Redis, Celery *worker*, Celery Beat y Flower) se orquestan conjuntamente, lo que garantiza la reproducibilidad del entorno y que los datos no salgan de la infraestructura propia.

3.4 Alcance y limitaciones

El alcance del proyecto abarca el diseño, la implementación y la validación de DATA-QUAL como plataforma *full-stack*, cubriendo los cinco objetivos específicos descritos en la sección 3.2: el módulo de ingesta con parseo robusto y versionado, el motor de evaluación con siete métricas parametrizables, la API RESTful con los módulos funcionales planificados, la aplicación web con cuadros de mando interactivos, y los mecanismos de identificación y seguimiento de problemas. El sistema se despliega íntegramente mediante Docker Compose, se valida con conjuntos de datos reales y se documenta en los anexos de la presente memoria (especificación de la API y catálogo de métricas). La pila tecnológica queda fijada por las decisiones de arquitectura adoptadas: Python con Flask y PostgreSQL en el *backend*, React con Next.js y TypeScript en el *frontend*, Celery con Redis para el procesamiento asíncrono, y alineación explícita con los estándares ISO/IEC 25012 (ISO/IEC, 2008) e ISO/IEC 5259 (ISO/IEC, 2024).

Fuera de este alcance quedan tres aspectos que, si bien son relevantes en un contexto de producción real, no forman parte del presente trabajo.

El primero es el **procesamiento de grandes volúmenes de datos**. El motor de evaluación carga el dataset completo en memoria mediante Pandas, lo que limita el tamaño práctico de los ficheros a lo que la memoria RAM disponible permita procesar sin degradación. La evaluación distribuida sobre datasets masivos mediante Apache Spark o Dask requiere una arquitectura diferente y queda como línea de trabajo futuro.

El segundo es la **corrección automática de datos**. La plataforma diagnostica y clasifica los problemas de calidad, pero la decisión y ejecución de las acciones correctoras (limpie-

za, imputación, deduplicación) corresponde al usuario. Incorporar sugerencias de corrección automatizadas es una de las líneas de trabajo futuro identificadas en el capítulo 7.

El tercero es la **integración directa con fuentes de datos externas**. La arquitectura de almacenamiento basada en MinIO es compatible con el protocolo S3, pero no se desarrollan conectores específicos para bases de datos (MySQL, BigQuery, Snowflake) ni para servicios de almacenamiento en la nube (AWS S3, Azure Blob, Google Cloud Storage). El usuario interactúa con la plataforma a través de ficheros CSV cargados manualmente.

Capítulo 4

Metodología y herramientas

Este capítulo presenta el marco metodológico completo que ha guiado el desarrollo de DATAQUAL. Se expone, en primer lugar, la metodología ágil de gestión del proyecto (Scrum adaptado a un entorno individual), junto con su justificación y adaptación práctica. A continuación se describe la metodología de desarrollo de software adoptada, el stack tecnológico seleccionado y el entorno de trabajo. El capítulo concluye con la planificación temporal del proyecto mediante un diagrama de Gantt, el backlog inicial con las historias de usuario, la estimación del esfuerzo, el análisis de costes y el plan de gestión de riesgos.

4.1 Gestión del proyecto

4.1.1 Metodología ágil adoptada: Scrum adaptado

Para la gestión del proyecto se adoptó **Scrum adaptado a un proyecto unipersonal** (Schwaber and Sutherland, 2020), una metodología ágil que organiza el trabajo en sprints de duración acotada y favorece la entrega incremental de valor. La elección se justifica por tres razones:

- **Adecuación al tipo de proyecto.** Los requisitos de DATAQUAL no estaban completamente definidos desde el inicio: a medida que se profundizaba en el dominio de la calidad de datos y los estándares ISO, surgían nuevas necesidades que había que incorporar al backlog. Scrum encaja bien con este tipo de proyectos exploratorios e iterativos.
- **Retroalimentación continua.** La estructura de sprints facilita la revisión periódica del incremento con los tutores, permitiendo ajustar prioridades y corregir la dirección del proyecto antes de que las desviaciones sean costosas.
- **Trazabilidad del avance.** Cada sprint produce un conjunto delimitado de entregables, lo que genera un histórico auditable del progreso y facilita la redacción de las retrospectivas incluidas en el Capítulo 5.

La adaptación concreta de Scrum a este proyecto se detalla en la subsección siguiente.

4.1.2 Adaptación de Scrum al TFG individual

Dado que Scrum está concebido para equipos de entre tres y nueve personas, su aplicación a un proyecto unipersonal requiere simplificar los roles y adaptar los eventos sin perder el núcleo del marco: sprints de duración acotada, backlog priorizado e inspección y adaptación continuas. Las adaptaciones adoptadas se detallan a continuación.

Sprints

El proyecto se divide en **6 sprints** de duración variable entre cuatro y ocho semanas, ajustada a la complejidad de cada incremento. La duración variable es una adaptación deliberada: los sprints iniciales y finales (S1, S2, S6) reciben más tiempo dada la carga de análisis y arquitectura (S1), implementación del núcleo del *backend* (S2) o documentación y validación (S6), mientras que los sprints centrales de alcance más acotado (S3–S5) se mantienen en un mes. La distribución temporal completa se recoge en la Sección 4.5.

Roles

Al ser un proyecto individual, el estudiante asume simultáneamente los tres roles de Scrum:

- *Product Owner*: define y prioriza el backlog en función de los objetivos del TFG y de la retroalimentación de los tutores.
- *Scrum Master*: vela por la aplicación coherente de la metodología y gestiona los bloqueos técnicos o de planificación.
- *Developer*: implementa las historias de usuario seleccionadas para cada sprint.

Sprint planning

Al inicio de cada sprint se define el objetivo, se seleccionan las historias de usuario del backlog y se descomponen en tareas concretas registradas en Trello con estimación en horas.

Daily standup

Sustituido por una **revisión personal diaria** informal: al comenzar cada sesión de trabajo se repasaba el estado de las tareas en curso y se decidía el foco de la jornada.

Sprint review

Al finalizar cada sprint (o cada dos en los meses de mayor densidad de trabajo) se acordaba una reunión con los tutores para revisar el incremento funcional y ajustar las prioridades del backlog. La convocatoria era bajo demanda, sin cadencia fija, para adaptarla al ritmo real del proyecto.

Sprint retrospective

Reflexión escrita al cierre de cada sprint siguiendo un guión estructurado: logros del sprint, estado acumulado del producto, incidencias detectadas, estimación frente a

tiempo real y acción de mejora para el sprint siguiente. Las retrospectivas completas se recogen en el Capítulo 5.

Esta adaptación conserva los elementos esenciales de Scrum (iteraciones cortas, entregables funcionales y mejora continua) mientras prescinde de los eventos que carecen de sentido en un contexto unipersonal, como el *daily standup* en grupo o la *sprint review* formal ante el equipo.

4.1.3 Herramientas de gestión

La gestión del proyecto se apoyó en dos herramientas complementarias: Trello para el seguimiento ágil del backlog y Git + GitHub para el control de versiones del código fuente.

Trello

Herramienta principal de gestión del backlog y seguimiento de sprints. Cada historia de usuario y tarea técnica se representa como una tarjeta con prioridad, estimación en horas y estado. El tablero Kanban organiza las tarjetas en cinco columnas (*Backlog*, *Sprint actual*, *En progreso*, *En revisión / Iterable*, *Terminado*), lo que permite visualizar el flujo de trabajo tanto dentro de cada sprint como la progresión entre sprints.

Git + GitHub

Sistema de control de versiones distribuido con repositorio remoto en GitHub. El desarrollo se realizó principalmente sobre la rama *main*, creando ramas de funcionalidad (*feature branches*) únicamente para las tareas de mayor envergadura o riesgo. Los mensajes de *commit* siguen una convención descriptiva que facilita la trazabilidad de los cambios y la redacción de las retrospectivas.

Ambas herramientas son de acceso libre y gratuito, lo que se refleja en la estimación de costes de la Sección 4.8.

4.2 Metodología de desarrollo

El desarrollo de DATAQUAL ha seguido una **metodología iterativa e incremental**, donde cada sprint produce un incremento funcional desplegable que amplía las capacidades de la plataforma. Los principios que han guiado todo el proceso (entrega frecuente, retroalimentación continua y adaptación del plan ante nuevos requisitos) son coherentes con los valores del *Agile Manifesto* (Beck et al., 2001).

4.2.1 Desarrollo iterativo

El proyecto se ha organizado en **iteraciones** de duración variable (entre cuatro y ocho semanas), cada una de las cuales ha producido un incremento funcional desplegable. Cada iteración ha comprendido las siguientes actividades:

1. **Planificación:** selección de las funcionalidades a implementar en la iteración, priori-

zando aquellas que aportan mayor valor o que desbloquean funcionalidades posteriores.

2. **Diseño:** definición de la arquitectura y el modelo de datos necesarios para las funcionalidades seleccionadas, tomando como referencia los estándares ISO/IEC 25012 (ISO/IEC, 2008) e ISO/IEC 5259 (ISO/IEC, 2024) para el diseño de las métricas de calidad.
3. **Implementación:** desarrollo del código del *backend* y del *frontend* de forma conjunta, garantizando que cada incremento sea funcional de extremo a extremo.
4. **Verificación:** ejecución de pruebas manuales y automatizadas para validar el correcto funcionamiento del incremento.
5. **Revisión con los tutores:** presentación del avance para obtener retroalimentación y ajustar la dirección del desarrollo en caso necesario.

Este ciclo se repite en cada sprint y puede observarse con detalle en las secciones de planificación y retrospectiva del Capítulo 5.

4.2.2 Prototipado

La interfaz de usuario se ha desarrollado siguiendo un enfoque de **prototipado evolutivo**: cada pantalla se construyó inicialmente con la funcionalidad mínima necesaria y se refinó en iteraciones sucesivas, incorporando mejoras de usabilidad, visualizaciones adicionales y tratamiento de casos extremos. Este enfoque permitió validar la experiencia de usuario de forma temprana sin invertir tiempo excesivo en el diseño visual antes de confirmar la utilidad funcional.

Un ejemplo representativo es el módulo de exploración de datos (EDA): en su primera versión solo mostraba un recuento de nulos y estadísticas básicas por columna; en iteraciones posteriores se incorporaron histogramas interactivos, diagramas de caja y una matriz de correlación, a medida que las pruebas con datos reales revelaban la necesidad de cada visualización.

4.3 Pila tecnológica

La pila tecnológica de DATAQUAL se estructura en cuatro capas independientes pero integradas: presentación (*frontend*), lógica de negocio (*backend*), persistencia de datos y orquestación asíncrona. La selección de cada tecnología responde a tres criterios: adecuación al tipo de problema (evaluación de calidad de datos, procesamiento asíncrono, visualización interactiva), madurez y soporte comunitario, y coherencia con las competencias adquiridas durante el grado en Ingeniería Informática.

4.3.1 Frontend

La capa de presentación se construyó sobre Next.js y React, combinación que ofrece enrutamiento basado en ficheros, reescritura de rutas para actuar como *proxy* hacia el *backend* y generación de contenido estático. Se descartó Vue.js por su ecosistema de componentes de visualización menos maduro en el momento del desarrollo. TypeScript se incorporó desde el inicio para garantizar la seguridad de tipos en un *frontend* con más de cuarenta componentes reutilizables. La Tabla 4.1 recoge el detalle de cada tecnología.

Cuadro 4.1: Tecnologías del *frontend*

Tecnología	Versión	Justificación
Next.js	13.3	<i>Framework</i> React con enrutamiento basado en ficheros, reescritura de rutas para <i>proxy</i> al <i>backend</i> y modo <i>standalone</i> para Docker.
React	18	Biblioteca estándar para interfaces de usuario. Los <i>hooks</i> y la Context API eliminan la necesidad de gestores de estado externos como Redux.
TypeScript	5.0	Tipado estático que detecta errores en tiempo de compilación, esencial para mantener un <i>frontend</i> con más de cuarenta componentes.
Material UI	5.12	Biblioteca de componentes con <i>theming</i> , diseño adaptable y accesibilidad incorporada.
Chart.js	4.2	Biblioteca de gráficos ligera con soporte para histogramas, diagramas de dispersión y de líneas. Se utiliza mediante el <i>wrapper</i> <code>react-chartjs-2</code> .
Axios	1.3	Cliente HTTP con interceptores para inyección automática de JWT y gestión centralizada de errores.
i18next	26.0	Internacionalización del <i>frontend</i> con detección automática de idioma y catálogos cargables; se integra con React mediante el <i>wrapper</i> <code>react-i18next</code> 17.0.

4.3.2 Backend

El *backend* se implementó en Python con Flask como *framework* web. Python fue la elección natural por su integración nativa con las bibliotecas de análisis de datos (Pandas, NumPy) que sustentan el motor de métricas. Flask se prefirió frente a Django porque no impone una estructura monolítica, lo que permitió diseñar una arquitectura modular basada en *blueprints* a medida de los requisitos del sistema; además, el autor contaba con experiencia previa con este *framework*, lo que redujo la curva de aprendizaje y aceleró el desarrollo inicial. La Tabla 4.2 recoge el detalle de cada biblioteca y herramienta empleada.

4.3.3 Base de datos y almacenamiento

La capa de persistencia separa deliberadamente dos tipos de datos con naturaleza distinta: los datos relacionales (proyectos, usuarios, métricas, resultados) se almacenan en Post-

Cuadro 4.2: Tecnologías del *backend*

Tecnología	Versión	Justificación
Python	3.10	Lenguaje estándar en ciencia de datos e IA, con excelente integración con Pandas y NumPy para el cálculo de métricas.
NumPy	1.24	Cálculo numérico de alto rendimiento sobre arrays. Se emplea en el motor de métricas para operaciones vectorizadas sobre columnas del conjunto de datos.
Flask	2.2	<i>Framework</i> web minimalista y flexible, ideal para APIs REST. No impone una estructura fija, lo que permite una arquitectura a medida.
SQLAlchemy	2.0	ORM maduro con soporte completo para PostgreSQL, incluyendo campos JSONB, tipos ENUM y <i>pool</i> de conexiones integrado.
Flask-JWT-Extended	4.4	Autenticación JWT con <i>tokens</i> de acceso y refresco, <i>blacklisting</i> y gestión de errores personalizada.
Flask-Migrate	4.0	Migraciones de base de datos versionadas con Alembic, que permiten evolucionar el esquema de forma controlada.
Pandas	1.5	Manipulación de datos tabulares: lectura de conjuntos de datos, cálculo de métricas y generación de estadísticas descriptivas.
Gunicorn	20.1	Servidor WSGI de producción con soporte multiproceso para gestionar la concurrencia.
Marshmallow	3.19	Validación y serialización declarativa de los <i>payloads</i> de la API REST; se emplea para verificar las configuraciones de métricas y los parámetros de evaluación antes de ejecutarlos.
Flask-Limiter	3.3	Limitación de tasa de peticiones por dirección IP (100 req/min por defecto), con Redis como almacén compartido entre los procesos de Gunicorn.

greSQL, mientras que los ficheros binarios subidos por los usuarios se delegan a MinIO. Esta separación evita saturar la base de datos relacional con objetos de gran tamaño y simplifica el escalado independiente de cada almacén. Redis actúa además como intermediario de mensajes para el sistema de colas asíncronas. La Tabla 4.3 detalla cada componente de esta capa.

Cuadro 4.3: Tecnologías de base de datos y almacenamiento

Tecnología	Versión	Justificación
PostgreSQL	14	Sistema de gestión de bases de datos relacional con soporte nativo para JSONB, tipos ENUM e índices avanzados. Permite combinar datos estructurados (relaciones entre entidades) con datos semiestructurados (configuraciones, resultados).
MinIO	<i>latest</i>	Almacenamiento de objetos compatible con S3, autoalojado. Separa los archivos binarios de los datos relacionales y mantiene los datos en la infraestructura propia sin depender de servicios externos en la nube.
Redis	6	Almacén en memoria con rol dual: <i>broker</i> de mensajes para Celery y <i>storage</i> para el limitador de tasa de peticiones. Su baja latencia es idónea para la gestión de colas.

4.3.4 Procesamiento asíncrono y orquestación

La ejecución de evaluaciones de calidad puede tardar varios segundos en conjuntos de datos de tamaño medio, por lo que realizarla de forma síncrona bloquearía la interfaz de usuario. Celery resuelve este problema desacoplando la petición del usuario de la ejecución real de la tarea: el *frontend* lanza la evaluación, recibe un identificador de tarea y consulta el estado periódicamente hasta que finaliza. Docker y Docker Compose unifican todos los servicios en un entorno reproducible que se despliega con un único comando. La Tabla 4.4 detalla las herramientas que componen esta capa.

El conjunto de estas cuatro capas forma una arquitectura desacoplada en la que cada componente puede sustituirse o escalarse de forma independiente. Todas las tecnologías seleccionadas son de código abierto, lo que elimina costes de licencia y garantiza la reproducibilidad del entorno de desarrollo y despliegue.

4.4 Entorno de desarrollo

El entorno de desarrollo está íntegramente basado en herramientas de código abierto y gratuitas, lo que garantiza que cualquier colaborador pueda reproducir el entorno sin coste adicional.

Editor de código

Visual Studio Code, con extensiones para Python (Pylance), TypeScript, Docker y $\text{\LaTeX}$ (LaTeX Workshop).

Cuadro 4.4: Tecnologías de procesamiento asíncrono y orquestación

Tecnología	Versión	Justificación
Celery	5.2	Cola de tareas distribuida estándar en Python, con soporte para reintentos, <i>timeouts</i> y seguimiento de progreso. Escalable horizontalmente mediante la adición de <i>workers</i> (Celery Project, 2024).
Flower	1.2	Cuadro de mando web para la monitorización en tiempo real de <i>workers</i> , tareas y colas de Celery.
Docker	—	Contenedorización de cada servicio, garantizando aislamiento de dependencias, portabilidad y reproducibilidad (Docker, Inc., 2024).
Docker Compose	—	Orquestación de los ocho servicios con <i>healthchecks</i> , volúmenes persistentes, red interna y variables de entorno. Un único comando (<code>docker-compose up</code>) despliega el <i>stack</i> completo.

Entorno de ejecución local

Docker Desktop sobre Windows 11, con Docker Compose para levantar los ocho servicios de forma simultánea. Este enfoque elimina la necesidad de instalar dependencias directamente en el sistema operativo anfitrión.

Gestor de paquetes *frontend*

pnpm, seleccionado por su eficiencia en el uso de disco y su velocidad de instalación frente a npm.

Gestor de paquetes *backend*

pip con fichero `requirements.txt` para la gestión de dependencias Python.

Pruebas

pytest como *framework* de pruebas del *backend*, con soporte para pruebas unitarias y de integración.

Asistente de IA

Claude Pro (Anthropic), utilizado como herramienta de apoyo para la resolución de dudas técnicas y de implementación durante el desarrollo.

El conjunto de estas herramientas, combinado con la contenedorización de todos los servicios, permite que el entorno de desarrollo sea indistinguible del entorno de producción, reduciendo los problemas habituales de incompatibilidad entre plataformas.

4.5 Planificación temporal

El desarrollo del proyecto se ha distribuido a lo largo del curso académico 2025–2026 en seis fases encadenadas. Las dos primeras sentaron las bases del sistema: entre octubre y noviembre 2025 se estudió el estado del arte y se analizaron los estándares ISO/IEC 25012 (ISO/IEC,

2008) e ISO/IEC 5259 (ISO/IEC, 2024), lo que permitió diseñar la arquitectura general; durante diciembre 2025 y enero 2026 se implementó el núcleo del *backend*: modelos de datos, autenticación JWT, módulo de ingesta multiformato con almacenamiento en MinIO y esqueleto de la API RESTful.

Las fases centrales construyeron el núcleo funcional. En febrero 2026 se diseñó e implementó el motor de métricas con arquitectura de *plugins* (clase base abstracta y registro dinámico) y las tres primeras métricas de calidad. En marzo 2026 se concentró el desarrollo del *frontend* con Next.js y React: cuadros de mando, páginas de gestión y el módulo de exploración de datos (EDA), si bien desde las fases anteriores ya existía una interfaz básica operativa; paralelamente se incorporaron nuevas métricas al catálogo.

Las dos últimas fases elevaron la madurez del sistema y cerraron el proyecto. Abril 2026 se dedicó a las funcionalidades avanzadas: *fingerprinting* de *issues*, *Quality Gates*, comparación con línea base y enmascaramiento de PII; en este mismo sprint se completó el catálogo de métricas con las restantes, incluida la métrica de diversidad. La fase final (mayo–junio 2026) abarca la redacción de la memoria, la validación con datos reales y la preparación de la defensa.

La Figura 4.1 resume los hitos principales entregados en cada sprint, y la Figura 4.2 muestra su distribución temporal a lo largo del curso.



Figura 4.1: Roadmap del proyecto: hitos entregados por sprint

El diagrama de Gantt refleja el *cuándo* de cada sprint, pero no la intensidad relativa del trabajo en cada área. La Figura 4.3 muestra esa distribución de esfuerzo por disciplinas y revela un patrón característico de los procesos ágiles: las actividades no se suceden en fases estancas, sino que se solapan a lo largo del proyecto.

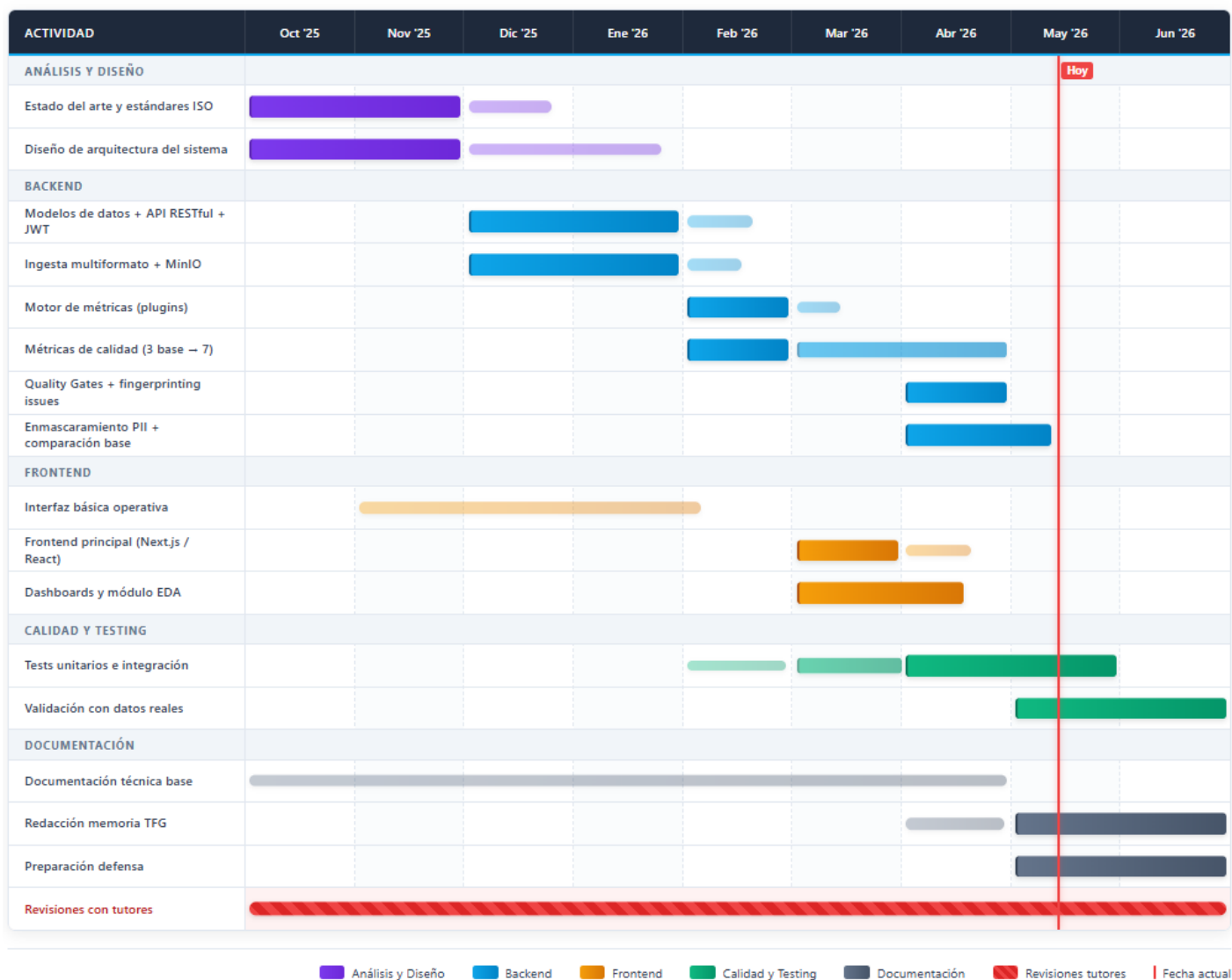


Figura 4.2: Diagrama de Gantt del proyecto (curso académico 2025–2026)

Los requisitos se concentran en el sprint inicial pero permanecen activos durante todo el desarrollo. El diseño y la arquitectura dominan los dos primeros sprints y decaen progresivamente. La implementación alcanza su pico en los sprints centrales, mientras las pruebas crecen de forma sostenida a medida que el sistema gana complejidad. La documentación, presente desde el inicio con un nivel de base, experimenta un salto significativo en el sprint final. Este patrón contrasta con la distribución secuencial del modelo en cascada (Somerville, 2016).

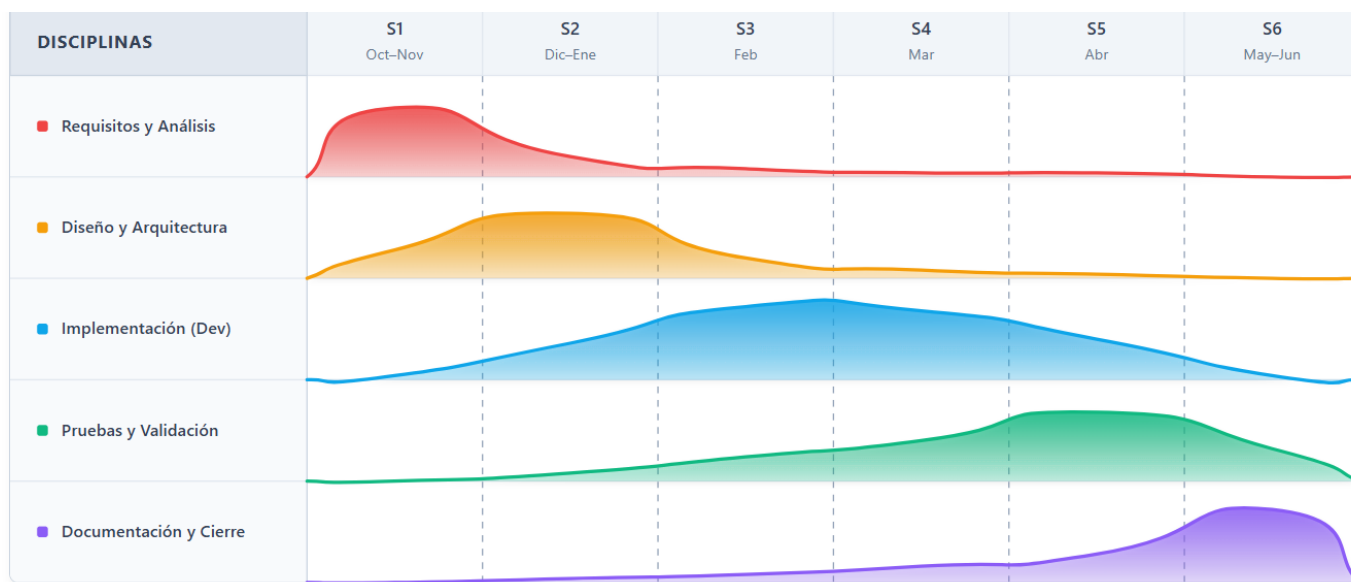


Figura 4.3: Distribución de esfuerzo por disciplinas a lo largo de los sprints

4.6 Backlog inicial e historias de usuario

La Tabla 4.5 recoge el backlog inicial del proyecto, organizado por épicas. Cada historia de usuario sigue el formato «*Como [rol], quiero [acción] para [beneficio]*» y se estima en *story points* (SP) usando la escala de Fibonacci. Las historias de usuario se formalizaron al inicio del proyecto como marco de planificación; las estimaciones en SP se ajustaron retrospectivamente al cierre de cada sprint con los tiempos aproximados de cada tarea. El backlog completo con la asignación a sprints se incluye en el Anexo C.

El backlog inicial planificado sumaba **71 puntos de historia**, de los cuales se entregaron **68**: HU-15 (panel de administración, 3 SP, prioridad baja) se descartó del alcance final ante la decisión de centrar el desarrollo en las funcionalidades del rol de usuario, que son las que aportan el valor diferencial de la plataforma. Las épicas de mayor peso entre las entregadas son Evaluación (13 SP) y, en segundo lugar, Proyectos, Datasets, Métricas, *Fingerprinting* y EDA (8 SP cada una).

Cuadro 4.5: Backlog inicial — historias de usuario principales

ID	Épica	Historia de usuario	Prior.	SP
HU-01	Auth	Como usuario, quiero registrarme con email y contraseña para acceder a la plataforma.	Alta	3
HU-02	Auth	Como usuario, quiero iniciar sesión y recibir un JWT para autenticar mis peticiones.	Alta	2
HU-03	Proyectos	Como usuario, quiero crear un proyecto para agrupar conjuntos de datos relacionados.	Alta	3
HU-04	Proyectos	Como usuario, quiero ver el resumen de calidad de mi proyecto en un cuadro de mando.	Alta	5
HU-05	Datasets	Como usuario, quiero cargar un fichero CSV para evaluarlo.	Alta	5
HU-06	Datasets	Como usuario, quiero ver el versionado de mis datasets para rastrear cambios.	Media	3
HU-07	Métricas	Como usuario, quiero configurar métricas con umbrales personalizados para adaptar la evaluación.	Alta	5
HU-08	Métricas	Como usuario, quiero usar plantillas de métricas para reutilizar configuraciones.	Media	3
HU-09	Evaluación	Como usuario, quiero lanzar una evaluación y ver el progreso en tiempo real.	Alta	8
HU-10	Evaluación	Como usuario, quiero ver los problemas detectados con su severidad y columna afectada.	Alta	5
HU-11	Quality Gates	Como usuario, quiero definir un Quality Gate para saber si mi dataset es apto.	Alta	5
HU-12	Fingerprinting	Como usuario, quiero que los issues se tracen entre evaluaciones para detectar recurrencias.	Media	8
HU-13	EDA	Como usuario, quiero explorar distribuciones, nulos y correlaciones de mis datos visualmente.	Media	8
HU-14	PII	Como usuario, quiero marcar columnas sensibles para que sus valores no aparezcan en resultados.	Media	5
HU-15 [†]	Admin	Como administrador, quiero gestionar usuarios y ver el estado del sistema.	Baja	3
Total SP planificados				71
Total SP entregados (sin HU-15)				68

[†] HU-15 fue descartada del alcance final: el proyecto no incluye interfaz de administración para gestión de usuarios. El blueprint técnico `/api/admin/` se mantiene en el *backend* con endpoints de salud y rendimiento únicamente.

4.7 Estimación de esfuerzo

4.7.1 Técnica de estimación

La estimación del esfuerzo combinó dos técnicas complementarias con distinto nivel de granularidad. A nivel de historia de usuario se emplearon *story points* para capturar la complejidad relativa sin comprometerse con una duración absoluta; a nivel de tarea técnica se estimó directamente en horas para facilitar el seguimiento diario de cada tarea.

- **Story Points con escala de Fibonacci** para la estimación relativa de las historias de usuario (véase Tabla 4.5). Se estableció una equivalencia orientativa de 4 horas por punto, lo que arroja una estimación base de 284 horas para las 71 SP del backlog. Esta cifra no incluye las tareas técnicas adicionales ni el sprint de documentación (S6), que se estimaron directamente en horas.
- **Descomposición en tareas y estimación en horas** para las tareas técnicas más granulares, así como para el sprint de documentación (S6), que no tiene historias de usuario asociadas y se estimó íntegramente en horas.

La Tabla 4.6 recoge la comparación entre la estimación inicial y el tiempo real invertido en cada sprint.

4.7.2 Estimación inicial vs. real

Cuadro 4.6: Estimación inicial vs. real por sprint

Sprint	Objetivo	SP	Est. (h)	Real (h)
S1	Análisis, arquitectura y setup inicial	13	52	60
S2	Backend core (auth, datasets, API REST)	18	72	85
S3	Motor de métricas (3 métricas iniciales)	16	64	70
S4	Frontend y visualizaciones	14	56	65
S5	Features avanzadas (fingerprinting, QG, PII)	10	40	55
S6	Documentación, pruebas y defensa	0	80	90
Total		71	364	425

La desviación media del **+16,8 %** sobre la estimación inicial es consistente con la literatura sobre proyectos de software de tamaño similar. Los sprints con mayor desviación fueron S5 (+38 %), S2 (+18 %) y S1 (+15 %), causados respectivamente por la complejidad del sistema de *fingerprinting*, el parseo robusto de CSV y la configuración de la infraestructura Docker. S6 registró una desviación del +12,5 %, atribuible al mayor volumen de documentación respecto a lo estimado inicialmente. S3 fue el sprint más ajustado (≈ 9 %), mientras que S4 se situó en torno al $\approx +16$ %, próximo a la media global, dentro del margen de incertidumbre habitual en proyectos de este tipo.

4.8 Estimación de costes

Con el objetivo de proporcionar una perspectiva de viabilidad económica, se estima el coste que tendría el proyecto en un entorno profesional real.

Cuadro 4.7: Estimación de costes del proyecto

Concepto	Detalle	Cantidad	Precio unit.	Total
Recursos humanos				
Desarrollador <i>full-stack</i> (junior)	425 h de desarrollo real	425 h	25 €/h	10 625 €
Supervisión de tutores	2 tutores × 10 h de reuniones y revisiones	20 h	60 €/h	1 200 €
Licencias de software				
Python, Flask, Next.js, PostgreSQL...	Todo el stack es <i>open source</i>	—	0 €	0 €
GitHub (plan Free)	Repositorio de código y control de versiones	1	0 €	0 €
Infraestructura				
Hardware de desarrollo	Amortización equipo propio (1 000 €, vida útil 3 años)	9 meses	28 €/mes	252 €
Electricidad	50 W × 8 h/día × 270 días	108 kWh	0,20 €/kWh	22 €
Servicios cloud	Despliegue local con Docker Compose	—	0 €	0 €
Coste total estimado				12 099 €

El coste total estimado de **12 099 €** se desglosa en 11 825 € de recursos humanos (10 625 € de desarrollo y 1 200 € de supervisión de tutores) y 274 € de infraestructura (252 € de amortización del equipo y 22 € de electricidad). Dado que todas las tecnologías utilizadas son de código abierto y el despliegue es local, el coste de licencias es nulo. Para un escenario de producción con despliegue en nube, el Anexo E estima un coste adicional de aproximadamente 90 €/mes con servicios equivalentes de AWS (RDS, S3, ElastiCache, ECS).

4.9 Gestión de riesgos

La gestión de riesgos en este proyecto siguió un enfoque ligero, propio del contexto uni-personal y de la naturaleza ágil del desarrollo. No se mantuvo un registro formal de riesgos actualizado en cada sprint; en su lugar, varias prácticas preventivas se aplicaron de forma implícita en el flujo de trabajo diario (*push* frecuente al repositorio remoto, fijación de versiones en `requirements.txt` y `package.json`, *healthchecks* en los servicios de Docker Compose, pruebas automatizadas del flujo de autenticación, etc.), y las acciones correctivas se decidieron *ad hoc* cuando aparecía una incidencia, dejando constancia en la retrospectiva del sprint correspondiente.

El análisis que se presenta a continuación tiene, por tanto, carácter **retrospectivo**: a partir de las incidencias documentadas en las retrospectivas del Capítulo 5, se reconstruyen los principales factores de riesgo a los que se enfrentó el proyecto, las prácticas que actuaron como mitigación preventiva y las acciones correctivas que efectivamente se aplicaron cuando alguno se materializó. Esta reconstrucción cumple un doble objetivo: documentar qué riesgos eran relevantes en un proyecto de estas características y extraer lecciones aprendidas que faciliten la planificación de proyectos futuros.

La Tabla 4.8 recoge los riesgos así reconstruidos con su probabilidad de ocurrencia (Alta/Media/Baja), impacto (Alto/Medio/Bajo) y las acciones asociadas.

Cuadro 4.8: Plan de gestión de riesgos

ID	Riesgo	Prob.	Imp.	Acción preventiva	Acción correctiva
R-01	Complejidad mayor de lo esperada en el motor de métricas	Media	Alto	Reservar un sprint completo (S3) para esta fase	Reducir el número de métricas en el MVP inicial
R-02	Problemas de integración entre servicios Docker	Media	Alto	Levantar el stack completo desde el sprint 1	Acotar el <i>healthcheck</i> y consultar documentación oficial
R-03	Desvío temporal por subestimación de tareas	Alta	Medio	Añadir un 20 % de margen en cada sprint	Repriorizar el backlog y mover HU a sprints siguientes
R-04	Pérdida de datos o código por accidente	Baja	Alto	Repositorio en GitHub con <i>push</i> frecuente	Restaurar desde el último <i>commit</i>
R-05	Cambios de requisitos por retroalimentación de tutores	Media	Medio	Revisiones periódicas con los tutores para detectar desviaciones	Incorporar cambios al backlog en la siguiente sprint planning
R-06	Problemas de rendimiento con ficheros grandes	Media	Medio	Establecer límite de 100 MB configurado en el servidor	Añadir endpoint de estado y aviso al usuario
R-07	Fallos en la autenticación JWT	Baja	Alto	Pruebas de integración del flujo auth completo	Revisión de configuración de expiración y <i>blacklisting</i>
R-08	Incompatibilidad de versiones entre dependencias	Media	Medio	Fijar versiones en <code>requirements.txt</code> y <code>package.json</code>	Revertir a la versión compatible e investigar la causa

De los ocho riesgos así reconstruidos, tres se materializaron de forma significativa: R-03 (desvío temporal) en los sprints S2 y S5, R-02 (integración Docker) en S5 con el sistema Celery+Redis, y R-08 (incompatibilidad de versiones) durante la migración de SQLAlchemy 1.x a 2.0 en S2. R-05 (cambios de requisitos por tutores) se materializó de forma menor, con ajustes puntuales al backlog en S5. R-01 tuvo una incidencia positiva: la complejidad del motor de métricas impulsó la ampliación del catálogo más allá de lo planificado inicial-

mente. Los tres riesgos restantes (R-04, R-06, R-07) no llegaron a producirse, lo que resulta coherente con las prácticas preventivas que se aplicaron de forma implícita en el flujo de trabajo. La principal lección extraída es que, en proyectos individuales de alcance acotado, un registro formal de riesgos puede sustituirse por un conjunto de prácticas preventivas integradas en el flujo de trabajo (control de versiones, fijación de dependencias, contenedorización, pruebas automatizadas) junto con la documentación de incidencias en las retrospectivas. El análisis detallado de ocurrencia se recoge en el Anexo D.

Capítulo 5

Desarrollo iterativo por sprints

Este capítulo constituye el núcleo del trabajo y describe el proceso de desarrollo de DATAQUAL en su dimensión real: qué se planificó hacer, qué se hizo efectivamente y cómo se hizo, iteración por iteración. El capítulo comienza con la definición global de requisitos y se estructura a continuación en seis sprints, cada uno con sus subsecciones de planificación, trabajo realizado, evidencias técnicas (*cómo se ha hecho*) y retrospectiva.

La estructura por sprints agrupa el desarrollo en **dominios funcionales**: análisis y arquitectura (S1), núcleo del *backend* (S2), motor de métricas (S3), interfaz de usuario (S4), funcionalidades avanzadas (S5) y consolidación (S6). Como es habitual en un proceso iterativo, las actividades de un dominio no se cerraron herméticamente al finalizar el sprint correspondiente: algunos componentes (como el *watchdog* de evaluaciones, el versionado de datasets, el sistema de *Quality Gates* o ciertos elementos de internacionalización) se iniciaron en un sprint y se refinaron o consolidaron en sprints posteriores a medida que las dependencias quedaban disponibles. Las tablas de *story points* y horas reflejan, para cada sprint, el trabajo principal del dominio correspondiente, mientras que la narrativa describe cada componente en el sprint donde alcanzó su versión funcional definitiva.

5.1 Requisitos globales del sistema

5.1.1 Historias de usuario

Las quince historias de usuario que definen el alcance funcional de DATAQUAL se recogen en la Tabla 4.5 del capítulo 4 (sección 4.6), donde se detallan también la épica, la prioridad y la estimación en *story points*. El backlog completo con todas las tareas técnicas y la asignación sprint a sprint se incluye en el Anexo C.

5.1.2 Casos de uso principales

El sistema soporta dos actores: **Usuario autenticado** (actor principal, único rol con interfaz de usuario en esta versión, con acceso a la gestión de proyectos, datasets, métricas, evaluaciones, Quality Gates y exploración de datos) y **Sistema Celery** (actor interno que ejecuta las tareas asíncronas de evaluación y supervisa su estado). Sobre estos dos actores se definen **ocho casos de uso** que cubren la totalidad del comportamiento funcional de DATAQUAL. El diagrama UML de casos de uso y la especificación textual de cada uno se recogen

en el Anexo H.

5.1.3 Burndown global del proyecto

La Figura 5.1 muestra el *burndown chart* del proyecto, representación gráfica de los *story points* pendientes en función del tiempo que permite evaluar si el ritmo de avance es suficiente para completar el alcance previsto. La figura compara la línea ideal de progreso con el avance real sprint a sprint (S1–S5; el Sprint 6 es de documentación y no consume *story points*).

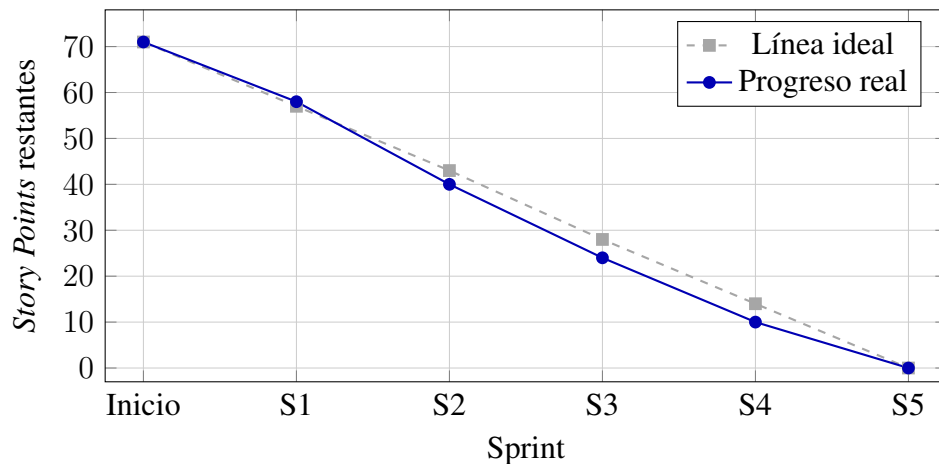


Figura 5.1: Burndown global del proyecto: *story points* restantes por sprint (línea ideal vs. progreso real)

El progreso real se mantuvo próximo a la línea ideal en todos los sprints, con convergencia total en S5. La posición favorable de la línea real respecto a la ideal desde S2 en adelante se debe a que los sprints centrales tenían más puntos asignados que la media uniforme (S2: 18 SP, S3: 16 SP, frente a los 14,2 SP/sprint de la distribución lineal), no a que se completara menos trabajo del planificado. Esta diferencia se mantiene en S4 y se anula en S5, donde ambas líneas convergen. Sin embargo, registrar un avance superior al estimado en SP no implica menor carga de trabajo: las horas reales superaron las estimadas en todos los sprints, poniendo de manifiesto que los *story points* subestiman la complejidad de las tareas técnicas de infraestructura.

5.2 Sprint 1: Análisis, arquitectura y setup inicial

Período: octubre–noviembre 2025 *Duración:* 6 semanas *SP:* 13

5.2.1 Planificación del sprint

Objetivo del sprint: diseñar la arquitectura completa del sistema, definir el modelo de datos inicial y poner en marcha el entorno de desarrollo Docker con todos los servicios básicos operativos.

Cuadro 5.1: Sprint 1: Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-01	Registro e inicio de sesión básico	3	10
HU-02	JWT: access + refresh token	2	8
–	Setup Docker Compose (8 servicios)	3	14
–	Diseño del modelo de datos (ERD)	3	12
–	Migraciones iniciales (Flask-Migrate)	2	16
–	Prueba de concepto: subida de dataset CSV	0	—
Total		13 SP	60 h

Dependencias y riesgos detectados: la configuración de Docker Compose con *healthchecks* y dependencias entre servicios era desconocida al inicio, constituyendo el principal riesgo (R-02 del plan de riesgos).

5.2.2 Trabajo realizado

Se completaron los ítems del sprint con las siguientes observaciones:

- El entorno Docker Compose quedó completamente operativo con los ocho servicios, *healthchecks*, red interna y volúmenes persistentes.
- Se diseñó el modelo de datos inicial con campos JSONB para flexibilidad futura. El esquema evolucionó en sprints posteriores hasta alcanzar las once tablas del sistema final.
- Se implementó el módulo de autenticación JWT completo con registro, login, refresco de token y revocación por JTI (en memoria; los tokens revocados no persisten entre reinicios del servidor).
- Se realizó una prueba de concepto de subida de archivo CSV al sistema: el endpoint básico de carga almacena el fichero en MinIO y registra los metadatos en la tabla *data-sets*. La implementación completa del módulo de ingesta se completó en el Sprint 2.
- **Cambio respecto a lo planificado:** la configuración de *healthchecks* en Docker Compose requirió más tiempo del estimado (+4h) debido a los tiempos de arranque de PostgreSQL y MinIO.

5.2.3 Cómo se ha hecho

Arquitectura general del sistema

DATAQUAL sigue una arquitectura de **servicios orquestados** mediante Docker Compose (Docker, Inc., 2024). Los ocho servicios que componen la plataforma se comunican a través de una red interna (*app-network*, *driver bridge*) y exponen al anfitrión únicamente los puertos necesarios.

El Cuadro 5.2 recoge los ocho servicios, sus puertos y su propósito.

Cuadro 5.2: Servicios Docker de DATAQUAL

Servicio	Puerto	Propósito
frontend	3000	Interfaz de usuario (Next.js + React)
backend	5000	API REST (Flask + Gunicorn)
postgres	5432	Base de datos relacional (PostgreSQL 14)
minio	9000/9001	Almacenamiento de objetos S3
redis	6379	<i>Broker</i> de mensajes y caché
celery-worker	—	Ejecución asíncrona de evaluaciones
celery-beat	—	Programación de tareas periódicas
flower	5555	Monitorización de Celery

El flujo de comunicación entre servicios se organiza de la siguiente manera: el *frontend* (Next.js) realiza peticiones HTTP/JSON al *backend* (Flask); éste lee y escribe en PostgreSQL (datos relacionales), MinIO (archivos binarios) y Redis (caché y *broker*); cuando el usuario lanza una evaluación, el *backend* encola una tarea en Redis que es consumida por el *Celery worker*.

El Cuadro 5.3 resume los protocolos de comunicación.

Cuadro 5.3: Protocolos de comunicación entre componentes

Origen	Destino	Protocolo	Mecanismo
Frontend	Backend	HTTP/JSON	Next.js (<i>rewrite /api/*</i>), cliente Axios
Backend	PostgreSQL	TCP	SQLAlchemy ORM (<i>pool</i> de 10 conexiones)
Backend	MinIO	HTTP/S3	Cliente MinIO para Python
Backend	Redis	TCP	Celery <i>broker</i> + Flask-Limiter
Worker	Redis	TCP	Consumo de tareas del <i>broker</i>
Worker	PostgreSQL	TCP	Actualización de progreso y resultados
Worker	MinIO	HTTP/S3	Descarga de conjuntos de datos

Todos los servicios poseen una política de reinicio *unless-stopped* y los servicios de infraestructura incluyen *healthchecks* que verifican su disponibilidad antes de que arranquen los servicios dependientes.

Diagrama de arquitectura

La Figura 5.2 representa la arquitectura del sistema como un diagrama por capas a alto nivel. Cada banda corresponde a una capa arquitectónica y las flechas discontinuas con punta abierta codifican la relación *depends-on* entre capas. La descomposición garantiza una separación nítida de responsabilidades: el *frontend* solo conoce la API REST, los *blueprints* delegan toda la lógica en los servicios, y el motor de métricas (*services/metrics/*, resaltado en el diagrama) concentra el cálculo algorítmico de la calidad como núcleo extensible mediante el patrón *plugin*. La especificación textual de cada paquete se recoge en el Ane-

xo H.

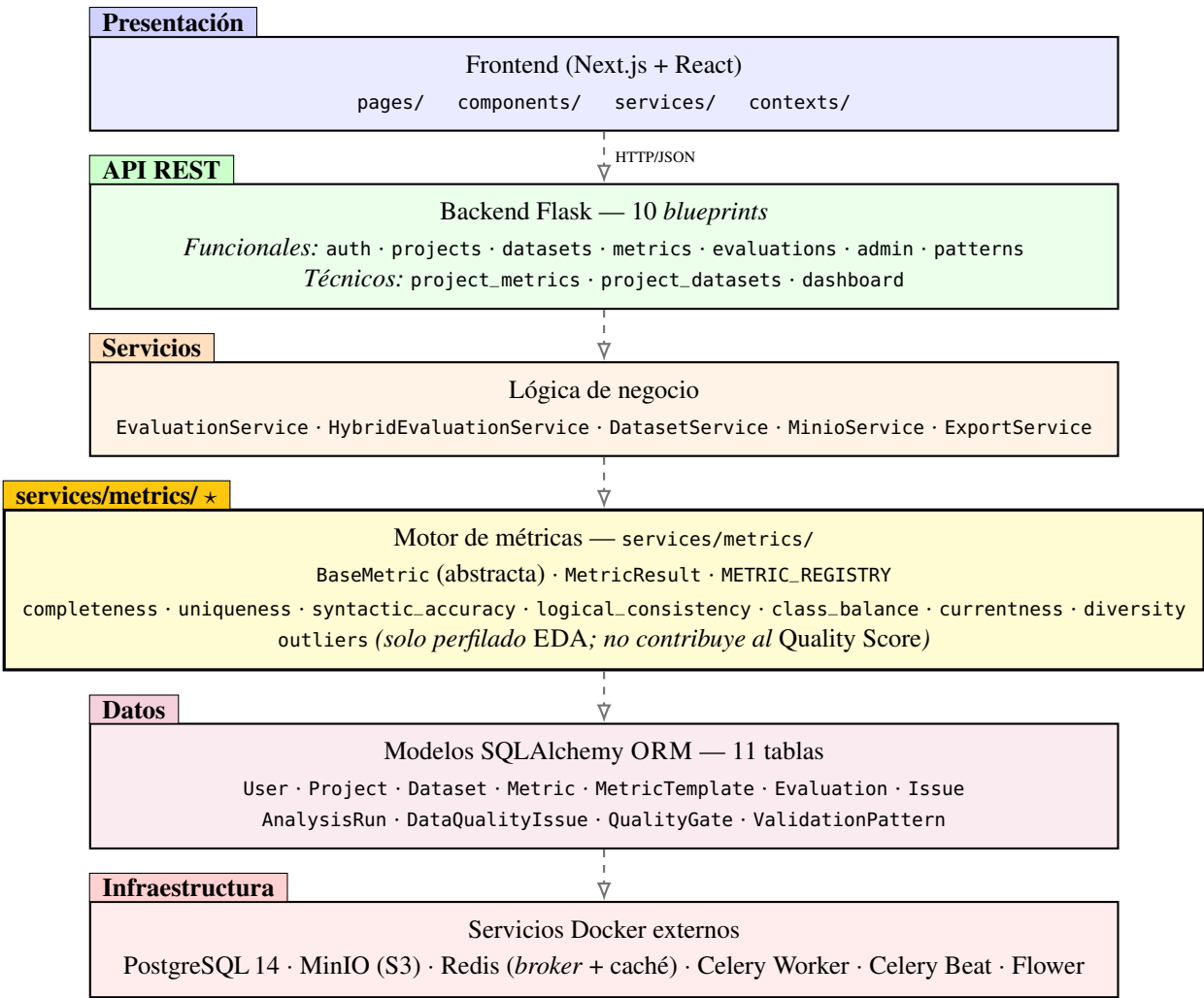


Figura 5.2: Arquitectura lógica por capas de DATAQUAL en su estado final (cierre del Sprint 5), mostrando las dependencias *depends-on* (flecha discontinua de punta abierta). El directorio services/metrics/ (*) constituye el núcleo extensible del motor de evaluación, alineado con el principio abierto–cerrado (Gamma et al., 1994)

Modelo de datos

PostgreSQL 14 (The PostgreSQL Global Development Group, 2024) almacena todos los datos relacionales y metadatos del sistema. El esquema final comprende once tablas, aunque en este sprint solo se crearon las siete iniciales; las cuatro tablas del modelo Sonar-Lite (analysis_runs, data_quality_issues, quality_gates y validation_patterns) se incorporaron en el Sprint 5. El diseño hace un uso extensivo de campos **JSONB** para almacenar datos semiestructurados cuya estructura varía según la métrica o la configuración.

users

Datos de usuario: nombre, correo electrónico, hash de contraseña y rol.

projects

Proyectos del usuario, con nombre, descripción y configuración de métricas almace-

nada como JSONB (`metrics_config`).

datasets

Conjuntos de datos asociados a un proyecto, con la ruta al archivo en MinIO, el esquema inferido (JSONB), las columnas sensibles (JSONB), la versión y el indicador `is_latest`. Cada conjunto de datos puede tener un `parent_dataset_id` que apunta a su versión anterior.

analysis_runs

Ejecuciones de evaluación (modelo Sonar-Lite): estado, puntuación de calidad, estado del *Quality Gate*, referencia a la evaluación base, contadores de *issues* nuevos, corregidos y recurrentes, y resultados detallados (JSONB).

data_quality_issues

Problemas de calidad detectados en cada evaluación: *fingerprint* SHA-256, severidad, tipo, descripción, columnas afectadas y muestras de filas.

quality_gates

Configuración de *Quality Gates* por proyecto, con umbrales almacenados como JSONB (`min_score`, `max_critical_issues`, `max_new_issues`).

metric_templates

Plantillas de configuración de métricas, reutilizables entre proyectos. Pueden ser del sistema (`is_system=true`) o del usuario.

metrics

Configuraciones de métricas activas por proyecto: tipo de métrica, parámetros, umbrales y peso relativo en el *Quality Score*. Enlazada a `projects` mediante clave foránea.

evaluations

Registro de alto nivel de cada evaluación (modelo original): estado (PENDING/RUNNING/COMPLETED/FAILED), referencia al dataset y al proyecto, y marca temporal. Coexiste con `analysis_runs` por razones de retrocompatibilidad (véase la nota sobre arquitectura dual más adelante).

issues

Incidencias de calidad del modelo original: métrica origen, columna afectada, severidad y descripción textual. Vinculadas a `evaluations`. Las evaluaciones nuevas utilizan `data_quality_issues`, que añade *fingerprint* SHA-256 y muestras de filas.

validation_patterns

Patrones de validación de formato definidos por el usuario (expresiones regulares con ejemplos válidos e inválidos), reutilizables entre proyectos. Pueden ser del sistema (`is_system=true`) o propios del usuario. Son la extensión dinámica del catálogo estático de trece formatos de la métrica `syntactic_accuracy`, gestionados mediante el *blueprint* patterns.

El empleo de campos JSONB permite almacenar estructuras flexibles dentro del esquema relacional: `projects.metrics_config` (lista de métricas con parámetros y peso), `datasets.schema` (tipos y estadísticas por columna), `datasets.sensitive_columns` (columnas PII) y `analysis_runs.results` (resultados completos por métrica).

5.2.4 Retrospectiva

Conseguido: entorno de desarrollo completamente operativo, arquitectura documentada, autenticación JWT funcional y prueba de concepto de subida de dataset CSV al sistema.

Estado del producto: *backend* operativo con ocho servicios Docker en marcha, modelo de datos inicial en PostgreSQL, autenticación JWT funcional y capacidad básica de ingesta de ficheros CSV, constituyendo una base funcional sobre la que desarrollar los sprints sucesivos. La plataforma carece aún de motor de métricas e interfaz de usuario.

Pendiente: ningún ítem del sprint quedó sin completar; sin embargo, las migraciones del esquema requerirán iteraciones adicionales conforme evolucione el modelo.

Qué fue bien: la decisión de definir el modelo de datos completo desde el inicio evitó migraciones disruptivas en sprints posteriores.

Qué fue mal: la configuración de Docker Compose con *healthchecks* fue más compleja de lo previsto (+4h de desviación), especialmente la secuencia de arranque con dependencias entre servicios.

Estimación vs. real: 52 h estimadas vs. 60 h reales (+15,4 %).

Lección aprendida: reservar tiempo explícito para la configuración de infraestructura, que tiende a subestimarse sistemáticamente.

Acción de mejora: añadir un margen del 20 % en las estimaciones de tareas de infraestructura para los sprints siguientes.

5.3 Sprint 2: *Backend* principal, datasets y API REST

Período: diciembre 2025–enero 2026 *Duración:* 8 semanas *SP:* 18

5.3.1 Planificación del sprint

Objetivo del sprint: implementar el módulo de ingesta y almacenamiento de datasets, la arquitectura completa del *backend* en capas, la API REST y la gestión de proyectos.

Dependencias: requiere Sprint 1 completado (base de datos y autenticación operativos).

Riesgo principal: la diversidad de formatos de archivo y codificaciones de CSV puede generar una estrategia de *fallback* más compleja de lo previsto (R-03).

Cuadro 5.4: Sprint 2: Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-03	CRUD de proyectos + configuración de métricas	3	12
HU-05	Carga y parseo robusto de archivos CSV	5	22
HU-06	Versionado de datasets (parent-child, is_latest)	3	10
–	Arquitectura del backend en capas (blueprints, servicios)	4	18
–	Especificación y pruebas manuales de la API REST	3	23
Total		18 SP	85 h

5.3.2 Trabajo realizado

- Implementada la arquitectura del *backend* en capas: *blueprints* Flask, capa de servicio y *middleware*.
- Desarrollado el módulo de ingesta CSV con estrategia de *fallback* múltiple (4 codificaciones $\times$ 5 delimitadores).
- Implementado el versionado de datasets con relaciones parent-child.
- Completados los seis módulos funcionales de la API REST con formato de respuesta estandarizado y paginación. Durante la implementación se añadieron tres *blueprints* adicionales por convención REST: `project_metrics` y `project_datasets` (sub-rutas anidadas `/projects/<id>/...`) y `dashboard` (agregación de solo lectura), elevando el total a nueve *blueprints* Flask registrados.
- **Cambio respecto a lo planificado:** el parseo robusto de CSV requirió una semana adicional por la casuística de ficheros con BOM, delimitadores inusuales y codificaciones mixtas.

5.3.3 Cómo se ha hecho

Arquitectura del *backend*

El *backend* sigue una **arquitectura en capas** con separación clara de responsabilidades.

La aplicación Flask se crea mediante el **patrón factoría** (`create_app()` en `app.py`) (Gamma et al., 1994), que inicializa las extensiones, registra los *blueprints* y configura el *middleware*. Los **seis módulos funcionales principales** de la API se montan bajo el prefijo `/api:`

- `/api/auth/`: autenticación (inicio de sesión, registro, refresco de *token*, perfil).
- `/api/projects/`: CRUD de proyectos y configuración de métricas.
- `/api/datasets/`: carga, consulta y versionado de conjuntos de datos.
- `/api/metrics/`: consulta de métricas y gestión de plantillas.
- `/api/evaluations/`: lanzamiento y consulta de evaluaciones.

- `/api/admin/:rendimiento` del sistema y comprobación de salud.

Siguiendo la convención REST de rutas anidadas (Fielding, 2000), se implementaron adicionalmente tres *blueprints* de organización técnica: `project_metrics` (`/projects/<id>/metrics`), `project_datasets` (`/projects/<id>/datasets`) y `dashboard` (agregaciones de solo lectura). El total de *blueprints* Flask registrados al cierre de este sprint asciende a **nueve**; durante el Sprint 5 se añadirá un séptimo módulo funcional (`patterns`, gestión de patrones de validación reutilizables) elevando el total final a diez, tal como se representa en el diagrama de arquitectura (Figura 5.2).

La lógica de negocio se encapsula en servicios independientes. En este sprint se establecieron los dos servicios fundamentales: `DatasetService` (parseo y extracción de esquema) y `MinioService` (abstracción sobre S3). En sprints posteriores se añadirán `EvaluationService` (S3, orquestación del flujo de evaluación), `HybridEvaluationService` (S5, *fallback* síncrono) y `ExportService` (S5, generación de informes).

El *backend* incorpora capas de *middleware*: generación de X-Request-ID, gestión centralizada de errores HTTP y monitor de rendimiento por *endpoint* (detecta peticiones lentas >500 ms), establecidos en este sprint. El *Evaluation Watchdog* (hilo *daemon* que detecta evaluaciones atascadas) se incorporó en el Sprint 3, una vez que las evaluaciones fueron operativas.

El sistema mantiene una **arquitectura dual de modelos** como resultado de su evolución incremental. El modelo original (`Evaluation + Issue`), implementado en el Sprint 2, ofrecía una estructura sencilla adecuada para el MVP. Al diseñar en el Sprint 5 el sistema de *fingerprinting* y los *Quality Gates*, resultó necesario un modelo semánticamente más rico: el **modelo Sonar-Lite** (`AnalysisRun + DataQualityIssue + QualityGate`), que soporta comparación con línea base, contadores de *issues* nuevos/recurrentes/corregidos y configuración de umbrales por proyecto. Ambos modelos coexisten para preservar la retrocompatibilidad con los *endpoints* implementados en sprints anteriores; la unificación completa en un único modelo queda documentada como mejora futura en la sección 7.4.

Módulo de ingesta y almacenamiento

Los usuarios pueden cargar archivos en formato CSV con un límite de tamaño configurable (por defecto, 100 MB). Tras superar la validación, el fichero se persiste en MinIO como objeto binario y sus metadatos (esquema inferido, tamaño, versión y proyecto al que pertenece) se registran en PostgreSQL mediante el modelo `Dataset`.

El servicio `DatasetService` implementa una estrategia de lectura robusta de CSV estructurada en tres fases:

1. **Validación del formato real:** se inspeccionan los primeros bytes del fichero para detectar archivos incorrectamente etiquetados como CSV. Los ficheros Excel (XLSX)

se rechazan con un mensaje claro al usuario, mientras que los archivos comprimidos con *gzip* se descomprimen de forma transparente antes de continuar.

2. **Detección del delimitador:** se aplica `csv.Sniffer` sobre los primeros 8 192 bytes del contenido, evaluando los candidatos coma, punto y coma, tabulador y *pipe*.
3. **Matriz de *fallback*:** si la detección del delimitador no es concluyente o la lectura no produce un *DataFrame* válido, se ensayan exhaustivamente combinaciones de cuatro codificaciones (UTF-8, UTF-8 con BOM, CP1252, Latin-1), seis estrategias de separador (el detectado, la inferencia automática de Pandas y los cuatro candidatos explícitos) y los dos motores de *parsing* de Pandas (Python y C). Se retorna el primer intento que produzca columnas válidas.

Al cargar un conjunto de datos, el sistema extrae automáticamente el nombre y tipo de cada columna, estadísticas descriptivas y histogramas para columnas numéricas. El versionado opera mediante un campo `parent_dataset_id` opcional y un indicador `is_latest`: las cargas sucesivas dentro de un mismo proyecto generan versiones encadenadas que conservan el historial completo. Las figuras 5.3 y 5.4 muestran la vista de versionado de un *dataset*: la primera recoge la lista cronológica de versiones con sus etiquetas de rama y puntuación de calidad; la segunda presenta el árbol visual interactivo (*Visual version tree*) con el diferencial de puntuación entre versiones padre-hijo. La Figura 5.5 ilustra la pantalla de comparación entre dos versiones, con el desglose de *issues* resueltos, nuevos y persistentes.

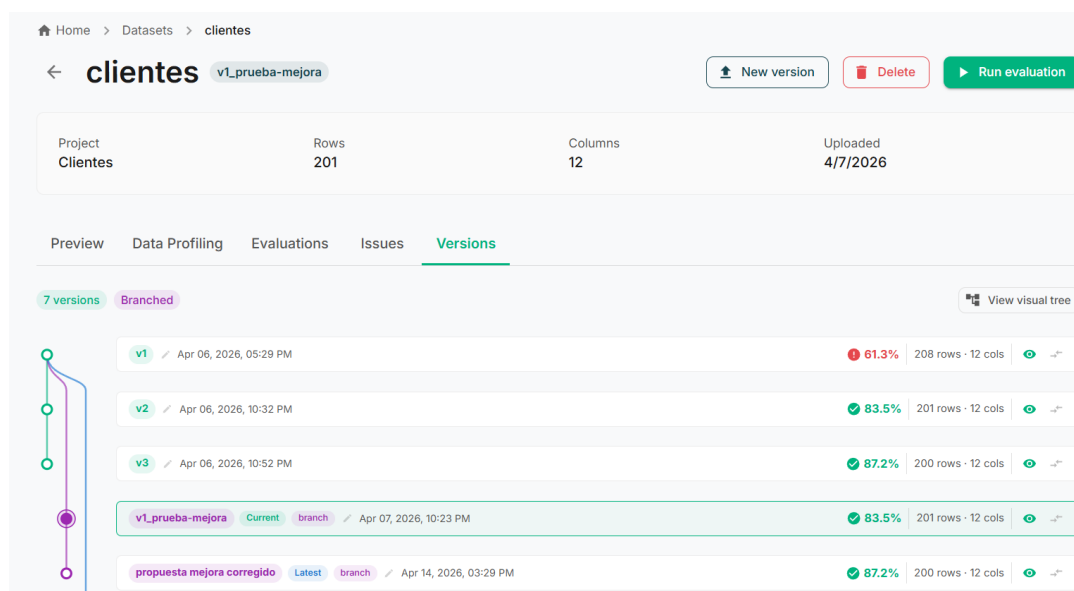


Figura 5.3: Pestaña *Versions* de un *dataset*: lista cronológica de versiones con etiquetas de rama, estado de calidad y puntuación por versión

API RESTful

La API RESTful presenta las siguientes características transversales: autenticación JWT en todos los *endpoints* protegidos, formato de respuesta estandarizado {`success`, `data`, `mes-`

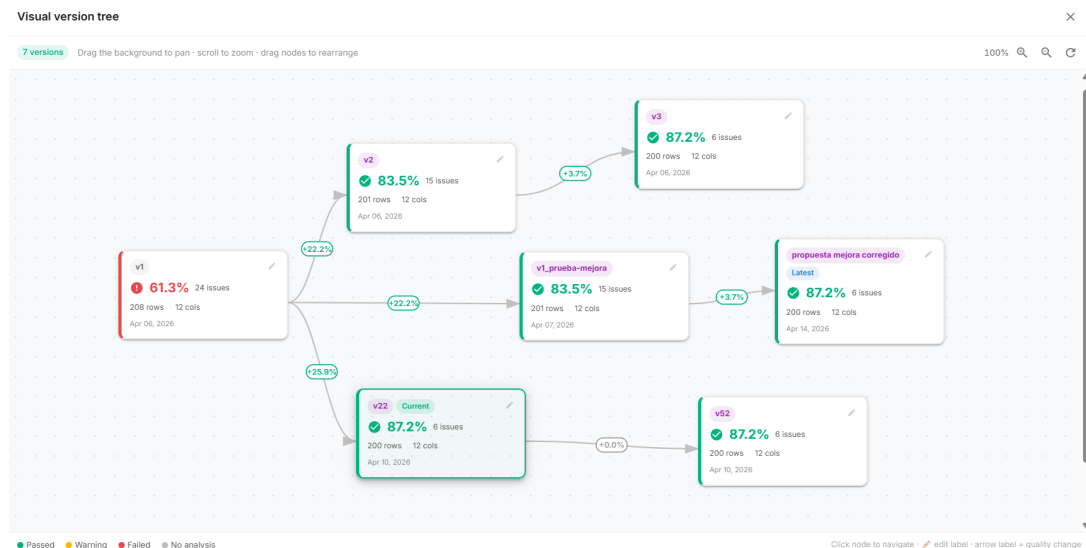


Figura 5.4: Canvas de árbol de versiones (*Visual version tree*): grafo interactivo con nodos coloreados por estado de *Quality Gate* y diferencial de puntuación entre versiones padre-hijo

sage}, paginación con parámetros `page/per_page`, y validación mediante esquemas `Marshmallow`. La especificación completa de *endpoints* se recoge en el Anexo A.

5.3.4 Retrospectiva

Conseguido: *backend* completamente funcional con todos los módulos principales, API REST operativa y datasets versionados.

Estado del producto: *backend* completo con nueve *blueprints* Flask registrados (seis módulos funcionales más tres de organización técnica REST: `project_metrics`, `project_datasets` y `dashboard`), módulo de ingesta CSV, versionado de datasets y autenticación JWT. La plataforma puede almacenar y gestionar conjuntos de datos, pero carece aún de motor de métricas e interfaz de usuario.

Pendiente: el motor de métricas (HU-07, HU-09, HU-10) y el *frontend* completo (HU-04, HU-13, HU-14) están programados para S3 y S4 respectivamente.

Qué fue bien: la separación en capas (*blueprints* + servicios) facilita significativamente las pruebas y la extensión futura.

Qué fue mal: el parseo robusto de CSV fue mucho más complejo de lo esperado. La diversidad de codificaciones y delimitadores reales encontrados en datasets de prueba obligó a extender el sprint una semana.

Estimación vs. real: 72 h estimadas vs. 85 h reales (+18,1 %).

Lección aprendida: los datos procedentes de entornos reales son más variados e irregulares de lo que sugieren los formatos estándar; es preferible implementar estrategias defensivas desde el inicio.

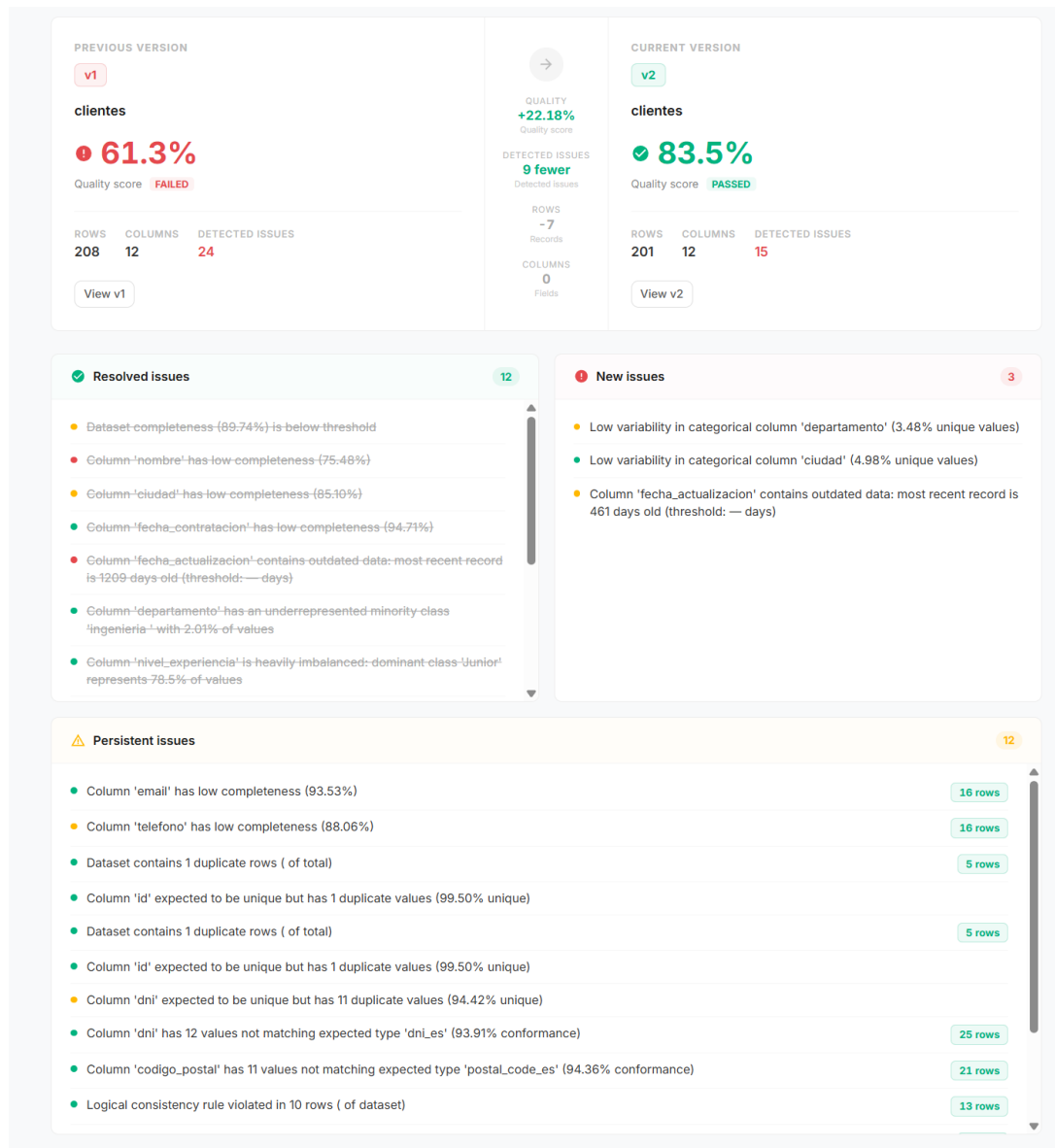


Figura 5.5: Comparación entre versiones de un *dataset*: *issues* resueltos, nuevos y persistentes entre dos versiones seleccionadas, con detalle de descripción y columna afectada

Acción de mejora: crear una batería de datasets de prueba con casos extremos (BOM, delimitadores inusuales, caracteres especiales) para validar el parseo en el siguiente sprint.

5.4 Sprint 3: Motor de métricas de calidad

Período: febrero 2026 Duración: 4 semanas SP: 16

5.4.1 Planificación del sprint

Objetivo del sprint: diseñar e implementar el motor de métricas con arquitectura de *plugins* y las tres primeras métricas de calidad, alineadas con ISO/IEC 25012 e ISO/IEC 5259.

Cuadro 5.5: Sprint 3: Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-07	Configuración de métricas con parámetros y pesos	5	20
HU-08	Plantillas de métricas del sistema	3	10
HU-09	Lanzamiento de evaluación (tarea Celery)	5	22
HU-10	Listado de issues con severidad	3	18
Total		16 SP	70 h

Dependencias: requiere Sprint 2 completado (API REST y datasets operativos).

5.4.2 Trabajo realizado

- Diseñada e implementada la arquitectura de *plugins* del motor de métricas (BaseMetric, MetricResult, METRIC_REGISTRY).
- Implementadas las tres primeras métricas evaluables del catálogo: completitud (completeness), unicidad (uniqueness) y consistencia lógica (logical_consistency). Paralelamente se implementó la clase OutliersMetric, usada solo en el módulo de perfilado de datos y excluida del *Quality Score* por diseño.
- Implementada la fórmula del *Quality Score* con penalización por issues.
- Integrada la ejecución asíncrona con Celery.

5.4.3 Cómo se ha hecho

Arquitectura del motor de métricas

El motor de métricas constituye el núcleo funcional de DATAQUAL. Su diseño se basa en una arquitectura de *plugins* que permite añadir nuevas métricas sin modificar el código existente.

La arquitectura se compone de tres elementos:

BaseMetric

Clase base abstracta que define la interfaz que toda métrica debe implementar: el método `evaluate()`, que recibe un *DataFrame* de Pandas, los parámetros configurados y

devuelve un objeto `MetricResult`. La clase base proporciona además métodos auxiliares para el cálculo de severidad dinámica, la generación de histogramas, la inferencia de tipos de columna y el enmascaramiento de valores sensibles.

MetricResult

Estructura de datos (*dataclass*) que encapsula el resultado de una evaluación: identificador de la métrica, puntuación (0,0–1,0), resultados detallados y lista de *issues* detectados.

METRIC_REGISTRY

Diccionario que asocia cada identificador de métrica a su clase implementadora, permitiendo la instanciación dinámica por nombre.

Para añadir una nueva métrica basta con: (1) crear una clase heredera de `BaseMetric` que implemente `evaluate()`; (2) registrarla en `METRIC_REGISTRY`; (3) añadir la función de *finger-print*; y (4) documentarla en el Anexo B.

Métricas implementadas

A continuación se describen las **siete métricas evaluables** del motor de calidad (implementadas progresivamente en S3, S4 y S5) y la clase adicional de detección de valores atípicos, disponible en el módulo de perfilado.

Complejitud (completeness) Mide el grado en que los datos esperados están presentes, calculando la ratio de valores no nulos por columna. La fórmula implementada corresponde directamente al *Completeness Ratio* definido en ISO/IEC 25024 (ISO/IEC, 2015):

$$r_{\text{comp}} = \frac{N - N_{\text{null}}}{N}$$

donde N es el número total de valores esperados y N_{null} el número de valores nulos o ausentes. La puntuación global es la media de r_{comp} sobre las columnas configuradas. Se generan *issues* si la completitud global queda por debajo del umbral (0,95 por defecto) o si alguna columna cae por debajo de ese mismo umbral (equivalente a más del 5 % de nulos con la configuración por defecto).

Unicidad (uniqueness) Evalúa la ausencia de duplicados a dos niveles: filas completas y variabilidad por columna. La ratio de unicidad sigue la fórmula del *Uniqueness Ratio* de ISO/IEC 25024 (ISO/IEC, 2015):

$$r_{\text{uniq}} = \frac{N_{\text{distinct}}}{N}$$

Los umbrales de variabilidad se adaptan al tipo semántico inferido: 0,95 para identificadores, 0,05 para categóricas y 0,30 para el resto.

Detección de valores atípicos (*outliers*):solo perfilado La clase `OutliersMetric` detecta *outliers* en columnas numéricas mediante IQR (Rango Intercuartílico) (Tukey, 1977) y Z-score. Esta clase no figura en el `METRIC_REGISTRY` y no contribuye al *Quality Score*: siguiendo la recomendación de ISO/IEC 5259 (ISO/IEC, 2024), los valores atípicos no se consideran una dimensión de calidad intrínseca sino un indicador de diagnóstico. La clase se instancia exclusivamente desde el módulo de perfilado de datos (*EDA*), donde los histogramas de *outliers* y los diagramas de caja se generan para orientar al usuario sin penalizar la puntuación global.

Exactitud sintáctica (*syntactic_accuracy*) Verifica la conformidad de los valores con patrones de formato predefinidos. El catálogo incluye trece tipos (*email*, *url*, teléfono español (*phone_es*) e internacional (*phone_intl*), DNI, fechas ISO y europeas, enteros, decimales, UUID, IPv4, código postal y tarjeta de crédito). Para columnas sin configuración explícita, detecta el tipo más probable automáticamente.

Consistencia lógica (*logical_consistency*) Valida reglas de negocio entre columnas, expresadas como reglas IF - THEN evaluadas mediante `pandas.DataFrame.query()`. Antes de ejecutar cualquier regla, se verifica la ausencia de *tokens* prohibidos (`import`, `exec`, `eval`) para prevenir inyección de código.

Equilibrio de clases (*class_balance*) Métrica específica para ML supervisado. Utiliza la entropía de Shannon (1948) para cuantificar el equilibrio de la variable objetivo: 100 indica distribución uniforme; cercano a 0 indica desequilibrio total. Se generan *issues* cuando la clase dominante supera el 90 % o la minoritaria representa menos del 5 %.

Actualidad (*currentness*) Mide la frescura de los datos calculando la antigüedad del valor de fecha más reciente respecto al momento actual. La degradación de la puntuación es lineal entre el umbral de obsolescencia y el doble de dicho umbral.

Fórmula del *Quality Score*

La puntuación global de calidad ($Q \in [0,1]$) se calcula mediante un modelo de **penalización por *issues* con corrección dimensional**. Cada métrica produce una puntuación individual (ratio de celdas válidas) que se expone en la interfaz como información de diagnóstico, pero **no alimenta directamente** la nota final: una proporción pequeña de errores graves puede hacer un conjunto de datos inutilizable aunque la ratio global sea elevada. El cálculo sigue tres pasos:

1. **Penalización bruta por severidad:** cada *issue* detectado contribuye con un peso fijo

según su nivel de severidad:

$$p_{\text{raw}} = 0,12 \cdot n_{\text{crit}} + 0,05 \cdot n_{\text{high}} + 0,01 \cdot n_{\text{med}} + 0,003 \cdot n_{\text{low}}$$

2. **Corrección dimensional:** los conjuntos de datos con más columnas generan naturalmente más *issues*. Un factor de escala basado en la raíz cuadrada del número de columnas normaliza la penalización para que la misma *densidad* de errores produzca la misma nota con independencia de la dimensionalidad del dataset:

$$\sigma = \sqrt{\frac{\max(10, c)}{10}}$$

donde c es el número de columnas del conjunto de datos y 10 es el número de referencia (dataset de anchura estándar).

3. **Puntuación final:**

$$Q = \max\left(0, 1 - \min(0,97, p_{\text{raw}}/\sigma)\right)$$

La cota 0,97 sobre la penalización normalizada garantiza que ningún dataset obtiene $Q = 0$ por un único *issue* crítico aislado, preservando la diferenciación entre conjuntos de datos muy deficientes.

5.4.4 Retrospectiva

Conseguido: motor de métricas operativo con las tres primeras métricas evaluables (completitud, unicidad y consistencia lógica), `OutliersMetric` para perfilado, *Quality Score* con corrección dimensional y arquitectura de *plugins* extensible.

Estado del producto: *backend* completo más motor de métricas con tres métricas evaluables iniciales, fórmula de *Quality Score* y ejecución asíncrona vía Celery. La plataforma puede lanzar evaluaciones y almacenar resultados; falta la interfaz de usuario para visualizarlos y las funcionalidades avanzadas (fingerprinting, Quality Gates).

Pendiente: *frontend* completo (HU-04, HU-13, HU-14) en S4 y funcionalidades diferenciadoras (HU-11, HU-12) en S5.

Qué fue bien: la arquitectura de *plugins* con `METRIC_REGISTRY` demostró su valor: añadir cada nueva métrica requirió modificar únicamente el fichero de implementación correspondiente.

Qué fue mal: la métrica de consistencia lógica presentó dificultades con la validación de seguridad de las expresiones `query()`. Se necesitaron varias iteraciones para equilibrar la flexibilidad de las reglas con la prevención de inyección de código.

Estimación vs. real: 64 h estimadas vs. 70 h reales (+9,4 %).

Lección aprendida: la seguridad de las métricas que ejecutan expresiones de usuario debe

diseñarse desde el principio, no añadirse como parche posterior.

Acción de mejora: documentar la interfaz de cada métrica (*inputs*, *outputs*, parámetros) antes de iniciar S4, para facilitar la integración con el *frontend*.

5.5 Sprint 4: *Frontend* e interfaz de usuario

Período: marzo 2026 *Duración:* 4 semanas *SP:* 14

5.5.1 Planificación del sprint

Objetivo del sprint: desarrollar la interfaz de usuario completa con Next.js y React, incluyendo el cuadro de mando, las páginas de gestión de proyectos y datasets, los resultados de evaluación y el perfilado exploratorio de datos (EDA).

Cuadro 5.6: Sprint 4: Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-04	Cuadro de mando principal con resumen	5	20
HU-09	Progreso de evaluación en tiempo real (polling)	3	14
HU-13	EDA: histogramas, boxplots, correlaciones	5	24
HU-14	Marcado de columnas PII en la UI	1	7
Total		14 SP	65 h

Dependencias: requiere Sprint 3 completado (motor de métricas operativo y API de evaluaciones funcional para poder visualizar resultados desde el *frontend*).

5.5.2 Trabajo realizado

- Incorporadas tres métricas evaluables adicionales al catálogo: exactitud sintáctica (*syntactic_accuracy*), equilibrio de clases (*class_balance*) y actualidad (*currentness*), completando seis de las siete métricas evaluables del sistema.
- Desarrollados más de 40 componentes React reutilizables.
- Implementado el cuadro de mando con tarjetas resumen y alertas de proyectos con problemas. El gráfico de distribución de estados de *Quality Gate* se completó en el Sprint 5, una vez que el modelo *AnalysisRun* estuvo operativo.
- Implementada la exploración de datos con histogramas, diagramas de caja, matrices de correlación Pearson/Spearman y detección interactiva de outliers.
- Integrado el perfilado (EDA) completo en *DataProfilingTab.tsx* (85 KB, componente principal del análisis exploratorio).
- **Cambios respecto a lo planificado:** la implementación de la matriz de correlación con Chart.js requirió un *plugin* personalizado, lo que añadió tiempo no estimado. Adicio-

nalmente, la métrica de exactitud sintáctica amplió su catálogo a trece tipos de formato (el plan inicial contemplaba ocho), lo que añadió complejidad positiva al sprint.

5.5.3 Cómo se ha hecho

Estructura de la aplicación *frontend*

La aplicación sigue las convenciones de Next.js 13.3 (Vercel, 2024) con enrutamiento basado en archivos (`pages/`) y componentes reutilizables (`components/`). El estado global se gestiona mediante la Context API de React con cuatro contextos: autenticación (`AuthContext`), notificaciones (`NotificationContext`), barra lateral (`SidebarContext`) e idioma (`LanguageContext`).

El módulo `services/api.ts` implementa una instancia Axios con interceptores para la inyección automática del *token* JWT, deduplicación de peticiones GET, gestión centralizada de errores y *timeouts* configurables (8–30 s según la operación).

Cuadros de mando y visualizaciones

La interfaz proporciona las siguientes visualizaciones principales:

- **Cuadro de mando principal** (`/dashboard`): tarjetas resumen, gráfico de distribución de *Quality Gates* y listado de proyectos que requieren atención.
- **Resultados de evaluación** (`/evaluations/[id]`): indicador circular de puntuación de calidad (*gauge*), pestañas especializadas por métrica, tabla de métricas por columna y listado de *issues*.
- **Exploración de datos** (`/datasets/[id]`, pestaña *Profiling*): histogramas, diagramas de caja, gráficos de barras, matrices de correlación (Pearson y Spearman) y detección interactiva de valores atípicos con factor IQR configurable. Las estadísticas descriptivas expuestas siguen el catálogo de métricas de perfilado propuesto por Abedjan et al. (2018).
- **Gráficos de tendencia**: los componentes `QualityTrendChart` y `VersionEvolutionChart` se crearon en este sprint; los datos reales que los alimentan (historial de `AnalysisRun`) estuvieron disponibles a partir del Sprint 5.

Las visualizaciones se implementan con Chart.js (Chart.js Contributors, 2024) mediante el *wrapper* `react-chartjs-2`.

Las figuras 5.6–5.9 muestran las pantallas principales de la interfaz de usuario desarrolladas en este sprint.

5.5.4 Retrospectiva

Conseguido: interfaz de usuario completa y funcional con todas las pantallas principales y el módulo EDA. Incorporadas además tres métricas evaluables al catálogo (`syntactic_accuracy`, `class_balance` y `currentness`), completando seis de las siete métricas del sistema.

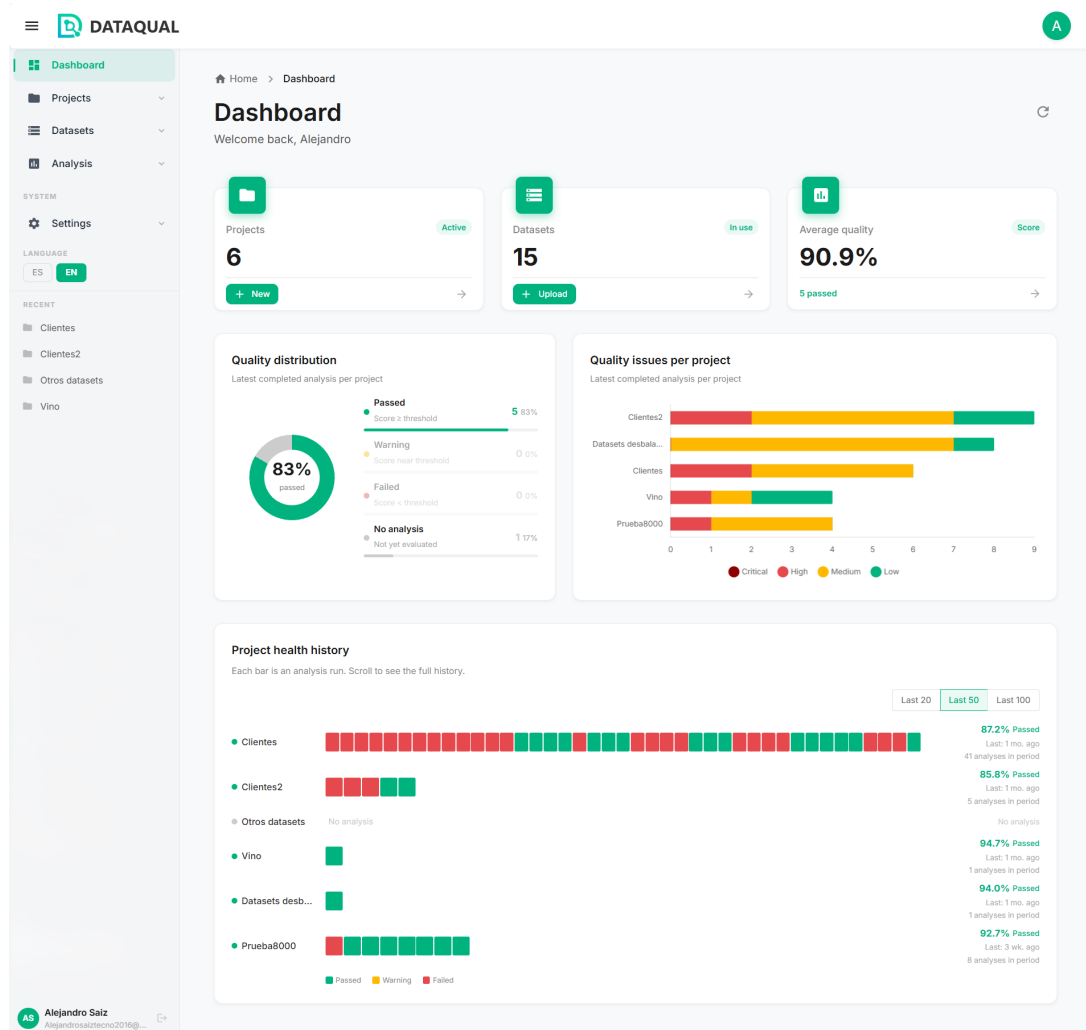


Figura 5.6: Cuadro de mando principal de DATAQUAL: tarjetas resumen de proyectos, distribución de estados de *Quality Gate* y alertas de proyectos con problemas detectados

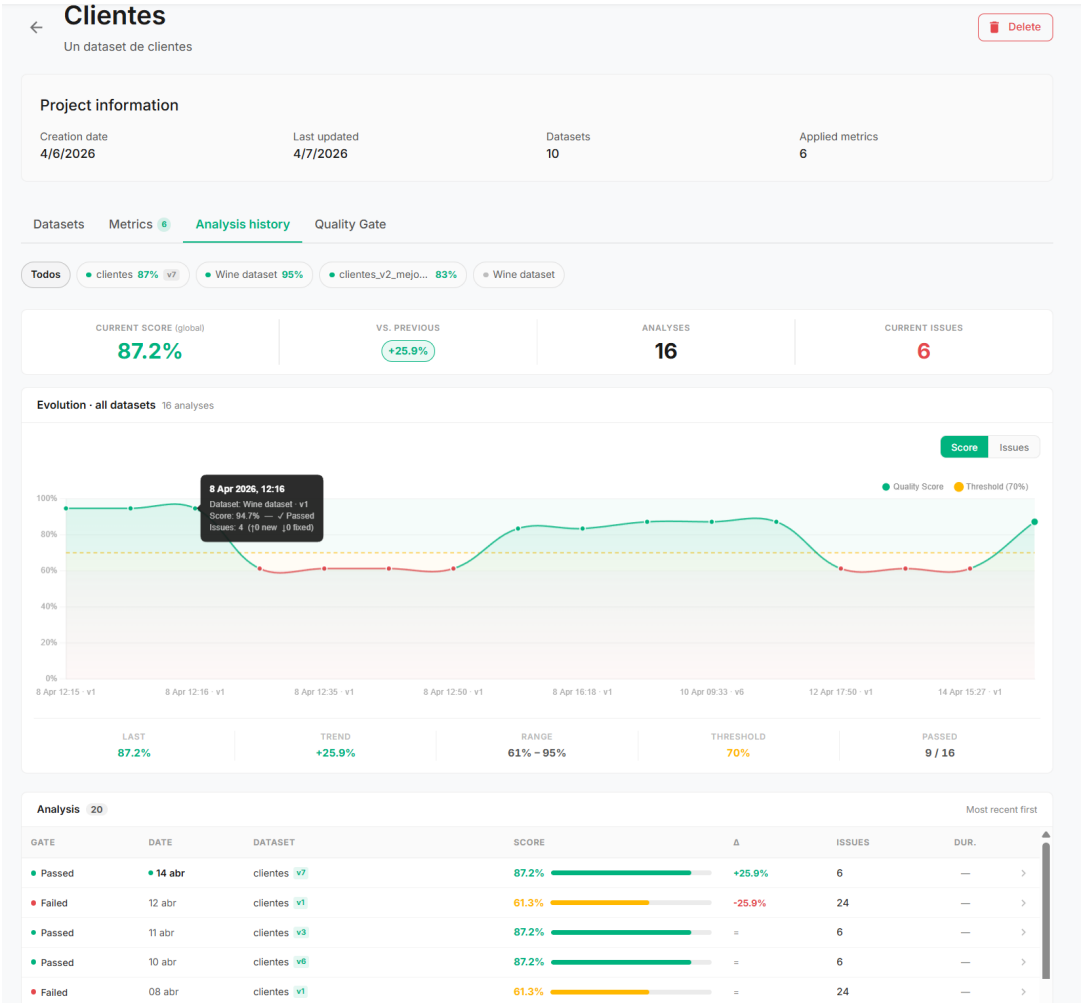


Figura 5.7: Historial de análisis de un proyecto: evolución temporal de la puntuación de calidad, tendencia por *dataset* y tabla de ejecuciones con estado de *Quality Gate*

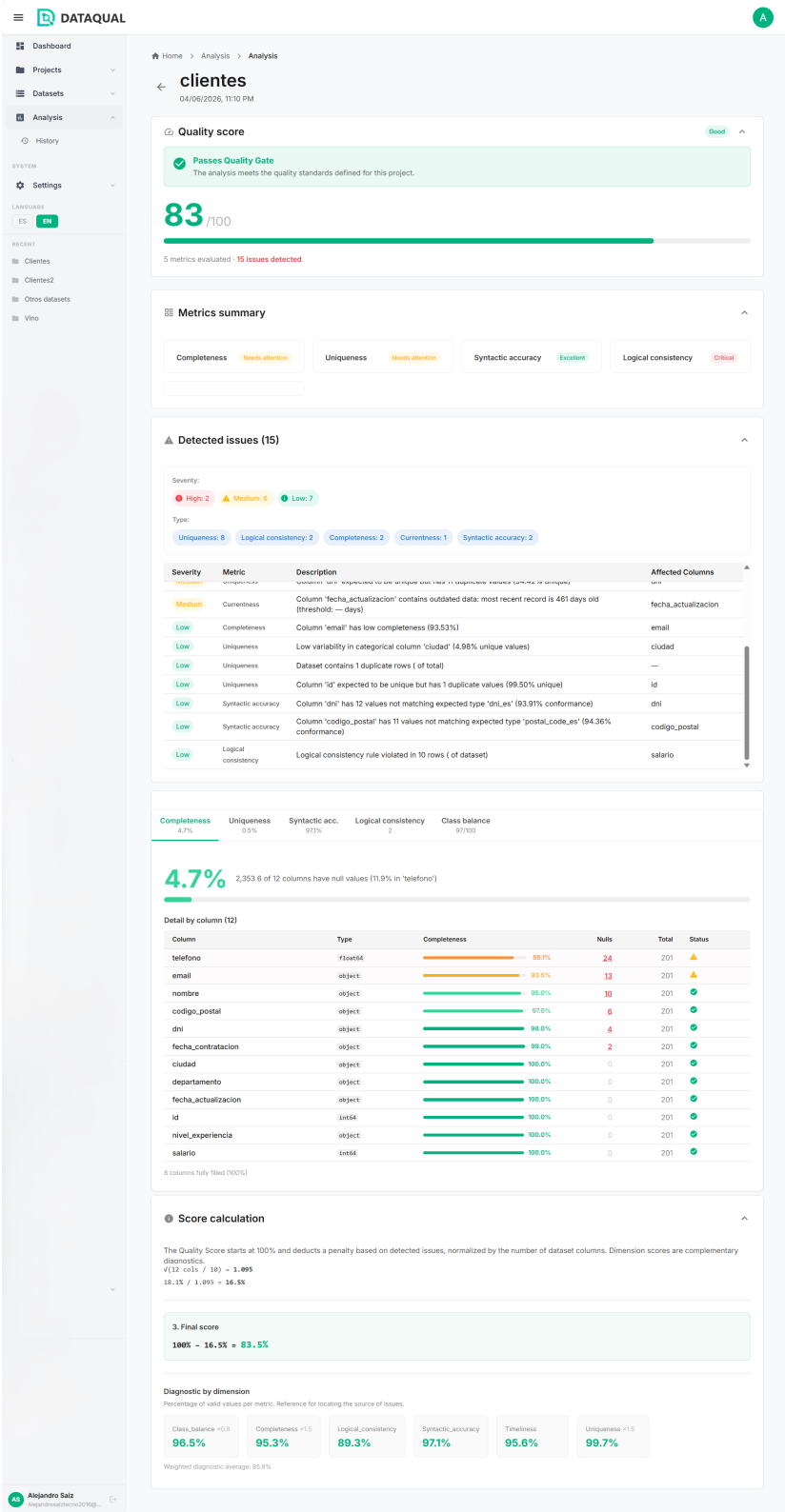


Figura 5.8: Página de resultados de evaluación: puntuación de calidad global, resumen de métricas por dimensión, listado de *issues* con severidad y columna afectada, y cálculo detallado de la puntuación final

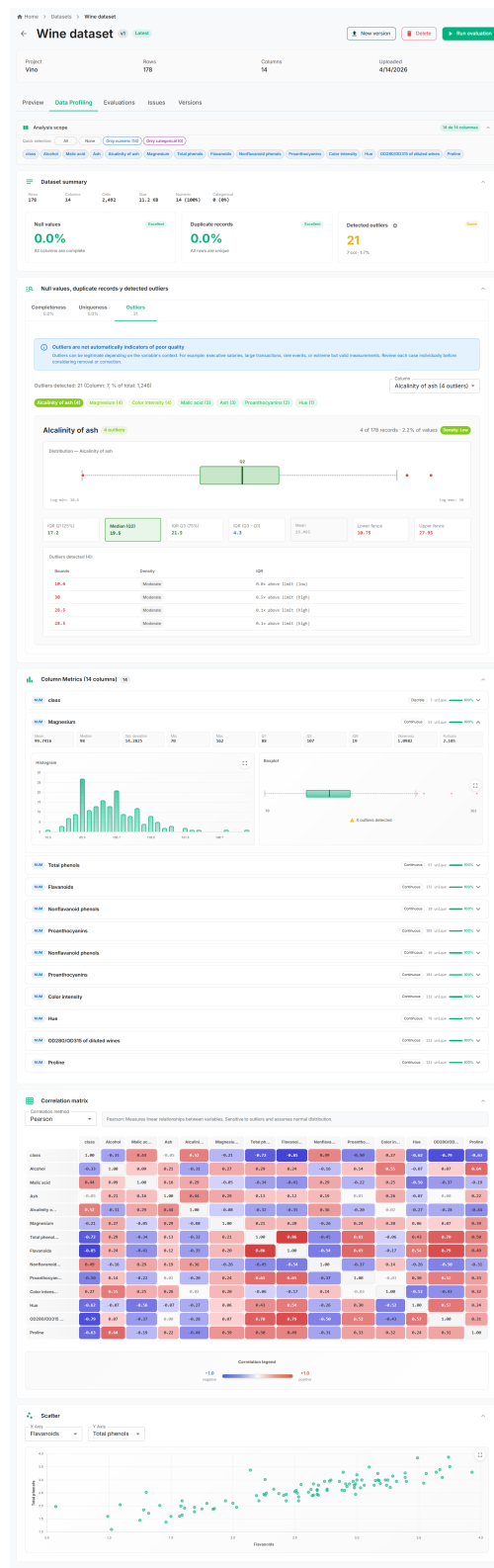


Figura 5.9: Módulo de exploración de datos (EDA): resumen estadístico, diagrama de caja (*boxplot*), distribuciones por columna, matriz de correlación Pearson/Spearman y diagrama de dispersión

Estado del producto: plataforma *full-stack* funcional de extremo a extremo: *backend*, motor de métricas e interfaz de usuario con cuadro de mando, gestión de proyectos, datasets y evaluaciones, y módulo de exploración de datos. Falta implementar las funcionalidades diferenciadoras (fingerprinting, Quality Gates) y las medidas de seguridad avanzadas.

Pendiente: fingerprinting SHA-256 (HU-12) y Quality Gates (HU-11) programados para S5.

Qué fue bien: el uso de Material UI aceleró significativamente el desarrollo de componentes, especialmente tablas, tarjetas y diálogos modales.

Qué fue mal: la complejidad del componente `DataProfilingTab.tsx` (85 KB) es excesiva. En iteraciones futuras sería conveniente refactorizarlo en subcomponentes más pequeños.

Estimación vs. real: 56 h estimadas vs. 65 h reales (+16,1 %).

Lección aprendida: en proyectos *full-stack*, el *frontend* tiende a consumir más tiempo que el *backend* equivalente, especialmente en las visualizaciones interactivas.

Acción de mejora: extraer los subcomponentes de `DataProfilingTab.tsx` (histogramas, boxplots, correlación) en ficheros independientes al inicio de S5, antes de añadir más funcionalidades al *frontend*.

5.6 Sprint 5: Funcionalidades avanzadas

Período: abril 2026 *Duración:* 4 semanas *SP:* 10

5.6.1 Planificación del sprint

Objetivo del sprint: implementar las funcionalidades diferenciadoras de DATAQUAL: sistema de *fingerprinting* para trazabilidad de issues, *Quality Gates* con tres estados, procesamiento asíncrono robusto y medidas de seguridad avanzadas.

Cuadro 5.7: Sprint 5: Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-11	Quality Gates con tres estados (PASSED/WARNING/FAILED)	5	18
HU-12	Fingerprinting SHA-256 para trazabilidad de issues	5	22
–	Seguridad: rate limiting, cabeceras, PBKDF2	0	15
Total		10 SP	55 h

Dependencias: requiere Sprint 4 completado (interfaz de usuario operativa para visualizar los estados de *Quality Gate* y los *badges* de trazabilidad de *issues*).

Riesgo principal: la trazabilidad de issues requiere una definición precisa y determinista del *fingerprint* para que sea fiable entre evaluaciones.

5.6.2 Trabajo realizado

- Implementado el sistema de *fingerprinting* SHA-256 con clasificación de issues en nuevos, corregidos y recurrentes.
- Implementados los *Quality Gates* con umbrales configurables y los tres estados en backend y frontend.
- Implementado el procesamiento asíncrono con Celery + *fallback* síncrono y *Evaluation Watchdog*.
- Añadidas las medidas de seguridad: limitación de tasa, cabeceras HTTP, PBKDF2-SHA256 para contraseñas, red Docker aislada.
- Incorporada la métrica de diversidad (*DiversityMetric*) al catálogo, completando las siete métricas evaluables del sistema. La métrica cuantifica la representación de valores en columnas categóricas y numéricas, con soporte de *fingerprinting* propio mediante `generate_diversity_fingerprint`.
- Como funcionalidad emergente surgida durante este sprint, se incorporó un séptimo *blueprint* funcional, `patterns`, que expone un CRUD de patrones de validación reutilizables entre proyectos (modelo `ValidationPattern`). Esta extensión del catálogo estático de trece formatos predefinidos de la métrica `syntactic_accuracy` permite a los usuarios definir y compartir sus propias expresiones regulares sin tocar código. Con esta adición, el total de *blueprints* Flask registrados pasa de nueve a **diez** (siete funcionales más tres de organización técnica).
- **Cambio respecto a lo planificado:** el diseño del *fingerprint* requirió más iteraciones de las esperadas para garantizar que sea completamente determinista ante parámetros de métricas flotantes. Adicionalmente, el *blueprint* `patterns` constituye una desviación positiva respecto al plan de OE3, que contemplaba seis módulos funcionales.

5.6.3 Cómo se ha hecho

Sistema de *fingerprinting*

Cada *issue* detectado recibe un *fingerprint*: un *hash* SHA-256 truncado a 16 caracteres hexadecimales, generado a partir de los atributos que definen la identidad del *issue* (tipo, columna afectada, regla, parámetros). El *fingerprint* es **determinista**: el mismo problema produce siempre el mismo *hash*.

La fórmula de construcción es:

$$fingerprint = \text{SHA-256}(\text{tipo} \mid \text{columna} \mid \text{id_fila} \mid \text{clave_regla} \mid \text{params_ord})[:16]$$

donde cada campo se normaliza antes del *hash*: cadenas en minúsculas y sin espacios super-

fluos, flotantes convertidos a su representación canónica en cadena (`str()`) y listas e *extra_params* ordenados alfabéticamente para garantizar el mismo resultado independientemente del orden de inserción.

La Tabla 5.8 resume los campos concretos que cada tipo de *issue* aporta al *hash*.

Cuadro 5.8: Inputs al *hash* por tipo de *issue*

Métrica	Clave de regla	<i>Extra_params</i>
Completeness	<code>completeness_column_check</code>	umbral (4 dec.)
Uniqueness (col.)	<code>column_duplicate_check</code>	—
Uniqueness (fila)	<code>row_duplicate_check</code>	—
Syntactic accuracy	<code>type_conformance_check</code>	tipo esperado; patrón regex (si aplica)
Logical consistency	<code>cross_field_check</code>	expresión de la regla
Class balance	<code>balance_{tipo}</code>	—
Currentness	<code>staleness_check</code>	umbral en días
Diversity	<code>diversity_{tipo}</code>	tipo de diversidad (<code>missing_value</code> , <code>underrepresented</code> , <code>low_range_coverage</code>)

Al finalizar cada evaluación, el servicio compara los *fingerprints* actuales con los de la evaluación anterior (*baseline*):

- **Issues nuevos:** presentes en la evaluación actual pero no en la *baseline*.
- **Issues corregidos:** presentes en la *baseline* pero no en la evaluación actual.
- **Issues recurrentes:** presentes en ambas.

Los contadores correspondientes se almacenan en el `AnalysisRun` y se visualizan en la interfaz mediante *badges*.

Procesamiento asíncrono

DATAQUAL utiliza Celery (Celery Project, 2024) con Redis (Redis Ltd., 2024) como *broker* para ejecutar las evaluaciones de forma asíncrona. La tarea `run_evaluation` orquesta el flujo completo: crea el contexto Flask, actualiza el estado (`PENDING` → `RUNNING` → `COMPLETED/FAILED`), actualiza el progreso (0–100 %) e invoca `EvaluationService`.

Si Celery no está disponible, el `HybridEvaluationService` ejecuta la evaluación de forma síncrona, garantizando que la funcionalidad esté disponible incluso sin *workers*. El *Evaluation Watchdog* (hilo *daemon*) detecta evaluaciones atascadas (sin progreso durante más de 5 minutos) y las marca automáticamente como fallidas.

Seguridad

La seguridad de la plataforma se aborda desde múltiples capas:

- **Autenticación JWT:** *access token* (24 h) + *refresh token* (30 días) con revocación por JTI.

- **Hashing de contraseñas:** PBKDF2-SHA256 con *salt* aleatorio por contraseña (Werkzeug).
- **Limitación de tasa:** Flask-Limiter restringe las peticiones a 100/min por IP, almacenando contadores en Redis.
- **Cabeceras de seguridad:** X-Content-Type-Options, X-Frame-Options y X-XSS-Protection en todas las respuestas.
- **Validación de entrada:** esquemas Marshmallow, verificación de extensiones y tamaño máximo de archivo.
- **Protección contra inyección:** las reglas de consistencia lógica se verifican contra una lista de *tokens* prohibidos antes de su ejecución.
- **Red Docker aislada:** los servicios se comunican por la red interna app-network sin exponer puertos innecesariamente.

La Figura 5.10 ilustra el panel de *Quality Gate* implementado en este sprint.

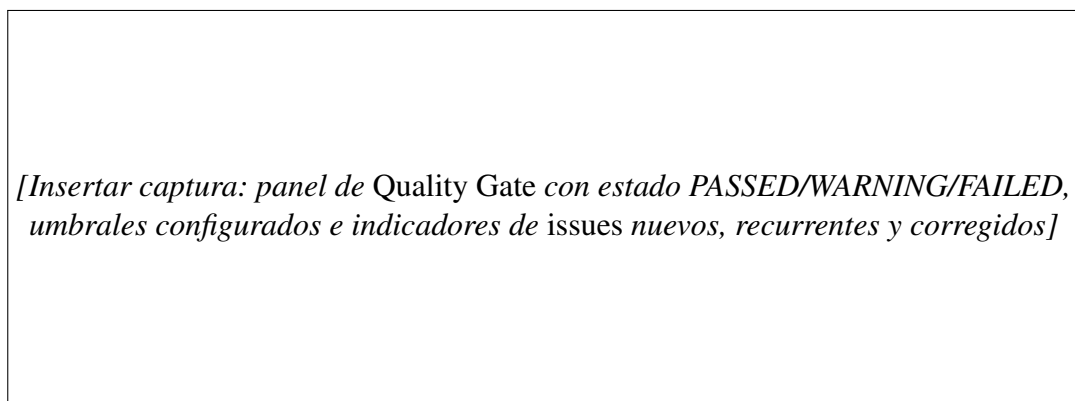


Figura 5.10: *Quality Gate* de DATAQUAL: estado global con código de color (PASSED/WARNING/FAILED), desglose de condiciones configuradas y *badges* de trazabilidad de *issues* por *fingerprinting*

5.6.4 Retrospectiva

Conseguido: funcionalidades diferenciadoras completamente implementadas: *fingerprinting*, *Quality Gates*, procesamiento asíncrono robusto y seguridad multicapa. Incorporada la séptima métrica evaluable (DiversityMetric) y el *blueprint* patterns para patrones de validación reutilizables.

Estado del producto: DATAQUAL es una plataforma completa con todas las funcionalidades principales: ingesta CSV, siete métricas de calidad evaluables, interfaz *full-stack*, *fingerprinting* SHA-256, *Quality Gates* y seguridad multicapa. El producto está funcionalmente cerrado; S6 se dedicará a pruebas, corrección de defectos y documentación.

Pendiente: cobertura de pruebas automatizadas (ninguna hasta este punto), corrección de defectos detectados y redacción de la memoria, planificados para S6.

Qué fue bien: el sistema de *fingerprinting* es elegante y determinista; la normalización de parámetros a cadena canónica garantiza resultados reproducibles independientemente del orden o formato de los valores de entrada (la corrección definitiva del bug de serialización se aplicó en S6).

Qué fue mal: la combinación Celery + Redis presentó problemas de conexión intermitentes en entornos Docker Windows, requiriendo ajustes en los *healthchecks* y en los tiempos de reconexión. Adicionalmente, la refactorización de `DataProfilingTab.tsx` planificada como acción de mejora de S4 no se ejecutó: la carga técnica del sprint no permitió abordarla sin comprometer las funcionalidades prioritarias.

Estimación vs. real: 40 h estimadas vs. 55 h reales (+37,5 %). La mayor desviación del proyecto, causada por los problemas de integración Docker + Celery (R-02) y la complejidad del *fingerprinting* determinista.

Lección aprendida: los sistemas de mensajería asíncrona presentan problemas de integración en entornos de desarrollo que no aparecen en pruebas unitarias; es esencial probar el stack completo desde el primer sprint.

Acción de mejora: reservar al menos un 20 % del tiempo de S6 para pruebas de regresión manual del flujo completo antes de la entrega final.

5.7 Sprint 6: Pruebas, refinamiento y documentación

Período: mayo–junio 2026 Duración: 8 semanas

5.7.1 Planificación del sprint

Objetivo del sprint: consolidar la plataforma con una batería de pruebas automatizadas, corregir los defectos detectados, refinar la interfaz de usuario y redactar la documentación técnica completa.

Cuadro 5.9: Sprint 6: Actividades y estimación

ID	Actividad	Est. (h)	Real (h)
–	Pruebas unitarias del motor de métricas y <i>Quality Score</i> (pytest)	18	22
–	Pruebas de integración: versionado, Quality Gates y autorización IDOR	14	16
–	Pruebas de <i>fingerprinting</i> y comparación con <i>baseline</i>	4	6
–	Corrección de defectos detectados en pruebas	16	18
–	Internacionalización del <i>frontend</i> (ES/EN, i18next)	0	—
–	Redacción de la memoria (TFG)	20	20
–	Preparación de la defensa	8	8
Total		80 h	90 h

5.7.2 Trabajo realizado

- Implementada una batería de pruebas automatizadas con `pytest` organizada en doce módulos de test que cubren autenticación (`test_auth`), API de proyectos (`test_projects_api`), versionado de datasets a nivel de servicio y de API (`test_dataset_versioning`, `test_dataset_versioning_api`), motor de métricas en sus versiones base y extendida (`test_metrics_engine`, `test_metrics_engine_extended`), Quality Score (`test_quality_score`), Quality Gates con umbrales y veredicto (`test_quality_gate`, `test_quality_gate_verdict`), *fingerprinting* (`test_fingerprinting`), comparación con línea base (`test_diff_baseline`) y autorización contra accesos indebidos (`test_idor_authorization`).
- Corregidos 11 defectos identificados durante las pruebas (issues de GitHub cerrados).
- Completada la documentación de la API REST en el Anexo A y el catálogo de métricas en el Anexo B.
- Implementada la internacionalización completa del *frontend* con `i18next` y `react-i18next`: dos idiomas (español e inglés), archivos de traducción de aproximadamente 1 500 entradas cada uno, integrada en 54 componentes y páginas mediante `useTranslation`, con persistencia del idioma seleccionado en `localStorage`.
- **Cambio respecto a lo planificado:** se detectaron tres defectos de severidad media en el módulo de *fingerprinting* que requirieron refactorización. La batería de pruebas se amplió respecto al plan inicial al incorporar suites adicionales para Quality Score, *fingerprinting*, *diff* contra línea base y autorización Insecure Direct Object Reference (IDOR); la internacionalización del *frontend* constituye asimismo una funcionalidad adicional no planificada inicialmente.

5.7.3 Cómo se ha hecho

Estrategia de pruebas

Las pruebas automatizadas se organizan en **doce módulos** en `backend/tests/` agrupados por dominio funcional:

- **Autenticación y API de proyectos (`test_auth.py`, `test_projects_api.py`):** verifican el flujo de autenticación completo (registro, inicio de sesión, refresco de *token*) y las operaciones CRUD sobre proyectos.
- **Versionado de datasets (`test_dataset_versioning.py`, `test_dataset_versioning_api.py`):** validan la relación padre-hijo entre versiones y la actualización del indicador `is_latest` ante cargas sucesivas, tanto a nivel de servicio como a través de los *endpoints* de la API. Comprueban que solo la versión más reciente tiene `is_latest=true` y que el historial se recupera ordenado.

- **Motor de métricas** (`test_metrics_engine.py`, `test_metrics_engine_extended.py`): cubren la ejecución unitaria de cada métrica del `METRIC_REGISTRY` sobre *DataFrames* sintéticos y casos límite (columnas vacías, tipos mixtos, valores extremos).
- **Quality Score** (`test_quality_score.py`): valida la fórmula del *score* y su penalización por número de *issues* sobre escenarios controlados.
- **Quality Gates** (`test_quality_gate.py`, `test_quality_gate_verdict.py`): verifican que los umbrales configurados producen los estados PASSED, WARNING y FAILED para distintas combinaciones de *score* e *issues*, incluyendo casos frontera (*score* exactamente en el umbral).
- **Fingerprinting y diff contra línea base** (`test_fingerprinting.py`, `test_diff_baseline.py`): comprueban que el *hash* es determinista bajo distintos órdenes de parámetros y representaciones flotantes, y que la clasificación de *issues* en nuevos, corregidos y recurrentes coincide con el resultado esperado al comparar con la *baseline*.
- **Autorización IDOR** (`test_idor_authorization.py`): verifica que un usuario no puede acceder a recursos (proyectos, datasets, evaluaciones) que pertenecen a otro usuario, comprobando que los *endpoints* devuelven 403/404 ante referencias directas no autorizadas a objetos.

Las pruebas se ejecutan con `pytest` desde `backend/` y pueden generar informes de cobertura con la opción `--cov`.

Defectos detectados y corregidos

La ejecución de las suites de pruebas reveló **11 defectos**, todos cerrados antes de la entrega final. Por categoría:

- **Módulo de fingerprinting (3 defectos, severidad media)**: el *hash* SHA-256 no era completamente determinista cuando los parámetros de la métrica contenían valores flotantes con distinta representación textual (0.95 vs. 0.950). Se corrigió normalizando los parámetros a cadena canónica antes del *hash*.
- **Métrica de consistencia lógica (2 defectos, severidad baja)**: reglas con columnas que contienen espacios en el nombre provocaban un error no controlado en `DataFrame.query()`. Se añadió un paso de sanitización del nombre de columna.
- **Versionado de datasets (2 defectos, severidad media)**: la carga concurrente de dos versiones del mismo dataset podía dejar más de un registro con `is_latest=true`. Se resolvió encapsulando la actualización del indicador y la inserción del nuevo registro en una transacción atómica de `SQLAlchemy`.
- **API REST, paginación (2 defectos, severidad baja)**: el parámetro `per_page` no estaba acotado superiormente, permitiendo consultas que devolvían todos los registros sin

límite. Se añadió un límite superior por *endpoint* (entre 100 y 200 registros según el volumen esperado de cada recurso).

- **Evaluación asíncrona (2 defectos, severidad baja):** el *Evaluation Watchdog* marcaba como fallidas evaluaciones que simplemente estaban pausadas esperando un *worker* Celery disponible. Se ajustaron los *healthchecks* y los tiempos de reconexión de Celery para distinguir las evaluaciones realmente atascadas de las que esperaban disponibilidad del *worker*.

5.7.4 Retrospectiva

Conseguido: plataforma completamente probada, documentada y lista para la defensa. Internacionalización completa del *frontend* (ES/EN). Las tres suites de pruebas automatizadas cubren los flujos de autenticación, versionado de datasets y *Quality Gates*.

Estado del producto: DATAQUAL es una plataforma funcional, probada y documentada. Tres suites de pruebas automatizan la verificación de las métricas, el versionado y los *Quality Gates*. La memoria del TFG está redactada y los siete anexos completos. La cobertura del *frontend* sigue siendo insuficiente, lo que queda registrado como deuda técnica en las líneas de trabajo futuro.

Pendiente (trabajo futuro): Swagger/OpenAPI, exportación de informes, WebSockets y pruebas *end-to-end* del *frontend*; documentados en la sección 7.4.

Qué fue bien: la batería de pruebas unitarias detectó tres defectos en el módulo de fingerprinting que habrían pasado desapercibidos, demostrando el valor de las pruebas automatizadas.

Qué fue mal: la cobertura de pruebas del *frontend* es escasa; la priorización de funcionalidad sobre pruebas durante los sprints anteriores dejó la interfaz de usuario sin cobertura automatizada.

Estimación vs. real: 80 h estimadas vs. 90 h reales (+12,5 %).

Lección aprendida: las pruebas deben integrarse en cada sprint, no dejarse para el final. Un sprint de consolidación es útil, pero no puede compensar la deuda técnica de pruebas acumulada.

Acción de mejora (para proyectos futuros): adoptar una política de *definition of done* que exija al menos pruebas unitarias de los módulos nuevos antes de cerrar cada historia de usuario.

Resumen global del proyecto: 364 h estimadas vs. 425 h reales (+16,8 % de desviación total), dentro del rango habitual para proyectos de software de esta envergadura.

Capítulo 6

Resultados y discusión

Este capítulo consolida los resultados globales de DATAQUAL una vez completados los seis sprints descritos en el capítulo 5. Presenta los resultados cuantitativos y cualitativos del sistema, verifica el cumplimiento de los objetivos específicos OE1–OE5, identifica las fortalezas y limitaciones de la plataforma, y la compara con las herramientas existentes revisadas en el capítulo 2.

6.1 Resultados alcanzados

DATAQUAL es una plataforma funcional de extremo a extremo que cumple con los cinco objetivos específicos definidos en la sección 3.2.

6.1.1 Resultados cuantitativos

El Cuadro 6.1 resume las principales magnitudes del sistema implementado.

Cuadro 6.1: Resultados cuantitativos de la implementación

Dimensión	Cantidad
Métricas de calidad evaluables (Quality Score)	7
Clases de métricas implementadas (incl. outliers para EDA)	8
Servicios Docker orquestados	8
Rutas / páginas del <i>frontend</i>	20
Componentes React reutilizables	50+
Módulos funcionales de API (<i>blueprints</i> principales)	7
<i>Blueprints</i> Flask totales (incl. sub-rutas REST)	10
Tablas en base de datos	11
Patrones de formato predefinidos (exactitud sintáctica)	13
Tipos de <i>issue</i> (por métrica y subtipo)	10+
Migraciones de base de datos	6
Módulos de pruebas automatizadas (pytest)	12

La batería de pruebas automatizadas se organiza en doce módulos pytest agrupados por dominio funcional, cubriendo autenticación, API de proyectos, versionado de datasets a nivel de servicio y de API, motor de métricas en sus versiones base y extendida, *Quality Score*, *Quality Gates* con umbrales y veredicto, *fingerprinting*, comparación con línea base

y autorización contra accesos indebidos (IDOR); el detalle de cada suite se describe en la sección 5.7.3. Las pruebas se ejecutan con `pytest` desde el directorio `backend/` y pueden generar informes de cobertura mediante la opción `--cov`. Estas suites unitarias y de integración se complementaron con pruebas manuales de extremo a extremo sobre los flujos completos de la plataforma (registro, creación de proyecto, carga de dataset, configuración de métricas, ejecución de evaluaciones y consulta de resultados), que verificaron la correcta integración de todos los componentes en escenarios realistas. La automatización de estas pruebas *end-to-end* con herramientas como Playwright o Cypress queda como tarea pendiente y se identifica explícitamente como limitación en la sección 6.3.

6.1.2 Esfuerzo y desviaciones

El Cuadro 6.2 consolida la estimación inicial de horas frente al esfuerzo real invertido en cada sprint.

Cuadro 6.2: Esfuerzo estimado vs. real por sprint

Sprint	Período	SP	H. est.	H. real	Desv.
S1 — Arquitectura y setup	oct.–nov. 2025	13	52	60	+15,4 %
S2 — Backend core	dic. 2025–ene. 2026	18	72	85	+18,1 %
S3 — Motor de métricas	feb. 2026	16	64	70	+9,4 %
S4 — Frontend y dashboard	mar. 2026	14	56	65	+16,1 %
S5 — Funcionalidades avanzadas	abr. 2026	10	40	55	+37,5 %
S6 — Pruebas y documentación	may.–jun. 2026	—	80	90	+12,5 %
Total		71	364	425	+16,8 %

La desviación global del +16,8 % se mantiene dentro del rango habitual para proyectos de software individuales. El sprint con mayor desviación fue el S5 (+37,5 %), causado por problemas de integración entre el sistema de *fingerprinting* y el módulo de comparación con la línea base que no fueron detectados en la planificación. El sprint con menor desviación fue el S3 (+9,4 %), cuyo motor de métricas aprovechó la arquitectura de *plugins* establecida en S2, reduciendo el esfuerzo de integración previsto. La tendencia a subestimar el esfuerzo es consistente en todos los sprints y refleja la dificultad de estimar tareas de integración entre servicios distribuidos.

6.1.3 Verificación del cumplimiento de objetivos

El Cuadro 6.3 recoge, para cada objetivo específico, la evidencia de su cumplimiento.

Los cinco objetivos se cumplen con el sistema tal como está desplegado. El único objetivo superado es OE3: la API creció de seis a siete *blueprints* funcionales durante el desarrollo, y los *blueprints* de organización técnica elevaron el total a diez. Los objetivos transversales de seguridad, escalabilidad, mantenibilidad y portabilidad se cumplen; las pruebas *end-to-end* del *frontend* quedan como trabajo futuro (sección 7.4).

Cuadro 6.3: Verificación del cumplimiento de los objetivos específicos

Obj.	Evidencia	Estado
OE1	Módulo de ingesta en formato CSV con parseo robusto mediante matriz de <i>fallback</i> (codificaciones × separadores); almacenamiento en MinIO; extracción automática de esquema; versionado padre-hijo con etiquetas personalizables.	Cumplido
OE2	Siete métricas implementadas como <i>plugins</i> ; parametrización completa; sistema de pesos; plantillas reutilizables; ejecución asíncrona con Celery.	Cumplido
OE3	Seis <i>blueprints</i> funcionales planificados, ampliados a siete con la adición de <i>patterns</i> (validación reutilizable); autenticación JWT; formato de respuesta estandarizado; paginación; validación Marshmallow.	Superado
OE4	Más de 20 páginas; cuadros de mando interactivos; EDA con histogramas, diagramas de caja y matrices de correlación; diseño adaptable.	Cumplido
OE5	Sistema de <i>issues</i> con cuatro severidades; <i>fingerprinting</i> SHA-256; comparación con línea base; <i>Quality Gates</i> con tres estados.	Cumplido

6.2 Fortalezas de la plataforma

A partir de los resultados consolidados en la sección anterior, las fortalezas de DATAQUAL se pueden agrupar en cuatro ejes: una base arquitectónica extensible, un conjunto de capacidades funcionales diferenciadoras, una integración nativa de la protección de información personal y un modelo de despliegue y operación robusto.

La principal fortaleza arquitectónica reside en el **patrón de *plugins* del motor de métricas**, materializado mediante la clase abstracta `BaseMetric` y el diccionario `METRIC_REGISTRY` (Gamma et al., 1994). Esta separación entre la interfaz pública del motor y las implementaciones concretas permite incorporar una nueva métrica sin modificar el código existente: basta con crear una clase heredera, registrarla en el diccionario y, opcionalmente, exponerla en la interfaz de configuración. Su valor quedó demostrado durante el propio desarrollo, en el que el catálogo creció de tres métricas en el Sprint 3 a siete al cierre del Sprint 5 sin necesidad de reescribir el orquestador de evaluaciones. La misma filosofía de extensibilidad se traslada al *frontend*, organizado en torno a más de cincuenta componentes React reutilizables que permiten ensamblar nuevas pantallas con bajo coste marginal.

Sobre esa base arquitectónica, DATAQUAL incorpora cuatro capacidades funcionales que la diferencian de las herramientas analizadas en el capítulo 2. El **sistema de *fingerprinting***, basado en un *hash* SHA-256 truncado y determinista por *issue*, permite trazar la evolución de un mismo problema entre ejecuciones sucesivas y clasificar los *issues* automáticamente como nuevos, recurrentes o corregidos: una capacidad poco común en herramientas de código abierto, que típicamente reportan listados planos sin continuidad temporal. Los ***Quality Gates*** trasladan al dominio de los datos un concepto consolidado en la calidad del código (SonarSource, 2024), ofreciendo un veredicto PASSED/WARNING/FAILED automati-

zable que puede integrarse en *pipelines* de MLOPS como criterio de promoción de datasets entre entornos. El **módulo de perfilado interactivo** concentra en una sola pantalla histogramas, diagramas de caja, matrices de correlación de Pearson y Spearman y detección de *outliers* con factor IQR configurable, sin obligar al usuario a abandonar la plataforma. Por último, las **métricas específicas para machine learning** (equilibrio de clases basado en entropía de Shannon y diversidad del *dataset*) responden a necesidades del ciclo de vida de modelos que las herramientas generalistas de calidad de datos no cubren.

La **protección nativa de información personal identificable (PII)** es la fortaleza con mayor coste de implementación pero también la de mayor valor diferencial en contextos regulados por el RGPD. Lejos de tratarse de una capa filtro añadida *a posteriori*, el enmascaramiento se ha implementado como un contrato que toda métrica debe respetar: las clases heredan de BaseMetric un método de enmascaramiento que se invoca antes de cualquier escritura de un valor a logs, *issues* o respuestas de la API, garantizando que ninguna columna marcada como sensible filtre valores a través de cualquier camino del sistema. Esta integración estructural marca la diferencia entre «la herramienta puede ofuscar PII si se configura» y «la herramienta no puede revelar PII por construcción».

Desde el punto de vista operativo, la plataforma se distingue por ser **íntegramente auto-alojada y desplegable con un único comando** (`docker-compose up`): los ocho servicios se levantan en una red interna, sin dependencia de servicios en la nube propietarios y manteniendo los datos del usuario bajo su control total. La capa de ingesta incorpora un **parseo robusto de CSV** mediante una matriz de *fallback* que combina cuatro codificaciones, seis estrategias de separador y los dos motores de Pandas, capaz de gestionar ficheros reales con BOM, codificaciones mixtas o delimitadores poco habituales que provocan errores en analizadores estándar. El **procesamiento asíncrono con Celery** (Celery Project, 2024) se complementa con un *watchdog* que detecta evaluaciones atascadas y un servicio híbrido que recurre al modo síncrono ante indisponibilidad del *worker*, asegurando que las evaluaciones se completen incluso en escenarios degradados.

6.3 Limitaciones

A pesar de los resultados alcanzados, la plataforma presenta limitaciones que conviene reconocer explícitamente. Estas se pueden agrupar en tres ámbitos: la madurez de los servicios de plataforma (autenticación, entrega de informes y documentación de la API), la escalabilidad y el rendimiento bajo cargas elevadas, y la cobertura de pruebas *end-to-end*.

El conjunto de limitaciones más relevantes para un escenario de producción afecta a los **servicios de plataforma**. La autenticación se limita a JWT con usuario y contraseña: no se han implementado OAuth 2.0, *Single Sign-On* corporativo ni autenticación multifactor, por lo que la plataforma no está preparada todavía para integrarse en entornos empresariales con identidad federada. Los resultados, además, no pueden exportarse a formatos de informe

formal como PDF o Excel, lo que dificulta el reporte a perfiles ajenos al equipo técnico y la incorporación de los hallazgos a documentación de auditoría. Finalmente, la API REST carece de documentación interactiva tipo Swagger/OpenAPI generada automáticamente: el contrato se mantiene íntegramente en el Anexo A de esta memoria, lo que ralentiza la integración por parte de equipos externos y obliga a mantener manualmente la sincronía entre código y documentación.

Otro grupo de limitaciones afecta a la **escalabilidad y al rendimiento** bajo cargas elevadas. El *frontend* obtiene el estado de las evaluaciones mediante *polling* periódico en lugar de notificaciones *push* por *WebSocket*: aunque el patrón es funcionalmente correcto y simplifica la arquitectura, introduce latencia perceptible y carga innecesaria contra la API cuando se ejecutan muchas evaluaciones simultáneamente. La capa de procesamiento, además, no se ha validado con múltiples *workers* de Celery en paralelo, por lo que la escalabilidad horizontal que el diseño permite es todavía teórica. El motor de métricas opera sobre *DataFrames* de Pandas cargados íntegramente en memoria, lo que impone como cota superior el tamaño de la RAM disponible y descarta procesar *datasets* de varios GB sin migrar a un motor distribuido como Dask o Spark. Por último, la detección de valores atípicos en el módulo de perfilado se limita al método estadístico clásico del IQR; algoritmos basados en aprendizaje automático como *Isolation Forest* o DBSCAN, que detectarían anomalías multidimensionales más sutiles, quedan fuera del alcance de esta versión.

Por último, la cobertura de **pruebas automatizadas**, aunque amplía en componentes individuales (motor de métricas, *Quality Score*, *Quality Gates*, *fingerprinting*, comparación con línea base, autorización IDOR y versionado de *datasets*), no incluye pruebas *end-to-end* que validen el flujo completo desde la subida de un *dataset* hasta la emisión del veredicto del *Quality Gate*. Esta validación se realizó de forma manual durante el desarrollo, pero su automatización con herramientas como Playwright o Cypress queda como tarea pendiente y constituye la mayor brecha en la estrategia de aseguramiento de calidad del propio proyecto.

6.4 Comparación con herramientas existentes

El Cuadro 6.4 retoma la comparación del capítulo 2 enriquecida con perspectiva de implementación: no solo si la funcionalidad existe, sino qué implicó reproducirla o superarla en la práctica.

La implementación aportó perspectivas que no eran evidentes en el análisis previo del estado del arte. El **parseo robusto de archivos** resultó ser un diferenciador real: en las pruebas con conjuntos de datos reales, una fracción significativa de los ficheros CSV presentó combinaciones de codificación y delimitador no estándar que ninguna herramienta *code-first* gestiona automáticamente sin configuración explícita. El **enmascaramiento nativo de PII** supuso un coste de implementación considerable —todas las clases de métricas deben

Cuadro 6.4: Comparación de DATAQUAL con herramientas existentes (perspectiva post-implementación)

Criterio	DATAQUAL	Great Exp.	Deequ	DQOps	Soda
UI integrada	Sí	No	No	Sí	Parcial ^a
Requiere código	No	Sí	Sí	Parcial	No
<i>Quality Gates</i>	Sí	No	No	Sí	Sí
<i>Fingerprinting</i>	Sí	No	No	Parcial	No
Diff entre ejecuciones	Sí	No	Parcial	Sí	Sí
Métricas para ML	Sí	No	Parcial	No	No
Enmascaramiento PII	Sí	No	No	No	No
Versionado de datasets	Sí	No	No	No	No
Autoalojado	Sí	Sí	Sí	Sí	Sí
Parseo robusto de archivos	Sí	Parcial	No	No	Parcial
Madurez	Prototipo	Producción	Producción	Producción	Producción

^a La interfaz web de Soda (Soda Cloud) está disponible únicamente en el nivel comercial.

respetar la lista de columnas sensibles —pero su valor es especialmente alto en contextos regulados por el RGPD, donde ninguna de las alternativas analizadas lo ofrece de forma integrada.

DATAQUAL no pretende competir con herramientas maduras de producción como Great Expectations (Great Expectations, 2024) en escala o número de métricas predefinidas, pero ocupa un nicho diferenciado al combinar interfaz web completa, métricas específicas para ML, *fingerprinting* determinista, *Quality Gates* y protección nativa de PII en un paquete autoalojado que no requiere escribir código.

Capítulo 7

Conclusiones y trabajo futuro

Este capítulo final recoge la revisión punto por punto de los objetivos del proyecto, las conclusiones y valoración global, las lecciones aprendidas, las líneas de trabajo futuro, las competencias adquiridas y una valoración personal del proceso.

7.1 Revisión de objetivos

El presente TFG estableció cinco objetivos específicos (OE1–OE5) y un conjunto de objetivos transversales. A continuación se revisa el grado de cumplimiento de cada uno.

OE1 — Módulo de ingesta y almacenamiento Cumplido. Se implementó soporte para CSV con estrategia de *fallback* múltiple (4 codificaciones $\times$ 5 delimitadores), almacenamiento seguro en MinIO, extracción automática de esquema y versionado padre-hijo con indicador `is_latest`.

OE2 — Motor de métricas parametrizables Cumplido. Se implementaron siete métricas de calidad evaluables (completitud, unicidad, exactitud sintáctica, consistencia lógica, actualidad, equilibrio de clases y diversidad) mediante arquitectura de *plugins* (`BaseMetric` + `METRIC_REGISTRY`), con parametrización completa, sistema de pesos y plantillas reutilizables. La clase `OutliersMetric` se implementó como herramienta de diagnóstico disponible en el módulo de perfilado, sin participar en el *Quality Score* (alineado con ISO/IEC 5259).

OE3 — API RESTful con especificación formal Cumplido y superado. Se implementaron los seis módulos funcionales definidos como *blueprints* Flask principales (`auth`, `projects`, `datasets`, `metrics`, `evaluations`, `admin`), con autenticación JWT, formato de respuesta estandarizado `{success, data, message}`, paginación y validación `Marshmallow`. Durante el Sprint 5 se incorporó como funcionalidad emergente un séptimo *blueprint* funcional, `patterns`, que expone la gestión de patrones de validación reutilizables entre proyectos. Adicionalmente, la convención REST de rutas anidadas añadió tres *blueprints* de organización técnica (`project_metrics`, `project_datasets`, `dashboard`), totalizando **diez** *blueprints* registrados. La especificación completa se recoge en el Anexo A.

OE4 — Aplicación web con cuadros de mando interactivos Cumplido. Se desarrollaron

más de 20 páginas con cuadros de mando, visualizaciones interactivas (*EDA*: histogramas, boxplots, matrices de correlación) y diseño adaptable.

OE5 — Mecanismos de identificación de problemas Cumplido. Se implementaron el sistema de *issues* con cuatro niveles de severidad, el *fingerprinting* SHA-256 para trazabilidad entre evaluaciones y los *Quality Gates* con tres estados (PASSED/WARNING/FAILED).

Objetivos transversales Cumplidos en su mayoría. Seguridad multicapa (JWT, PBKDF2, rate limiting, cabeceras), escalabilidad asíncrona (Celery + Redis), mantenibilidad (módulos independientes), portabilidad (Docker Compose) e internacionalización (ES/EN con i18next) han sido alcanzados. Las pruebas *end-to-end* del *frontend* quedan como trabajo futuro.

Valoración global del proyecto: DATAQUAL es una plataforma funcional de extremo a extremo que supera en funcionalidades específicas para ML (equilibrio de clases, *fingerprinting*, *Quality Gates*, PII nativo) a las alternativas open source revisadas. Su madurez es la de un prototipo funcional, no de una herramienta de producción, lo cual está alineado con el alcance de un TFG.

Limitaciones encontradas: ausencia de Swagger/OpenAPI, sin exportación de informes, *polling* en lugar de WebSockets, escalabilidad limitada a un nodo y cobertura de pruebas del *frontend* insuficiente.

7.2 Conclusiones

El presente TFG ha abordado el diseño, la implementación y la validación de DATAQUAL, una plataforma web integral para la evaluación automatizada de la calidad de datos en proyectos de IA. A lo largo de los seis sprints descritos en el capítulo 5, se ha logrado construir un sistema funcional de extremo a extremo que cumple los cinco objetivos específicos establecidos en el capítulo 3, tal como se ha verificado en la sección 6.1.

Las conclusiones que se extraen del trabajo se articulan en torno a cuatro ejes: la respuesta a una necesidad práctica del ecosistema actual de calidad de datos, el valor de los estándares internacionales como marco de rigor metodológico, la validez de las decisiones arquitectónicas fundacionales y la contribución diferencial de las dos funcionalidades distintivas de la plataforma.

En primer lugar, el proyecto confirma que **la calidad de datos para IA requiere herramientas más accesibles** que los enfoques *ad hoc* basados en *scripts* y cuadernos Jupyter. Estas aproximaciones, aunque flexibles, presentan barreras de estandarización, historización y accesibilidad que dificultan su adopción sistemática en organizaciones donde los responsables de la calidad no son perfiles puramente técnicos. DATAQUAL demuestra que es viable ofrecer una interfaz web accesible sin renunciar a la potencia de una API RESTful programática, conciliando ambos modos de uso bajo el mismo sistema.

En segundo lugar, **los estándares internacionales han demostrado ser un marco sólido para el diseño de métricas**. La alineación con ISO/IEC 25012 (ISO/IEC, 2008) e ISO/IEC 5259 (ISO/IEC, 2024) ha evitado definiciones arbitrarias y ha aportado un vocabulario común que facilita la comparabilidad entre proyectos. La adopción explícita de estándares como referencia, lejos de constreñir el diseño, ha aportado claridad a decisiones que de otro modo habrían requerido justificación *ad hoc*.

En tercer lugar, **las decisiones arquitectónicas fundacionales se han demostrado acertadas**. La arquitectura de *plugins* del motor de métricas, basada en una clase base abstracta y un registro dinámico, permitió incorporar nuevas métricas en sprints sucesivos sin tocar el orquestador de evaluaciones, materializando en la práctica el principio de abierto-cerrado (Gamma et al., 1994): las siete métricas evaluables se implementaron de forma independiente y el proceso para añadir una nueva quedó formalizado en cuatro pasos. La contenerización con Docker Compose (Docker, Inc., 2024), por su parte, garantiza que la plataforma pueda reproducirse en cualquier entorno sin enviar datos a servicios externos, alineando el diseño con los requisitos de soberanía del dato propios del RGPD.

Por último, las dos funcionalidades distintivas de la plataforma (el sistema de *fingerprinting* y los *Quality Gates*) aportan capacidades que no están presentes en la mayoría de las herramientas de código abierto analizadas en el estado del arte. El *fingerprinting* habilita la trazabilidad de *issues* entre ejecuciones mediante *hashes* deterministas que permiten clasificar automáticamente cada problema como nuevo, recurrente o corregido. Los *Quality Gates*, inspirados en el concepto homónimo de SonarQube (SonarSource, 2024), trasladan a la calidad de datos una señal objetiva y automatizable (PASSED, WARNING, FAILED) sobre la aptitud de un conjunto de datos para su uso, integrable en *pipelines* de MLOPS como criterio de promoción entre entornos.

7.3 Lecciones aprendidas

El desarrollo del proyecto ha proporcionado lecciones que cabe agrupar en dos planos: el metodológico, relativo a cómo se organiza el trabajo, y el técnico, relativo a las decisiones de diseño que conviene tomar pronto.

En el plano metodológico, la lección principal es que el **desarrollo iterativo no es solo una metodología formal, sino una forma de pensar que reduce el riesgo real de los proyectos**. La organización en iteraciones con entregas funcionales permitió detectar y corregir problemas de diseño de forma temprana, evitando retrabajos costosos en fases avanzadas. Paralelamente, el desarrollo confirmó que **el diseño de métricas de calidad de datos no es un ejercicio puramente algorítmico**: requiere comprender los estándares internacionales, las necesidades reales de los usuarios y los matices de cada dimensión de calidad (qué significa exactamente «completitud» cuando una columna tiene valores codificados como cadenas vacías, etiquetas N/A o ceros con sentido distinto al de nulo). Sin este trabajo previo

de modelado del dominio, las métricas resultantes habrían sido superficiales o francamente engañosas.

En el plano técnico, dos decisiones tomadas pronto resultaron especialmente rentables. **La seguridad debe integrarse desde el inicio del diseño**, no añadirse como capa posterior: decisiones como la protección contra inyección en las reglas de consistencia lógica o el enmascaramiento nativo de PII fueron mucho más sencillas de implementar al considerarlas desde las fases iniciales que si hubiera sido necesario reconstruir el sistema sin ellas. Por contraste, **la gestión de la complejidad del *frontend* requiere una disciplina de modularización que conviene aplicar antes de que duela**: componentes como el módulo de *profiling* de datos crecieron significativamente durante el desarrollo y, en retrospectiva, una mayor división temprana en subcomponentes habría facilitado el mantenimiento en las iteraciones finales. La asimetría entre ambas lecciones resulta reveladora: lo que se asume cuidar desde el principio (seguridad) sale bien, mientras que lo que se posterga (refactor del *frontend*) acaba doliendo.

7.4 Líneas de trabajo futuro

A partir de las limitaciones identificadas en la sección 6.3 y de las oportunidades detectadas durante el desarrollo, las líneas de evolución de DATAQUAL se articulan en cuatro ejes: documentación y comunicación de resultados, escalabilidad y rendimiento, integraciones empresariales, y aseguramiento de calidad e inteligencia analítica.

En el eje de **documentación y comunicación** se sitúa la incorporación de documentación interactiva tipo Swagger/OpenAPI para la API RESTful, generada automáticamente a partir del código y consultable en línea, que aceleraría la integración por parte de equipos externos. Complementariamente, la exportación de informes a PDF y Excel facilitaría la comunicación de resultados a *stakeholders* no técnicos y la incorporación de los hallazgos a documentación de auditoría; y la incorporación de *webhooks* permitiría notificar a sistemas externos cuando una evaluación se complete o un *Quality Gate* falle, integrando DATAQUAL en flujos automatizados de MLOPS.

El eje de **escalabilidad y rendimiento** concentra tres mejoras con cota técnica clara. La sustitución del mecanismo actual de *polling* por notificaciones *push* con *WebSockets* reduciría tanto la latencia perceptible como la carga innecesaria sobre la API. El procesamiento por fragmentos (*chunk-based*) elevaría la cota actual del tamaño de RAM disponible y abriría la plataforma a *datasets* de varios GB sin necesidad de migrar a un motor distribuido como Dask o Spark. Por último, las evaluaciones programadas mediante Celery Beat (servicio ya disponible en la pila tecnológica pero todavía sin interfaz de usuario) habilitarían la monitorización continua de la calidad de datos sin intervención manual.

El tercer eje, **integraciones empresariales**, recoge las limitaciones que separan a la plataforma de un escenario corporativo real. La incorporación de OAuth 2.0 y *Single Sign-On*

permitiría integrar DATAQUAL en sistemas empresariales con identidad federada. La integración directa con bases de datos relacionales y analíticas (PostgreSQL, MySQL, BigQuery) como fuentes de datos eliminaría la necesidad de exportar previamente los *datasets* a fichero. Y la colaboración en equipo con roles y permisos granulares haría posible que múltiples usuarios trabajaran sobre los mismos proyectos, completando el modelo actual basado en un único propietario por proyecto.

Por último, el eje de **aseguramiento de calidad e inteligencia analítica** aborda tanto las pruebas del propio software como la sofisticación analítica del producto. La automatización de pruebas *end-to-end* con Cypress o Playwright cubriría los flujos principales de la interfaz y cerraría la mayor brecha actual en la estrategia de Quality Assurance (QA) del proyecto. En el plano analítico, la incorporación de algoritmos de detección de anomalías basados en aprendizaje automático (*Isolation Forest*, *Autoencoders*) ampliaría el catálogo de métricas más allá del método estadístico clásico IQR, mientras que las sugerencias de corrección automatizadas sobre los *issues* detectados cerrarían el ciclo de diagnóstico con propuestas concretas de acción.

7.5 Aportación del proyecto

Desde el punto de vista técnico, DATAQUAL aporta al panorama de herramientas de calidad de datos una solución *full-stack* autoalojada que combina en un único producto una interfaz web accesible sin necesidad de programar, métricas específicas para proyectos de ML (equilibrio de clases, diversidad del dataset), sistema de *fingerprinting* determinista para la trazabilidad de *issues*, *Quality Gates* configurables y protección nativa de PII, una combinación que no está disponible en ninguna de las herramientas de código abierto analizadas en el estado del arte (sección 2.4).

Desde el punto de vista académico, el proyecto materializa la convergencia entre la ingeniería del software moderna (arquitecturas de microservicios, patrones de diseño como *plugin* y factoría) y el dominio emergente de la calidad de datos para IA, proporcionando una implementación de referencia alineada con los estándares ISO/IEC 25012 e ISO/IEC 5259 que puede servir como base para investigación futura sobre métricas de calidad específicas para ML.

Desde el punto de vista profesional, el proyecto ha permitido al autor desarrollar competencias en un conjunto amplio de tecnologías de producción (desarrollo *full-stack* con Flask y Next.js, bases de datos relacionales con JSONB, almacenamiento de objetos S3, colas de tareas asíncronas con Celery y Redis, y despliegue contenerizado con Docker Compose) en un contexto de proyecto real con requisitos de seguridad, escalabilidad y mantenibilidad.

7.6 Competencias adquiridas

El desarrollo de este TFG en la intensificación de Sistemas de Información ha permitido adquirir y consolidar las siguientes competencias:

- **Desarrollo *full-stack*:** diseño e implementación de una aplicación web completa con separación clara entre *frontend* (Next.js, React, TypeScript) y *backend* (Python, Flask, SQLAlchemy).
- **Arquitecturas distribuidas y contenerización:** orquestación de ocho servicios Docker con Docker Compose, gestión de dependencias entre servicios, *healthchecks* y volúmenes persistentes.
- **Procesamiento de datos tabulares:** manipulación de datasets en múltiples formatos con Pandas, cálculo de métricas estadísticas y gestión de casos extremos (nulos, tipos mixtos, codificaciones).
- **Calidad de datos e ingeniería de la calidad:** comprensión profunda de los estándares ISO/IEC 25012 e ISO/IEC 5259 y su aplicación práctica en el diseño de métricas.
- **Seguridad en aplicaciones web:** autenticación JWT, *hashing* de contraseñas con PBKDF2, limitación de tasa, cabeceras de seguridad y prevención de inyección de código.
- **Diseño de APIs REST:** especificación y documentación de *endpoints*, gestión de versiones, paginación y validación de entradas.
- **Gestión ágil con Scrum adaptado:** planificación de sprints, gestión del backlog con GitHub Issues, retrospectivas y adaptación del plan ante imprevistos.
- **Control de versiones avanzado:** estrategia de *feature branches*, mensajes de *commit* descriptivos y trazabilidad entre código e issues.
- **Estimación y planificación:** uso de *story points*, análisis de desviaciones y ajuste de estimaciones a lo largo del proyecto.
- **Documentación técnica:** redacción de una memoria de TFG completa en $\text{\LaTeX}$, especificaciones de API y catálogos de métricas.
- **Resolución autónoma de problemas:** diagnóstico y resolución de problemas de integración entre servicios Docker, incompatibilidades de formato de datos y gestión de errores en sistemas distribuidos.
- **Capacidad de síntesis y abstracción:** diseño de interfaces que permiten extender el sistema (motor de métricas, arquitectura de *plugins*) sin conocer los detalles de las implementaciones concretas.
- **Trabajo autónomo bajo supervisión:** gestión del tiempo y la priorización en un proyecto individual de nueve meses con reuniones de seguimiento periódicas.

7.7 Valoración personal

Este TFG ha supuesto un reto de una dimensión cualitativamente diferente a los proyectos realizados durante el grado. Por primera vez, he tenido que diseñar y construir un sistema completo desde cero, tomando decisiones de arquitectura con consecuencias a largo plazo, integrando tecnologías que no había utilizado antes (Celery, MinIO, Redis en producción) y manteniendo la coherencia de un proyecto de nueve meses en solitario.

Lo que más me ha sorprendido es la complejidad inherente a los detalles: los ficheros CSV del mundo real no se comportan como los de los tutoriales, la configuración de Docker Compose con dependencias entre servicios exige una comprensión profunda de los tiempos de arranque, y el diseño de un *fingerprint* determinista para issues requirió mucho más pensamiento del esperado. Estas dificultades, en retrospectiva, son precisamente las más valiosas porque son las que ocurren en proyectos reales.

Me llevo especialmente la convicción de que una buena arquitectura inicial (el modelo de datos pensado desde el principio, la arquitectura de *plugins* para el motor de métricas) ahorra enormemente el trabajo en sprints posteriores, y que el desarrollo iterativo no es solo una metodología formal sino una forma de pensar que reduce el riesgo real de los proyectos.

Estoy satisfecho con el resultado: DATAQUAL es una plataforma funcional, segura y extensible que cubre un hueco real en el ecosistema de herramientas de calidad de datos para IA. Queda trabajo por hacer (Swagger, WebSockets, pruebas de *frontend*), pero eso es precisamente lo que convierte un TFG en el punto de partida de algo mayor.

ANEXOS

Anexo A

Especificación de la API RESTful

Este anexo documenta los principales *endpoints* de la API RESTful de DATAQUAL, organizados por módulo (*blueprint*). Todos los *endpoints* protegidos requieren un *token* JWT válido en la cabecera `Authorization: Bearer <token>`. Las respuestas siguen el formato estandarizado `{success, data, message}`.

A.1 Módulo de autenticación (`/api/auth`)

POST `/api/auth/register`

Registro de un nuevo usuario. Requiere `username`, `email` y `password`.

POST `/api/auth/login`

Inicio de sesión. Devuelve *access_token* y *refresh_token*.

POST `/api/auth/refresh`

Refresco del *access token* utilizando el *refresh token*.

POST `/api/auth/logout`

Cierre de sesión. Revoca el *token* actual.

GET `/api/auth/profile`

Consulta del perfil del usuario autenticado.

PUT `/api/auth/profile`

Actualización del perfil del usuario.

A.2 Módulo de proyectos (`/api/projects`)

GET `/api/projects/`

Lista de proyectos del usuario autenticado, con conteo de conjuntos de datos y puntuación de la última evaluación. Soporta paginación.

POST `/api/projects/`

Creación de un nuevo proyecto. Requiere `name` y, opcionalmente, `description`.

GET `/api/projects/<id>`

Detalle de un proyecto.

PUT /api/projects/<id>

Actualización de un proyecto (nombre, descripción, configuración de métricas).

DELETE /api/projects/<id>

Eliminación de un proyecto y todos sus recursos asociados (conjuntos de datos, evaluaciones, *issues*).

GET /api/projects/<id>/metrics/config

Consulta de la configuración de métricas del proyecto.

POST /api/projects/<id>/metrics/config

Actualización de la configuración de métricas del proyecto.

A.3 Módulo de conjuntos de datos (/api/datasets)

POST /api/projects/<id>/datasets

Carga de un nuevo conjunto de datos (formulario *multipart*). Acepta archivos en formato CSV.

GET /api/datasets/<id>

Consulta de metadatos, esquema y estadísticas de un conjunto de datos.

GET /api/datasets/<id>/versions

Historial de versiones de un conjunto de datos.

PUT /api/datasets/<id>/sensitive-columns

Actualización de las columnas marcadas como PII.

GET /api/datasets/<id>/download

Descarga del archivo original.

A.4 Módulo de métricas (/api/metrics)

GET /api/metrics/

Catálogo de métricas disponibles con sus parámetros de configuración.

GET /api/metrics/templates

Lista de plantillas de métricas (del sistema y del usuario).

POST /api/metrics/templates

Creación de una nueva plantilla de métricas.

DELETE /api/metrics/templates/<id>

Eliminación de una plantilla del usuario.

A.5 Módulo de evaluaciones (/api/evaluations)

POST /api/evaluations/

Lanzamiento de una nueva evaluación. Requiere `project_id` y `dataset_id`.

GET /api/evaluations/<id>

Consulta de los resultados completos de una evaluación: métricas, puntuación, *issues* y estado del *Quality Gate*.

GET /api/evaluations/<id>/status

Consulta del estado y progreso de una evaluación en curso.

GET /api/evaluations/<id>/issues

Lista de *issues* detectados en la evaluación, con severidad, descripción, columnas afectadas y muestras de filas.

GET /api/projects/<id>/evaluations

Historial de evaluaciones de un proyecto, con puntuaciones y estados de *Quality Gate*.

A.6 Módulo de administración (/api/admin)

GET /api/admin/health

Comprobación de salud del sistema. Verifica la conectividad con PostgreSQL, MinIO y Redis.

GET /api/admin/performance

Estadísticas de rendimiento por *endpoint*: número de peticiones, tiempo medio, mínimo y máximo.

Anexo B

Catálogo de métricas de calidad

Este anexo recoge el catálogo completo de las métricas de calidad de datos implementadas en DATAQUAL. El sistema define **ocho clases de métricas** organizadas en cuatro categorías:

- **Métricas fundamentales** (siempre puntuables): completitud, unicidad, exactitud sintáctica y actualidad.
- **Métricas condicionales** (*opt-in*): equilibrio de clases y diversidad —solo contribuyen al *Quality Score* cuando el usuario configura explícitamente las columnas a evaluar.
- **Consistencia lógica**: puntuable solo si se definen reglas de negocio; devuelve *score=None* en caso contrario.
- **Diagnóstico** (no puntuable): detección de valores atípicos, disponible exclusivamente desde el módulo de perfilado de datos.

Todas las métricas evaluables heredan de la clase abstracta *BaseMetric* e implementan el método *evaluate()*, que recibe un *DataFrame* de Pandas y devuelve un objeto *MetricResult* con tres campos: *score* (puntuación $[0,1]$ o *None*), *results* (diccionario con datos detallados) e *issues* (lista de incidencias detectadas).

Cálculo del *Quality Score* global. El *Quality Score* **no** es la media de las puntuaciones individuales de cada métrica. En su lugar, parte de 100 % y se reduce mediante una penalización acumulada basada en el *número* y *severidad* de las incidencias detectadas:

$$QS = \max\left(0, 1 - \min\left(0,97, \frac{\text{penalización bruta}}{\text{factor de escala}}\right)\right)$$

donde la penalización bruta es $\sum_s n_s \cdot w_s$ con pesos $w_{\text{critical}} = 0,12$, $w_{\text{high}} = 0,05$, $w_{\text{medium}} = 0,01$ y $w_{\text{low}} = 0,003$; y el factor de escala $\sqrt{\max(10, C)}/10$ normaliza por el número de columnas C del dataset para que la misma densidad de incidencias produzca la misma calificación independientemente del ancho del conjunto de datos.

B.1 Completitud (completeness)

Objetivo. Mide el grado en que los valores esperados están presentes en el conjunto de datos, alineada con la dimensión de completitud de ISO/IEC 25012 (ISO/IEC, 2008).

Algoritmo. Para cada columna se calcula la ratio de valores no nulos: $\text{completitud}_c = 1 - \overline{\text{nulos}_c}$. La puntuación global es la media aritmética de las ratios de las columnas seleccionadas (o de todas, si no se especifican).

Antes del cálculo, se aplican los **patrones de nulidad** configurados (`null_patterns`): cadenas vacías, literales como "null", "N/A", "nan", "none", guiones y espacios en blanco se convierten a NaN para que no inflen artificialmente la completitud. El usuario puede definir patrones *preset* (7 predefinidos) o expresiones regulares personalizadas.

Generación de incidencias. Se emiten dos tipos de *issues*:

1. **Global:** si la completitud media del dataset cae por debajo del umbral configurado.
2. **Por columna:** cada columna individual cuya completitud esté por debajo del umbral genera su propia incidencia.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
columns	lista	todas	Columnas a evaluar.
threshold	float	0,95	Umbral mínimo de completitud.
null_patterns	dict	—	{presets: [...], custom: [...]}: patrones de nulidad adicionales.
weight	float	1,0	Peso de la métrica.

B.2 Unicidad (uniqueness)

Objetivo. Evalúa la ausencia de registros duplicados a nivel de fila y la variabilidad por columna individual.

Algoritmo. La evaluación opera en dos niveles simultáneos:

1. **Unicidad de filas:** $U_{\text{filas}} = \text{filas_únicas} / \text{total_filas}$.
2. **Variabilidad por columna:** para cada columna se calcula $V_c = \text{valores_únicos}_c / \text{valores_no_nulos}_c$ y se compara con un **umbral adaptativo** que depende del tipo inferido de la columna:
 - Columnas de identificador (*_id, *_uuid, *_key): umbral = 0,95.
 - Columnas categóricas (≤ 20 valores únicos, no numéricas): umbral = 0,05.
 - Resto de columnas: umbral = 0,30.

La **puntuación combinada** pondera ambos niveles cuando existen columnas de identificador: $S = 0,7 \cdot U_{\text{filas}} + 0,3 \cdot \overline{V_{\text{ID}}}$; si no las hay, $S = U_{\text{filas}}$.

Generación de incidencias. Se emiten tres tipos de *issues*:

1. **Baja variabilidad:** columnas cuya variabilidad está por debajo de su umbral adaptativo. La severidad para columnas no-ID se calcula con distancia relativa al umbral (no absoluta) para evitar falsos críticos en columnas categóricas legítimas.
2. **Filas duplicadas:** si $U_{\text{filas}} < \text{threshold}$, se adjuntan hasta 5 filas de ejemplo (con enmascaramiento de columnas sensibles).
3. **Identificador no único:** columnas que el usuario marca como únicas pero contienen valores repetidos.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
threshold	float	1,0	Umbral mínimo de unicidad de filas.
columns	lista	—	Columnas que deben ser estrictamente únicas.
weight	float	1,0	Peso de la métrica.

B.3 Exactitud sintáctica (*syntactic_accuracy*)

Objetivo. Verifica que los valores de cada columna se ajustan a patrones de formato predefinidos (correo electrónico, teléfono, DNI, etc.), alineada con la dimensión de exactitud de ISO/IEC 25012 (ISO/IEC, 2008).

Catálogo de patrones integrados. El sistema incluye 13 expresiones regulares predefinidas:

Tipo	Ejemplo / Descripción
email	user@example.com
url	https://...
phone_es	Teléfono español: +34 6XXXXXXX
phone_intl	Teléfono internacional genérico
dni_es	DNI español: 8 dígitos + letra
date_iso	Fecha ISO: AAAA-MM-DD
date_eu	Fecha europea: DD/MM/AAAA
integer	Número entero (con signo)
decimal	Número decimal (coma o punto)
uuid	Identificador UUID v4
ip_v4	Dirección IPv4
postal_code_es	Código postal español (5 dígitos)
credit_card	Tarjeta de crédito (4 grupos de 4 dígitos)

Además, los usuarios pueden definir **patrones personalizados** almacenados en la base de datos (*ValidationPattern*), que se cargan en tiempo de ejecución y sobrescriben los predefinidos cuando comparten la misma clave.

Auto-detección de tipos. Para columnas de tipo object sin configuración explícita, se muestrea hasta 100 valores y se prueban todos los patrones del catálogo. Se asigna el tipo con mayor tasa de coincidencia **si y solo si** supera el 85 %. Si dos o más tipos superan el umbral simultáneamente, la columna se marca como **formato mixto** (`mixed_format`) y se emite un *issue* de severidad media en lugar de asignar arbitrariamente un tipo.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
<code>columns</code>	lista	—	Lista de {column, expected_type, pattern}.
<code>auto_detect_types</code>	bool	true	Activar auto-detección para columnas sin tipo.
<code>custom_patterns</code>	dict	{ }	Mapa {columna: regex} adicional.
<code>threshold</code>	float	0,95	Umbral mínimo de conformidad.
<code>weight</code>	float	1,0	Peso de la métrica.

B.4 Consistencia lógica (`logical_consistency`)

Objetivo. Valida reglas de negocio entre columnas del dataset, expresadas como expresiones de consulta o reglas condicionales IF-THEN.

Tipos de reglas soportados:

1. **Violación directa** (`type: "violation"`): la expresión selecciona las filas que violan la regla. Tasa de cumplimiento: $1 - \text{violaciones} / \text{total_filas}$.
2. **Condicional** (`type: if_then`): se evalúa `condition` para seleccionar el ámbito y después se verifica `assertion` sobre ese subconjunto. La tasa de cumplimiento se normaliza sobre las filas que cumplen la condición, no sobre el total del dataset, para que una regla selectiva con 100 % de violaciones en su ámbito se refleje como un problema grave.

Las reglas también se pueden expresar en lenguaje natural con la sintaxis IF <condición> THEN <aserción> (o sus equivalentes en español Si... Entonces), que el motor parsea automáticamente mediante expresión regular.

Seguridad. Antes de ejecutar cualquier regla, el sistema verifica que no contenga *tokens* prohibidos (`import`, `exec`, `eval`, `os.`, `subprocess`, `lambda`, etc.) para prevenir inyección de código. Las reglas bloqueadas generan un *issue* de severidad alta.

Si no se configuran reglas, la métrica devuelve `score=None` y no contribuye al *Quality Score*.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
rules	lista	—	Lista de reglas: {name, type, expression, condition, assertion}.
weight	float	1,0	Peso de la métrica.

B.5 Equilibrio de clases (class_balance)

Objetivo. Cuantifica el equilibrio de la distribución de variables categóricas, especialmente relevante para la variable objetivo en tareas de ML supervisado.

Algoritmo. Se utiliza la entropía de Shannon (1948):

$$H = - \sum_i p_i \cdot \log_2(p_i), \quad H_{\text{máx}} = \log_2(n_{\text{clases}}), \quad \text{Índice de equilibrio} = \frac{H}{H_{\text{máx}}} \times 100$$

La métrica distingue dos modos de operación:

- **Modo explícito** (columnas configuradas por el usuario): la puntuación contribuye al *Quality Score*.
- **Modo auto-detectado**: se analizan automáticamente las columnas categóricas con hasta 50 valores únicos (y ratio de unicidad ≤ 25 % para excluir identificadores), pero sus resultados son solo informativos y **no** penalizan la calificación global, ya que el desequilibrio natural (p. ej., 97/3 en detección de fraude) no es un defecto de calidad per se según ISO/IEC 5259 (ISO/IEC, 2024).

Conformidad con distribución esperada. Opcionalmente, el usuario puede definir rangos porcentuales esperados para cada clase (p. ej., junior: [70, 85], senior: [10, 20]). Se calcula una puntuación de conformidad basada en cuántas clases caen dentro de su rango, y la puntuación final combina entropía y conformidad al 50 %.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
columns	lista	—	Columnas objetivo (modo explícito).
auto_detect	bool	true	Activar auto-detección.
max_cardinality	int	50	Máx. valores únicos para auto-detección.
imbalance_threshold_high	float	0,90	Umbral de clase dominante.
imbalance_threshold_low	float	0,05	Umbral de clase minoritaria.
expected_distribution	dict	{ }	Rangos esperados por clase: {clase: [min%, max%]}.
weight	float	1,0	Peso de la métrica.

B.6 Actualidad (currentness)

Objetivo. Mide la frescura de los datos calculando la antigüedad del registro de fecha más reciente de cada columna temporal. Implementa la dimensión *Currentness* (ΔT_2) de ISO/IEC 5259-2:2024 §6.2.5, medida Cur-ML-1 (*Feature currentness*). No debe confundirse con *Timeliness* (ΔT_1), que mide la latencia de ingesta y requiere *timestamps* externos.

Algoritmo de frescura. La puntuación de frescura por columna sigue una degradación lineal con factor de escala configurable (DECAY_SCALE=3):

$$\text{frescura} = \begin{cases} 1,0 & \text{si } d \leq U \\ \text{máx}\left(0, 1 - \frac{d-U}{U \cdot k}\right) & \text{si } d > U \end{cases}$$

donde d son los días de antigüedad, U el umbral de obsolescencia y $k = 3$ el factor de escala. Con el umbral por defecto de 30 días, la puntuación alcanza 0 a los 120 días.

Auto-detección. Columnas con `dtype=datetime64` se incluyen directamente. Para columnas de texto, se parsea una muestra de 50 valores y se incluyen si al menos el 50 % se reconoce como fecha. Si la tasa de parseo de una columna incluida es inferior al 80 %, se genera un *issue* informativo adicional.

Severidad de obsolescencia. La severidad se determina por la ratio d/U : *medium* ($\geq 1\times$), *high* ($\geq 3\times$) y *critical* ($\geq 10\times$).

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
<code>columns</code>	lista	—	Columnas temporales a evaluar.
<code>auto_detect</code>	bool	true	Activar auto-detección.
<code>staleness_threshold_days</code>	int	30	Umbral de obsolescencia en días.
<code>weight</code>	float	1,0	Peso de la métrica.

B.7 Diversidad (diversity)

Objetivo. Mide si el dataset cubre adecuadamente los valores o rangos que el usuario espera encontrar en cada columna, alineada con la dimensión de diversidad de ISO/IEC 5259-2 (ISO/IEC, 2024).

Esta métrica es **opt-in por columna**: solo se evalúan las columnas para las que el usuario define expectativas. Si no hay expectativas definidas, devuelve `score=None` y queda excluida del *Quality Score*.

Evaluación categórica. Para columnas de tipo `categorical`, se calcula la ratio de valores esperados presentes en el dataset. Cada valor presente con representación adecuada ($\geq \text{min_}$

representation) recibe crédito completo; los presentes pero subrepresentados reciben medio crédito; los ausentes, cero.

Evaluación numérica. Para columnas de tipo `numeric`, se divide el rango esperado `[mín, máx]` en n bins uniformes y se calcula la cobertura como la ratio de bins con al menos un dato: $S = \text{bins_con_datos} / \text{total_bins}$.

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
<code>columns</code>	<code>dict</code>	<code>{ }</code>	Mapa de columnas con sus expectativas.
<code>threshold</code>	<code>float</code>	<code>0,60</code>	Umbral mínimo de diversidad.

Para cada columna el usuario especifica el tipo (`categorical` o `numeric`) junto con `expected_values` (lista de valores esperados) o `expected_range` (`[mín, máx]`) y opcionalmente `min_representation` (defecto: 1 %) o `num_bins` (defecto: 10).

B.8 Valores atípicos (outliers)

Nota. Esta clase **no es una métrica evaluable**: no produce puntuación ni contribuye al *Quality Score*. No está registrada en el `METRIC_REGISTRY` y se invoca exclusivamente desde el módulo de perfilado de datos (*Data Profiling*), en línea con la recomendación de ISO/IEC 5259 (ISO/IEC, 2024).

Métodos de detección. El sistema soporta dos métodos:

- IQR** (por defecto): un valor x es atípico si $x < Q_1 - f \cdot \text{IQR}$ o $x > Q_3 + f \cdot \text{IQR}$, donde f es el factor configurable.
- Z-score**: un valor es atípico si $|z| > f$, donde $z = (x - \mu) / \sigma$.

Cada incidencia reporta la proporción de valores atípicos, hasta 5 valores de ejemplo (enmascarados si la columna es sensible) y las estadísticas de la distribución (Q_1 , Q_3 , IQR o μ , σ).

Parámetros de configuración:

Parámetro	Tipo	Defecto	Descripción
<code>columns</code>	<code>lista</code>	todas las numéricas	Columnas a evaluar.
<code>method</code>	<code>string</code>	<code>iqr</code>	Método: <code>iqr</code> o <code>zscore</code> .
<code>factor</code>	<code>float</code>	<code>1,5</code>	Factor multiplicador (f).

B.9 Modelo de severidad dinámica

Todas las métricas comparten un modelo de **severidad dinámica** implementado en `BaseMetric.calculate_dynamic_severity()`. La severidad de cada incidencia se calcula automáti-

camente en función de la distancia entre el valor real y el umbral configurado, y varía según el tipo de métrica. El Cuadro B.1 resume los criterios.

Cuadro B.1: Criterios de severidad dinámica por tipo de métrica.

Tipo de métrica	Critical	High	Medium	Low
Compleitud / Unicidad	$< 50 \%$	$< 70 \%$ o $\text{dist.} > 15 \text{ pp}$	$\text{dist.} > 5 \text{ pp}$	$\text{dist.} \leq 5 \text{ pp}$
Outliers (proporción)	$\geq 20 \%$	$\geq 10 \%$	$\geq 5 \%$	$< 5 \%$
Clase dominante	$\geq 99 \%$	$\geq 95 \%$	$\geq 90 \%$	$< 90 \%$
Actualidad (ratio d/U)	$\geq 10\times$	$\geq 3\times$	$\geq 1\times$	$< 1\times$
Diversidad	$\leq 20 \%$	$\leq 40 \%$	$< \text{umbral}$	$\geq \text{umbral}$

Anexo C

Backlog completo del proyecto

Este anexo recoge el backlog completo del proyecto con todas las historias de usuario e ítems técnicos, organizados por sprint. Las estimaciones en *story points* se formalizaron retrospectivamente a partir de los tiempos aproximados dedicados a cada tarea. Los ítems marcados con asterisco (*) no estaban en el backlog inicial y se añadieron durante el desarrollo.

Sprint 1 (octubre–noviembre 2025)

Cuadro C.1: Backlog Sprint 1

ID	Ítem	SP	Estado
HU-01	Registro con email y contraseña	3	Cerrado
HU-02	Login y JWT (access + refresh token)	2	Cerrado
T-01	Setup Docker Compose (8 servicios + healthchecks)	3	Cerrado
T-02	Diseño ERD y modelo de datos inicial	3	Cerrado
T-03	Migraciones iniciales con Flask-Migrate	2	Cerrado

Sprint 2 (diciembre 2025–enero 2026)

Cuadro C.2: Backlog Sprint 2

ID	Ítem	SP	Estado
HU-03	CRUD de proyectos	3	Cerrado
HU-05	Carga de archivos CSV con parseo robusto	5	Cerrado
HU-06	Versionado de datasets (parent-child)	3	Cerrado
T-04	Arquitectura backend en capas (blueprints + servicios)	4	Cerrado
T-05	Pruebas manuales de la API REST	3	Cerrado
T-06*	Parseo robusto CSV con matriz de fallback (4×5×2)	2	Cerrado

Cuadro C.3: Backlog Sprint 3

ID	Ítem	SP	Estado
HU-07	Configuración de métricas con parámetros y pesos	5	Cerrado
HU-08	Plantillas de métricas del sistema	3	Cerrado
HU-09	Lanzamiento de evaluación (Celery) + progreso	5	Cerrado
HU-10	Listado de issues con severidad y columna afectada	3	Cerrado

Cuadro C.4: Backlog Sprint 4

ID	Ítem	SP	Estado
HU-04	Cuadro de mando principal con resumen de calidad	5	Cerrado
HU-09	Progreso de evaluación en tiempo real (polling)	3	Cerrado
HU-13	EDA: histogramas, boxplots, correlaciones	5	Cerrado
HU-14	Marcado de columnas PII en la UI	1	Cerrado

Cuadro C.5: Backlog Sprint 5

ID	Ítem	SP	Estado
HU-11	Quality Gates con tres estados (PASSED/WARNING/FAILED)	5	Cerrado
HU-12	Fingerprinting SHA-256 para trazabilidad de issues	5	Cerrado
T-07	Seguridad: rate limiting, cabeceras, PBKDF2	0	Cerrado
T-08*	HybridEvaluationService (fallback síncrono)	2	Cerrado
T-09*	Evaluation Watchdog (detección de evaluaciones atascadas)	1	Cerrado

Sprint 3 (febrero 2026)

Sprint 4 (marzo 2026)

Sprint 5 (abril 2026)

Sprint 6 (mayo–junio 2026)

Cuadro C.6: Backlog Sprint 6

ID	Ítem	Estado
HU-15	Panel de administración (UI)	Descartada
T-10	Suite de pruebas unitarias del motor de métricas y <i>Quality Score</i> (pytest)	Cerrado
T-11	Suite de pruebas de integración: versionado, Quality Gates y autorización IDOR	Cerrado
T-12	Suite de pruebas de <i>fingerprinting</i> y <i>diff</i> contra línea base	Cerrado
T-13	Corrección de defectos detectados en pruebas (11 issues)	Cerrado
T-14	Redacción memoria TFG + Anexo A y B	Cerrado
T-15*	Internacionalización del <i>frontend</i> (ES/EN, i18next, 54 componentes)	Cerrado

Anexo D

Análisis retrospectivo de riesgos

Este anexo amplía el análisis retrospectivo de riesgos resumido en la sección 4.9 con el detalle de ocurrencia de cada riesgo y las acciones correctivas efectivamente aplicadas durante el proyecto, reconstruidas a partir de las incidencias documentadas en las retrospectivas de cada sprint. Conviene recordar que no existió un registro formal de riesgos mantenido durante el desarrollo: las prácticas preventivas que se enumeran fueron las que se aplicaron de manera implícita en el flujo de trabajo diario, y las acciones correctivas se decidieron *ad hoc* cuando aparecía la incidencia.

Resumen de ocurrencia: de los ocho riesgos reconstruidos, tres se materializaron de forma significativa (R-02, R-03, R-08), uno ocurrió de forma menor (R-05) y uno tuvo un efecto positivo (R-01). Los tres riesgos restantes (R-04, R-06, R-07) no llegaron a producirse, lo que resulta coherente con las prácticas preventivas que se aplicaron implícitamente en el flujo de trabajo.

Cuadro D.1: Reconstrucción retrospectiva de riesgos con ocurrencia y acciones aplicadas

ID	Riesgo	P	I	Preventiva	Correctiva	Ocurrió
R-01	Complejidad del motor de métricas	M	A	Sprint completo reservado para S3	Alcance de exactitud sintáctica ampliado a 13 tipos (mejora)	Parcialmente (positivo)
R-02	Problemas integración Docker	M	A	Stack completo desde S1	Ajuste de health-checks y tiempos de reconexión en S5	Sí (S5, Celery+Redis)
R-03	Desvío temporal	A	M	Margen del 20 % en estimaciones	Repriorización del backlog; HU-13 reducida en alcance	Sí (S2 y S5)
R-04	Pérdida de datos/código	B	A	Push diario a GitHub	N/A	No
R-05	Cambios de requisitos por tutores	M	M	Revisiones periódicas (bajo demanda)	Añadidos ítems al backlog (T-08, T-09) en S5	Sí (menor)
R-06	Rendimiento con ficheros grandes	M	M	Límite de 100 MB configurado en el servidor	N/A (dentro del límite en todos los tests)	No
R-07	Fallos auth JWT en producción	B	A	Pruebas de integración del flujo auth	N/A	No
R-08	Incompatibilidad de versiones	M	M	Versiones fijas en requirements.txt	Actualización de SQLAlchemy 1.x → 2.0 en S2	Sí (menor, S2)

P = Probabilidad (A=Alta, M=Media, B=Baja) I = Impacto (A=Alto, M=Medio, B=Bajo)

Anexo E

Estimación de costes y viabilidad económica

Este anexo amplía el análisis de costes de la sección 4.8 con un desglose detallado por sprint y un análisis de viabilidad económica.

Desglose de horas reales por sprint

Cuadro E.1: Horas reales de desarrollo por sprint y categoría

Sprint	Backend	Frontend	Infraestr.	Pruebas	Doc./Otro	Total
S1 — Análisis y arquitectura	20	0	30	0	10	60
S2 — Backend core	65	0	10	10	0	85
S3 — Motor de métricas	55	0	5	10	0	70
S4 — Frontend	10	50	0	5	0	65
S5 — Features avanzadas	40	10	5	0	0	55
S6 — Pruebas y doc. técnica	10	0	0	30	0	40
S6 — Documentación (memoria)	0	0	0	0	50	50
Total	200	60	50	55	60	425

Coste total del proyecto

Análisis de viabilidad

Para evaluar la viabilidad de convertir DATAQUAL en un producto SaaS, se estiman los costes de infraestructura en un entorno AWS:

Con un coste de infraestructura de aproximadamente 90 €/mes, un modelo de suscripción de 15 €/usuario/mes requeriría solo 6 usuarios de pago para cubrir los costes operativos, lo que indica una **viabilidad económica favorable** para el modelo SaaS.

Cuadro E.2: Desglose completo de costes del proyecto

Categoría	Concepto	Uds.	€/ud.	Total
Mano de obra	Desarrollador full-stack junior (425h real)	425 h	25 €/h	10 625 €
Mano de obra	Supervisión tutores (2 tutores × 10h)	20 h	60 €/h	1 200 €
Software	Python, Flask, Next.js, PostgreSQL, Redis, MinIO	Licencias open source	0 €	0 €
Software	GitHub (plan Free)	1	0 €	0 €
Software	Docker Desktop (personal, gratuito)	1	0 €	0 €
Hardware	Equipo personal (amortización 3 años, 9 meses uso)	9 meses	28 €/mes	252 €
Hardware	Electricidad (estimada 50W × 8h/día × 270 días)	108 kWh	0,20 €/kWh	22 €
TOTAL				12 099 €

Cuadro E.3: Estimación de costes de producción en AWS (por mes)

Servicio AWS	Equivalente	€/mes
Amazon RDS (db.t3.small)	PostgreSQL	30 €
Amazon S3 (10 GB)	MinIO	0,25 €
Amazon ElastiCache (cache.t3.micro)	Redis	15 €
Amazon ECS (2 tareas)	Backend + Celery	40 €
Amazon CloudFront + S3	Frontend estático	5 €
Total infraestructura cloud/mes		~90 €

Anexo F

Plan de pruebas integral

Este anexo formaliza el plan de pruebas de DATAQUAL y complementa la visión sintética presentada en el capítulo de resultados. Se redacta siguiendo las recomendaciones del estándar ISO/IEC/IEEE 29119 (*Software Testing*) y la pirámide clásica de pruebas de Cohn, adaptada a una arquitectura de tres capas (*frontend*, API REST, motor analítico asíncrono). Su finalidad es doble: documentar la estrategia de calidad aplicada durante el desarrollo y servir de referencia operativa para futuras iteraciones del producto.

F.1 Objetivos y alcance

Objetivos del plan

- Verificar que DATAQUAL cumple los requisitos funcionales y no funcionales recogidos en el capítulo de objetivos.
- Detectar regresiones en los flujos críticos (autenticación, ejecución de análisis, *Quality Gate*) antes de cada incremento.
- Validar la lógica del dominio *Sonar-Lite* (*AnalysisRun*, *baseline* y *fingerprinting* de incidencias).
- Garantizar la integridad de los datos almacenados en PostgreSQL y MINIO!, incluyendo el versionado de *datasets*.
- Documentar de forma trazable el estado de calidad del producto en el momento de la defensa.

Alcance

Dentro del alcance: la API REST (módulos *auth*, *projects*, *datasets*, *metrics*, *evaluations*, *dashboard*, *admin*), el motor de métricas, las tareas Celery, el modelo de datos y los flujos de interfaz crítica.

Fuera del alcance: pruebas de penetración profundas, auditoría formal de accesibilidad WCAG! AA!, pruebas de carga sostenida en producción y pruebas de compatibilidad con navegadores heredados.

F.2 Estrategia y niveles de prueba

La estrategia se articula sobre cuatro niveles complementarios, distribuidos conforme a una pirámide de pruebas con peso mayoritario en los niveles inferiores (mayor velocidad, menor coste por ejecución):

Cuadro F.1: Niveles de prueba y distribución prevista

Nivel	Tipo	Peso	Propósito
L1	Unitario	50 %	Lógica pura: cálculo de métricas, helpers, validadores, serializadores.
L2	Integración (API)	35 %	<i>Endpoints</i> con base de datos en memoria, fixtures y JWT.
L3	Sistema / E2E	10 %	Flujos completos a través del <i>frontend</i> contra el <i>backend</i> real (Playwright o equivalente).
L4	No funcional	5 %	Carga, seguridad, rendimiento de consultas y observabilidad.

F.3 Tipos de pruebas aplicadas

Funcionales. Validan el comportamiento esperado de cada *endpoint*, métrica y vista de la UI.

Integración. Comprueban la colaboración correcta entre Flask, SQLAlchemy, Celery, **REDIS!** y **MINIO!**.

Regresión. Suite ejecutada antes de cada *merge* a *main* para garantizar que los cambios no rompen funcionalidad existente.

Smoke. Conjunto reducido (<30 s) que verifica que el sistema arranca y los flujos básicos responden.

Negativas / límite. Validan respuestas ante entradas inválidas, datasets vacíos, valores extremos y errores de autenticación.

Seguridad. Revisión de JWT, control de acceso por propietario, sanitización de *uploads* y revisión de dependencias con *pip-audit*.

Rendimiento. Mediciones puntuales de tiempo de respuesta de *endpoints* clave y de ejecución de análisis sobre *datasets* de hasta 100 MB!.

Usabilidad (exploratorias). Ejecutadas manualmente sobre los flujos principales para detectar problemas de **UX!** no capturados por las pruebas automáticas.

F.4 Suites de prueba (resumen por capa)

A continuación se enumeran las *suites* previstas y su propósito de alto nivel. Los casos detallados (*TC-ID*, precondiciones, pasos y datos de entrada) se mantienen en el documento técnico `docs/PLAN_PRUEBAS.md`, fuera de la memoria, para facilitar su ejecución y evolución sin afectar al documento académico.

F.4.1 Capa de *backend* (Flask + Celery)

F.4.2 Capa de dominio (*Sonar-Lite*)

F.4.3 Capa de *frontend* (Next.js)

F.4.4 Pruebas no funcionales

F.5 Entornos y datos de prueba

Entornos

- **Local (desarrollador).** *pytest* con SQLite en memoria; ejecuta L1 y L2 en menos de 30 s.
- **Docker compose.** *Stack* completo (PostgreSQL, **REDIS!**, **MINIO!**, Celery) para pruebas de integración realista y manuales.
- **Pre-producción.** No disponible en el alcance del TFG; queda planteado como trabajo futuro.

Datos de prueba

Se utilizan tres categorías de *datasets* sintéticos almacenados en *samples/*: (i) conjuntos limpios para escenarios positivos; (ii) conjuntos contaminados deliberadamente (nulos, duplicados, formatos incorrectos, valores fuera de rango) para validar la detección de incidencias; y (iii) conjuntos grandes (50–100 MB!) para pruebas de rendimiento. Los *fixtures* de *pytest* construyen usuarios, proyectos y *datasets* de referencia en cada ejecución.

F.6 Criterios de entrada, salida y aceptación

Criterios de entrada (*ready to test*)

- Código integrado en la rama de trabajo y compilando sin errores.
- Migraciones de base de datos aplicadas correctamente.
- Documentación de la **HU!** disponible (criterios de aceptación redactados).

Criterios de salida (*exit criteria*)

- 100 % de los casos críticos (P1) ejecutados y *aprobados*.
- Ningún defecto de severidad *bloqueante* o *crítica* abierto.
- Cobertura de líneas ≥ 70 % en módulos críticos (*api/auth*, *api/datasets*, *tasks/*).
- Suite *smoke* verde en el último *commit* de *main*.

Criterios de aceptación por HU!

Cada **HU!** del *backlog* (anexo ??) incluye sus propios criterios de aceptación; el plan de pruebas mantiene una matriz de trazabilidad **HU!** $\leftrightarrow$ *suite* que se detalla en el documento técnico complementario.

Cuadro F.2: Suites de *backend*

ID	Suite	Propósito	Estado
BE-01	test_auth	Registro, <i>login</i> , refresco y revocación de JWT.	Implementada
BE-02	test_projects_api	CRUD de proyectos y configuración de métricas.	Implementada
BE-03	test_dataset_versioning	Lógica de versionado padre-hijo y árbol de linaje.	Implementada
BE-04	test_dataset_versioning_api	<i>Endpoints</i> de versionado y consulta de historial.	Implementada
BE-05	test_quality_gate	Umbrales del <i>Quality Gate</i> y veredicto del análisis.	Implementada
BE-06	test_metrics_engine + test_metrics_engine_extended	Comportamiento de las ocho clases de métricas frente a casos límite: completitud, unicidad, actualidad, exactitud sintáctica, equilibrio de clases, diversidad, valores atípicos y consistencia lógica (42 casos).	Implementada
BE-07	test_analysis_run	Creación de AnalysisRun, comparación con <i>baseline</i> y cálculo de <i>new/fixed issues</i> .	Pendiente
BE-08	test_celery_tasks	Ejecución asíncrona, reintentos y manejo de errores en tareas diferidas.	Pendiente
BE-09	test_minio_storage	Subida, descarga e	Pendiente

Cuadro F.3: Suites de dominio *Sonar-Lite*

ID	Suite	Propósito	Estado
DM-01	test_fingerprinting	Estabilidad del <i>fingerprint</i> de <i>DataQualityIssue</i> entre ejecuciones (mismo <i>issue</i> $\Rightarrow$ misma huella; 28 casos).	Implementada
DM-02	test_diff_baseline	Cálculo correcto de incidencias <i>nuevas</i> , <i>resueltas</i> y <i>persistentes</i> respecto al <i>base-line</i> (7 casos).	Implementada
DM-03	test_quality_score	Verificación numérica de la fórmula de <i>Quality Score</i> con casos canónicos (sin issues, una severidad, mezcla, <i>cap</i> de penalización, corrección de dimensionalidad; 30 casos).	Implementada
DM-04	test_quality_gate_verdict	Combinaciones de umbrales (<i>min_score</i> , <i>max_critical</i> , <i>max_new</i>) y veredicto PASSED/WARNING/FAILED con precedencia (12 casos).	Implementada

Cuadro F.4: Suites de *frontend*

ID	Suite	Propósito	Estado
FE-01	<i>Smoke</i> de páginas	Renderizado sin errores de las páginas públicas (<i>login</i> , registro, recuperación).	Manual
FE-02	Componentes de formulario	Validación de campos, mensajes de error y estado deshabilitado.	Manual
FE-03	Flujo de subida de <i>datasets</i>	<i>Drag & drop</i> , validación de tamaño y tipo, progreso.	Manual
FE-04	Configuración de métricas	Edición del <i>metrics_config</i> , persistencia tras recarga.	Manual
FE-05	Ejecución y consulta de análisis	Lanzamiento de <i>AnalysisRun</i> , <i>polling</i> de estado y visualización del dashboard.	Manual
FE-06	Canvas de linaje	Render correcto del árbol de versiones y navegación entre nodos.	Manual
FE-07	E2E críticos (Playwright)	Automatización futura de FE-03 + FE-05 contra <i>stack</i> Docker.	Pendiente

Cuadro F.5: Suites no funcionales

ID	Suite	Propósito	Estado
NF-01	Seguridad JWT	<i>Tokens</i> expirados, manipulados y reutilización tras <i>logout</i> .	Parcial (BE-01)
NF-02	Autorización por recurso	Un usuario no puede acceder a proyectos o <i>datasets</i> de otro (cubierta por BE-02 para proyectos y BE-10 para <i>datasets</i>).	Implementada
NF-03	Validación de entradas	<i>Fuzzing</i> ligero de cuerpos JSON! y <i>uploads</i> maliciosos (CSV con <i>payloads</i>).	Pendiente
NF-04	Auditoría de dependencias	Ejecución de <i>pip-audit</i> y <i>npm audit</i> en cada <i>release</i> .	Manual
NF-05	Rendimiento de análisis	Tiempos de ejecución sobre <i>datasets</i> de 10, 50 y 100 MB! .	Manual
NF-06	Rendimiento de <i>endpoints</i> clave	Latencia <i>p50/p95</i> de GET <i>/api/dashboard</i> y listados paginados con 1 000 elementos.	Pendiente

F.7 Métricas y KPI! del plan

Cuadro F.6: Indicadores de calidad del proceso de pruebas

KPI	Descripción	Objetivo	Estado
Cobertura líneas	<i>pytest -cov</i> sobre backend/.	$\geq 70\%$	Por medir
Tasa de paso	Casos <i>passed</i> / total ejecutados.	$\geq 95\%$	100 % (232/233)
Defectos críticos abiertos	Bugs P1 sin resolver al cierre.	0	0
Tiempo suite L1+L2	Duración total en local.	< 60 s	~17 s
Densidad de defectos	Defectos / KLOC en el periodo.	Decreciente	N/A

F.8 Riesgos del plan de pruebas

- RP-01.** Cobertura limitada del *frontend* automatizado. *Mitigación:* pruebas manuales guiadas y *checklist* de regresión antes de cada hito.
- RP-02.** Pruebas dependientes del entorno Docker (red, volúmenes). *Mitigación:* *fixtures* aislados y uso preferente de SQLite en memoria para L2.
- RP-03.** Datos sintéticos no representativos de producción. *Mitigación:* incorporación de *datasets* públicos (**UCI!**, Kaggle) en muestras de validación.
- RP-04.** Falta de pruebas de carga sostenida. *Mitigación:* documentado como trabajo futuro; mediciones puntuales con ficheros de 100 **MB!**.

F.9 Roles y responsabilidades

Dado el carácter individual del TFG, el autor asume los roles de *QA Lead*, *Test Engineer* y *ejecutor manual*. Los tutores actúan como *stakeholders* y validan los criterios de aceptación en las revisiones de cada *sprint* (capítulo de metodología).

F.10 Trazabilidad y herramientas

- **Pruebas *backend*.** `pytest + pytest-cov; fixtures` en `conf/test.py`.
- **Pruebas *frontend*.** Pruebas manuales guiadas; Playwright planificado para FE-07.
- **Análisis estático.** `ruff`, `eslint` y `tsc -noEmit`; SonarCloud configurado vía `sonar-project.properties`.
- **Gestión de defectos.** *Issues* de GitHub etiquetados `bug/P1..P4`.
- **Matriz de trazabilidad.** Mantenida en `docs/PLAN_PRUEBAS.md`, vinculando **HU!**, suite y veredicto.

F.11 Estado actual y trabajo futuro

A la fecha de redacción de la memoria se han implementado doce *suites* automatizadas que cubren las áreas de mayor riesgo: BE-01 a BE-05 (autenticación, proyectos, versionado y *Quality Gate* básico), BE-06 (motor completo de métricas: 8 clases), BE-10 (autorización cruzada IDOR) y DM-01 a DM-04 (*fingerprinting*, comparación con *baseline*, *Quality Score* y combinaciones de veredicto del *gate*). En conjunto, la suite acumula 233 casos de prueba (232 *passed* y 1 *skipped*) y se ejecuta completa en aproximadamente 17 segundos sobre SQLite en memoria.

Las suites pendientes (BE-07–BE-09 y FE-07) se han priorizado para una próxima iteración del producto, junto con la integración continua mediante GitHub Actions y la generación automatizada de informes de cobertura.

Este plan deberá revisarse al inicio de cada nueva fase de desarrollo para incorporar las funcionalidades añadidas y reajustar los criterios de aceptación correspondientes.

Anexo G

Manual de usuario

Este anexo ofrece una guía paso a paso de las operaciones principales de DATAQUAL desde la perspectiva del usuario final. Se asume que el sistema está desplegado y accesible en `http://localhost:3000`. La interfaz está disponible en español e inglés y el idioma se puede alternar en cualquier momento desde el selector ES / EN situado en la parte inferior de la barra lateral.

Requisitos previos

1. Docker Desktop instalado y en ejecución.
2. Repositorio clonado: `git clone <url-repositorio>`.
3. Fichero `backend/.env` configurado con las variables de entorno (plantilla en `backend/.env.example`).

Arranque del sistema

```
cd TFG_CALIDAD_DATOS_IA
docker-compose up --build    # Primera vez (compila imágenes)
docker-compose up           # Arranques posteriores
```

El sistema tarda aproximadamente 30–60 segundos en estar completamente operativo. Una vez listos los servicios, acceder a `http://localhost:3000`.

Navegación general

La interfaz se organiza en torno a tres elementos: una barra lateral persistente que agrupa la navegación principal, una zona central de contenido y un encabezado contextual con *breadcrumbs* que indica la posición actual (proyecto, *dataset* o evaluación activos).

La barra lateral se divide en dos secciones. La sección principal contiene los apartados **Dashboard**, **Proyectos**, **Datasets** y **Análisis**; los tres últimos se despliegan en accesos directos a la vista de listado, a la acción de creación o carga, y a las entradas recientes (los proyectos del usuario aparecen como atajos directos bajo **Proyectos**). La sección de **Configuración** agrupa el perfil del usuario y la gestión de plantillas de métricas reutilizables. En la parte inferior se sitúan el selector de idioma ES / EN, el indicador del usuario activo y el

botón de cierre de sesión. La barra lateral se puede colapsar a un panel de iconos pulsando el *chevron* de su cabecera, lo que maximiza el área útil para el contenido.

Cuadro de mando

Tras iniciar sesión el sistema redirige al cuadro de mando (`/dashboard`), que ofrece una visión global del estado de calidad de todos los proyectos del usuario. La pantalla agrupa varias secciones de información:

- **Indicadores clave (KPI):** número de proyectos activos, *datasets* en uso y puntuación media de los últimos análisis completados.
- **Distribución de *Quality Gates*:** desglose visual del estado del último análisis de cada proyecto, con contadores de **Aprobado**, **Advertencia** y **Fallido**.
- **Requiere atención:** listado de los proyectos cuyo *Quality Gate* ha fallado, está en advertencia, o cuya puntuación de calidad se encuentra por debajo del 70 %.
- **Historial de salud:** gráfico de barras con el histórico de análisis recientes por proyecto, accesible mediante *scroll* horizontal para revisar la evolución a lo largo del tiempo.

Si el usuario aún no ha creado ningún proyecto, el cuadro de mando muestra un estado vacío con una llamada a la acción **Crear proyecto**.

Flujo principal de uso

1. Registro e inicio de sesión

La autenticación se realiza desde la pantalla unificada `/auth`, que ofrece dos pestañas: **Iniciar sesión** (activa por defecto) y **Registrarse**.

1. Para crear una cuenta nueva, seleccionar la pestaña **Registrarse** (o acceder directamente a `/auth?mode=register`) e introducir nombre de usuario, correo electrónico y contraseña. Tras el registro, el sistema inicia sesión automáticamente y redirige al cuadro de mando.
2. Para los inicios de sesión sucesivos, introducir el usuario o correo y la contraseña en la pestaña **Iniciar sesión**. El *token* JWT se almacena en `localStorage` y se inyecta automáticamente en todas las peticiones a la API mediante el cliente Axios; la renovación del *token* de acceso es transparente para el usuario gracias al *refresh token* de larga duración.

2. Crear un proyecto

Los proyectos son la unidad organizativa de DATAQUAL: agrupan los *datasets* cargados, las métricas configuradas, las evaluaciones ejecutadas y el *Quality Gate* del conjunto.

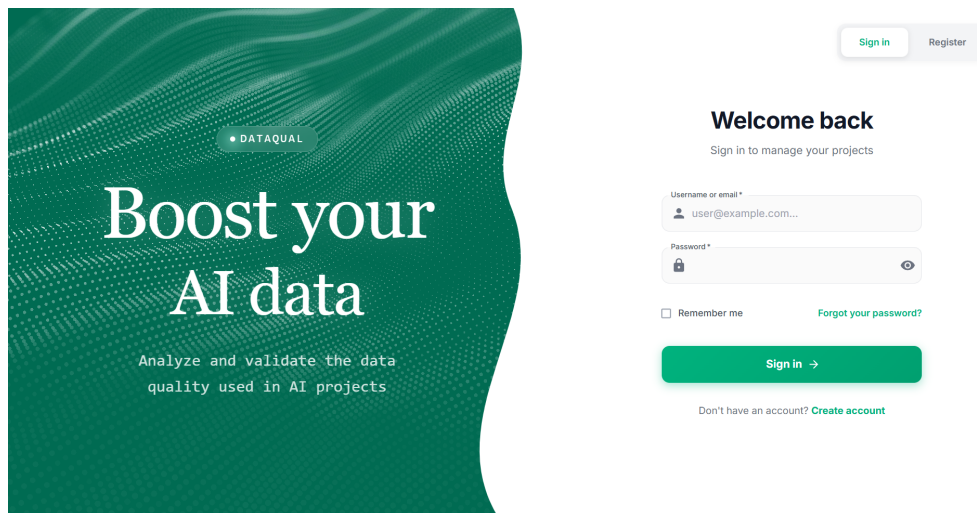


Figura G.1: Pantalla unificada de autenticación con pestañas de inicio de sesión y registro

1. Pulsar **Nuevo proyecto** desde el cuadro de mando, desde el listado de proyectos (/projects) o desde el acceso directo de la barra lateral. La aplicación abre un asistente de tres pasos:
 - a) **Información básica:** nombre del proyecto y descripción opcional.
 - b) **Plantilla y métricas:** elección entre aplicar una plantilla del sistema (que pre-configura un conjunto coherente de métricas y umbrales) o una configuración personalizada activando individualmente las métricas deseadas. Los parámetros detallados se pueden refinar posteriormente desde la página del proyecto.
 - c) **Resumen y confirmación:** revisión final del proyecto antes de su creación.
2. Al confirmar, el proyecto aparece en el listado con estado *Sin evaluaciones* y queda disponible como atajo en la barra lateral para acceso rápido en sesiones posteriores.

[Insertar captura: asistente Nuevo proyecto (/projects/new), mostrando los pasos Información básica, Plantilla y métricas y Resumen]

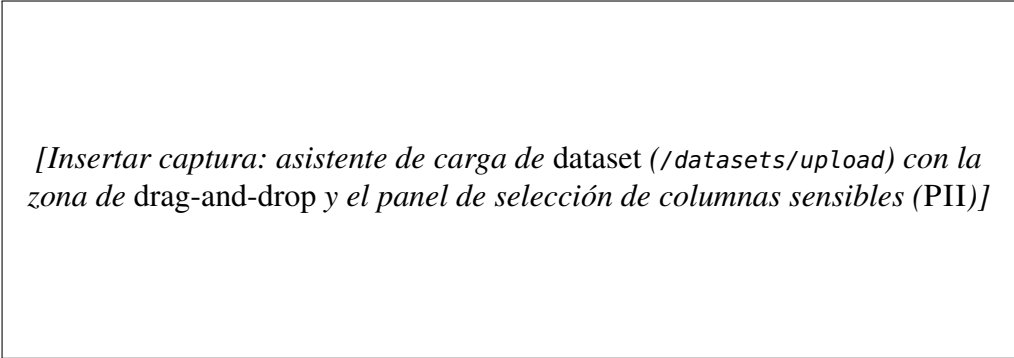
Figura G.2: Asistente de creación de un nuevo proyecto en tres pasos

3. Cargar un *dataset*

La carga de *datasets* sigue un asistente de tres pasos accesible desde /datasets/upload, desde el botón **Añadir dataset** en la pestaña *Datasets* de un proyecto, o desde el acceso directo **Subir dataset** en la barra lateral.

1. **Selecciona un proyecto:** elegir el proyecto al que se asociará el *dataset*. Si la carga se inicia desde la página de un proyecto concreto, este paso se completa de forma automática.
2. **Añade un archivo:** arrastrar y soltar un fichero CSV en la zona de carga o pulsar para abrir el selector de archivos. El sistema solo admite ficheros con extensión `.csv` y un tamaño máximo de 10 MB, y aplica una estrategia de *fallback* para codificaciones y delimitadores poco habituales descrita en la sección 5.3.3.
3. **Revisa y confirma:** introducir el nombre del *dataset*, una descripción opcional y, opcionalmente, una etiqueta de versión (por ejemplo `v2.0`, *producción* o *baseline*). En la misma pantalla, el panel *Privacidad de datos* permite marcar las columnas sensibles (PII) cuyos valores se enmascararán automáticamente en todas las salidas posteriores del sistema (logs, *issues*, vistas previas, perfilado).

Al confirmar la carga, el sistema valida el fichero, extrae el esquema y las estadísticas descriptivas básicas, almacena el binario en MinIO y registra los metadatos en PostgreSQL.



[Insertar captura: asistente de carga de dataset (/datasets/upload) con la zona de drag-and-drop y el panel de selección de columnas sensibles (PII)]

Figura G.3: Asistente de carga de *dataset* CSV con marcado opcional de columnas sensibles (PII)

4. Configurar métricas del proyecto

Cada proyecto mantiene su propia configuración de métricas, accesible desde la pestaña **Métricas** de la página del proyecto o desde el botón **Configurar métricas**, que abre la página dedicada `/metrics/configure/[id]`. El configurador presenta tres pestañas:

1. **Métricas disponibles:** catálogo de las siete métricas evaluables del sistema (completitud, unicidad, exactitud sintáctica, consistencia lógica, equilibrio de clases, actualidad y diversidad). Cada métrica incluye una breve descripción de lo que mide y un botón para activarla en el proyecto.
2. **Métricas seleccionadas:** para cada métrica activa, se pueden ajustar sus parámetros específicos (umbrales, pesos, columnas objetivo, patrones de validación para la exactitud sintáctica, reglas IF-THEN para la consistencia lógica, etc.).

3. **Plantillas:** aplicar una plantilla predefinida del sistema o una creada por el usuario (véase paso 10) para reutilizar configuraciones entre proyectos similares sin tener que repetir el ajuste manual.

Pulsar **Guardar configuración** aplica los cambios y devuelve al usuario a la página del proyecto.

[Insertar captura: configurador de métricas (/metrics/configure/[id]) con sus tres pestañas Métricas disponibles, Métricas seleccionadas y Plantillas]

Figura G.4: Configurador de métricas con catálogo, parametrización detallada y plantillas reutilizables

5. Ejecutar una evaluación

Una vez configuradas las métricas y cargado al menos un *dataset*, el usuario puede lanzar un análisis desde la página del proyecto pulsando el botón **Ejecutar análisis**.

1. Seleccionar el *dataset* (y opcionalmente la versión concreta) que se desea evaluar.
2. El sistema encola la tarea en Celery y muestra una barra de progreso en tiempo real (0–100 %) actualizada mediante *polling* cada pocos segundos. Si Celery no estuviera disponible, el servicio híbrido (HybridEvaluationService) ejecuta el análisis de forma síncrona como mecanismo de *fallback*, garantizando que la evaluación se complete.
3. Al finalizar, el sistema registra los resultados como un *AnalysisRun* inmutable y muestra el *Quality Score* obtenido junto con el veredicto del *Quality Gate*. Si la evaluación se quedara atascada por un tiempo prolongado, el *watchdog* de evaluaciones la detectaría y marcaría como fallida automáticamente.

6. Consultar resultados

Los resultados de los análisis son accesibles desde tres puntos de entrada: la pestaña **Análisis** de la página del proyecto (histórico de análisis del proyecto), el listado global en /evaluations (histórico cruzado de todos los proyectos), y la pestaña **Evaluaciones** de la página del *dataset*.

1. Pulsar sobre un análisis completado para abrir su detalle.
2. El indicador circular (*gauge*) muestra el *Quality Score* global (0–100), acompañado

[Insertar captura: lanzamiento de Ejecutar análisis desde la página del proyecto, con la barra de progreso asíncrono (0–100 %)]

Figura G.5: Lanzamiento de un análisis con seguimiento del progreso asíncrono mediante *polling*

del veredicto del *Quality Gate* (PASSED, WARNING o FAILED) y de un desglose por dimensiones de calidad.

3. Las pestañas dedicadas a cada métrica detallan el resultado por columna y permiten profundizar en los *issues* detectados, indicando severidad (crítica, alta, media, baja), columna afectada, descripción del problema y, cuando procede, el porcentaje de filas afectadas.
4. Los *badges* que acompañan a cada *issue* indican su estado respecto a la evaluación anterior: **Nuevo**, **Recurrente** o, en la pestaña de cambios, **Corregido**. Esta trazabilidad se sustenta en el sistema de *fingerprinting* descrito en la sección 5.6.3, que asigna un *hash* determinista a cada problema.

7. Explorar datos (EDA)

El módulo de perfilado interactivo es accesible desde la pestaña **Perfilado** de cualquier *dataset* (/datasets/[id]). Concentra en una única pantalla las visualizaciones habituales del análisis exploratorio de datos:

1. Histogramas y diagramas de caja (*boxplots*) por columna, con vistas separadas para columnas numéricas y categóricas.
2. Matriz de correlación, alternable entre los coeficientes de Pearson (relaciones lineales entre variables numéricas) y Spearman (relaciones monótonas, incluidas variables ordinales).
3. Detección de valores atípicos (*outliers*) con factor IQR ajustable interactivamente para explorar distintos niveles de tolerancia.
4. Análisis de duplicados a nivel de fila completa y de variabilidad por columna, útil para identificar columnas quasi-constantes o claves duplicadas.

Las columnas previamente marcadas como sensibles aparecen con sus valores ofuscados en todas las visualizaciones, garantizando que la exploración exploratoria no exponga información personal.

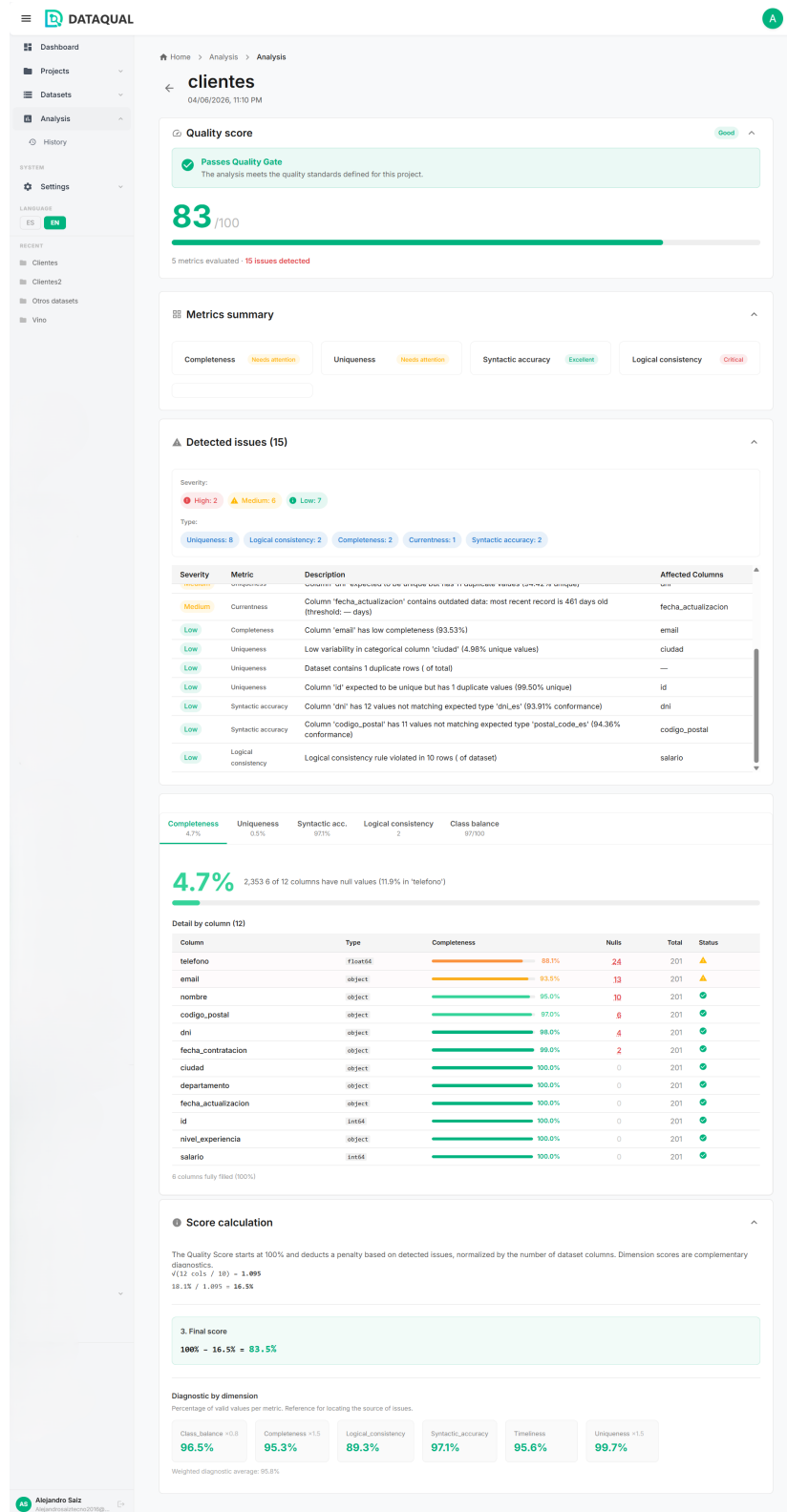


Figura G.6: Detalle de un análisis: *Quality Score*, métricas e *issues* con *badges* de trazabilidad entre ejecuciones

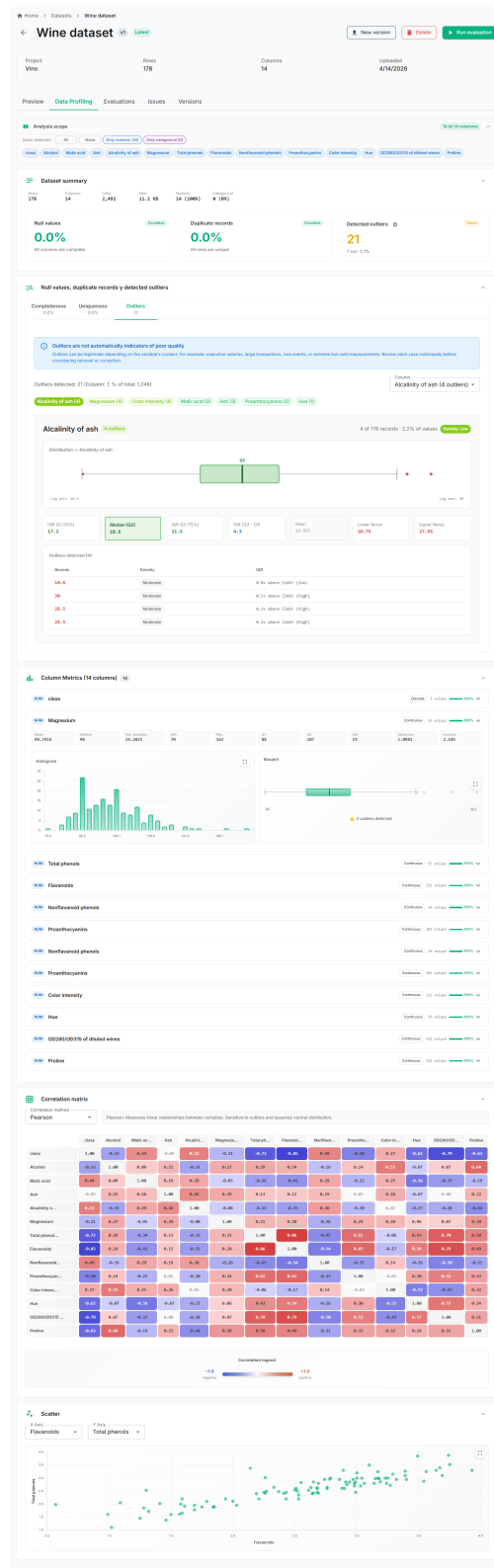


Figura G.7: Módulo de perfilado interactivo (EDA): histogramas, diagramas de caja, correlaciones y detección de *outliers*

8. Configurar un *Quality Gate*

Cada proyecto puede definir su propio *Quality Gate* desde la pestaña **Quality Gate** de la página del proyecto. El editor ofrece dos modos de configuración complementarios:

1. **Preajustes de rigor:** botones de selección rápida para los perfiles **Estricto**, **Moderado** y **Permisivo**, que aplican un conjunto coherente de umbrales sin necesidad de ajustarlos individualmente. Útiles como punto de partida.
2. **Configuración manual:** ajuste fino de los tres umbrales que determinan el veredicto del *gate*:
 - **Score mínimo** (defecto 70): puntuación de calidad global por debajo de la cual el *Quality Gate* falla.
 - **Máx. issues críticos** (defecto 0): número máximo de *issues* de severidad crítica que se permiten antes de fallar el *gate*.
 - **Máx. issues nuevos** (defecto 10): número máximo de *issues* nuevos respecto a la línea base permitidos.

El veredicto del *Quality Gate* adopta tres estados, mostrados tanto en la página del análisis como en el cuadro de mando: PASSED (verde), WARNING (amarillo, dentro del margen de advertencia configurado) y FAILED (rojo). Las opciones **Guardar cambios** y **Restaurar valores por defecto** controlan la persistencia de la configuración.

[Insertar captura: pestaña Quality Gate del proyecto con los preajustes Estricto / Moderado / Permisivo y los tres umbrales de configuración manual]

Figura G.8: Configuración del *Quality Gate* con preajustes de rigor y umbrales manuales (PASSED, WARNING, FAILED)

9. Gestionar versiones de un *dataset*

La pestaña **Versiones** de cualquier *dataset* muestra el árbol completo de versiones del fichero, con las relaciones padre-hijo entre cargas sucesivas y el indicador `is_latest` sobre la versión activa (la representación visual se ilustra en la Figura 5.4 del Capítulo 5). Desde esta vista se puede:

- Subir una nueva versión del *dataset*: la nueva carga se vincula automáticamente como hija de la versión seleccionada y se convierte en la nueva `is_latest`.

- Asignar una etiqueta descriptiva a una versión (v2.0, producción, baseline...) para facilitar su identificación posterior.
- Comparar dos versiones del mismo *dataset* desde `/datasets/compare` para ver diferencias en esquema, estadísticas descriptivas y resultados de evaluación.

10. Plantillas de métricas reutilizables

La sección `/settings/templates`, accesible desde el apartado **Configuración** de la barra lateral, permite gestionar plantillas del usuario que encapsulan una configuración completa de métricas (selección, parámetros y umbrales). Una vez guardada, una plantilla puede aplicarse durante el paso *Plantilla y métricas* del asistente de creación de un proyecto nuevo, o desde la pestaña *Plantillas* del configurador de métricas de un proyecto existente. Esta funcionalidad ahorra configuración manual repetitiva entre proyectos similares y favorece la consistencia de criterios de calidad entre equipos.

Monitorización del sistema

Más allá de la interfaz de usuario, dos paneles auxiliares permiten supervisar el estado del *stack* desde el navegador:

- **Flower** (monitor de Celery): `http://localhost:5555`. Visualiza en tiempo real los *workers* activos, las tareas en curso y completadas, sus tiempos de ejecución y el histórico de fallos, y permite reintentar manualmente tareas concretas.
- **MinIO Console**: `http://localhost:9001` (usuario y contraseña configurados en `.env`). Permite inspeccionar los *buckets* de almacenamiento y verificar que los *datasets* se han subido correctamente.

Apagado del sistema

```
docker-compose down          # Apagar (conserva datos en volúmenes)
docker-compose down -v       # Apagar y eliminar volúmenes (borra datos)
```

Anexo H

Diagramas UML del sistema

Este anexo recoge las descripciones estructuradas de los principales diagramas UML del sistema. El diagrama de arquitectura a alto nivel se incluye como figura en el cuerpo del documento (Figura 5.2); el resto se describe textualmente en este anexo y, cuando proceda, se complementa con representaciones generadas con PlantUML o draw.io.

Diagrama de paquetes

La arquitectura del sistema se organiza en seis paquetes principales, representados visualmente en la Figura 5.2 del Capítulo 5. La descripción detallada de cada paquete es la siguiente:

- **Presentación** (*frontend* Next.js + React): `pages/` (rutas), `components/` (más de 40 componentes React reutilizables), `services/` (cliente API con Axios) y `contexts/` (estado global: `AuthContext`, `NotificationContext`, `SidebarContext`, `LanguageContext`).
- **API REST** (*backend* Flask, 10 *blueprints*): siete *blueprints* funcionales (`auth`, `projects`, `datasets`, `metrics`, `evaluations`, `admin`, `patterns`) y tres de organización técnica REST (`project_metrics`, `project_datasets`, `dashboard`).
- **Servicios** (lógica de negocio): `EvaluationService` orquesta el flujo completo de evaluación; `HybridEvaluationService` permite ejecución asíncrona o síncrona según disponibilidad de Celery; `DatasetService` parsea archivos y extrae el esquema; `MinioService` abstrae el almacenamiento S3; `ExportService` genera informes a partir de los resultados.
- **Motor de métricas** (`services/metrics/`, núcleo extensible): `BaseMetric` (clase base abstracta), `MetricResult` (*dataclass*) y `METRIC_REGISTRY` (registro de *plugins*). Implementa siete métricas evaluables (`completeness`, `uniqueness`, `syntactic_accuracy`, `logical_consistency`, `class_balance`, `currentness`, `diversity`) y la clase `OutliersMetric`, usada exclusivamente desde el módulo de perfilado EDA.
- **Datos** (modelos SQLAlchemy ORM, 11 tablas): `User`, `Project`, `Dataset`, `Metric`, `MetricTemplate`, `Evaluation`, `Issue` (modelo original) y `AnalysisRun`, `DataQualityIssue`, `QualityGate`, `ValidationPattern` (modelo Sonar-Lite).

- **Infraestructura externa:** PostgreSQL 14, MinIO, Redis, Celery Worker, Celery Beat y Flower, orquestados con Docker Compose.

Las dependencias siguen una topología estrictamente vertical (*depends-on*): cada capa solo depende de la inmediatamente inferior, salvo la conexión adicional Celery Worker → Servicios para la ejecución asíncrona de tareas.

Diagrama de casos de uso

La Figura H.1 representa los ocho casos de uso del sistema y su asignación a los dos actores: Usuario autenticado y Sistema Celery (actor interno). La especificación textual de cada caso de uso se enumera a continuación.

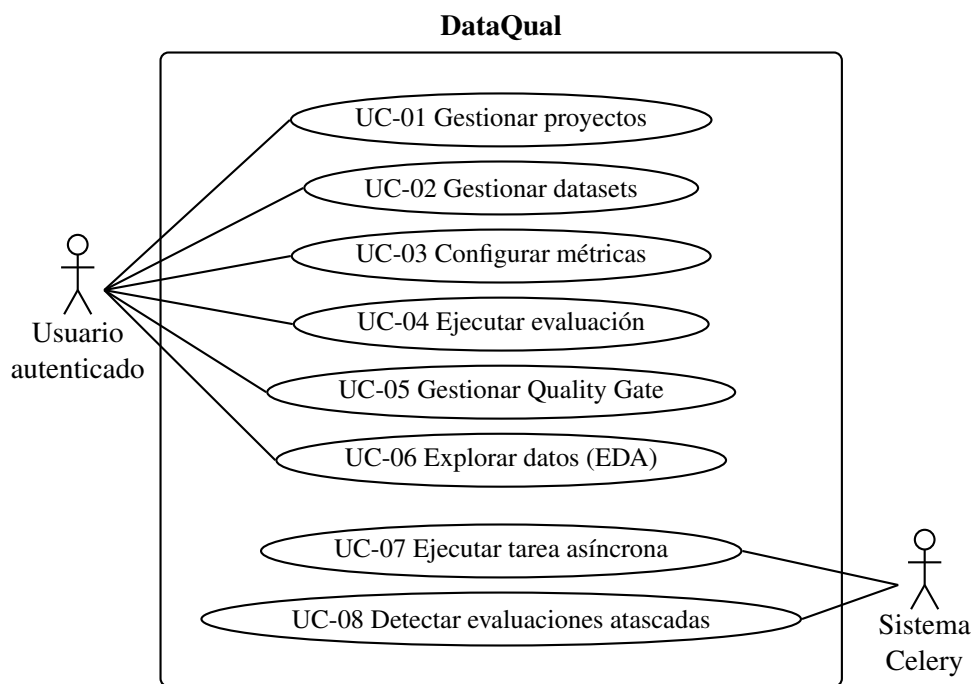


Figura H.1: Diagrama UML de casos de uso de DATAQUAL

■ Actor: Usuario autenticado

- UC-01: Gestionar proyectos (crear, editar, eliminar, listar)
- UC-02: Gestionar datasets (cargar, versionar, marcar PII)
- UC-03: Configurar métricas (seleccionar, parametrizar, plantillas)
- UC-04: Ejecutar evaluación (lanzar, monitorizar progreso, ver resultados)
- UC-05: Gestionar Quality Gate (configurar umbrales, consultar estado)
- UC-06: Explorar datos (EDA: histogramas, boxplots, correlaciones)

■ Actor: Sistema Celery (interno)

- UC-07: Ejecutar tarea de evaluación asíncrona
- UC-08: Detectar evaluaciones atascadas (Watchdog)

Diagrama entidad–relación (ERD)

Entidades y relaciones principales del esquema PostgreSQL:

- users (1) — (N) projects
- projects (1) — (N) datasets
- projects (1) — (N) evaluations
- projects (1) — (1) quality_gates
- datasets (1) — (N) datasets [versionado: parent_dataset_id]
- evaluations (1) — (N) issues
- evaluations (1) — (1) analysis_runs
- analysis_runs (1) — (N) data_quality_issues
- metric_templates (N) — referenciados por projects.metrics_config (JSONB)

Diagrama de secuencia: Ejecución de evaluación

Flujo principal de la ejecución de una evaluación (camino feliz):

1. **Frontend** → POST /api/evaluations/run [dataset_id, project_id]
2. **Backend** → Crea registro Evaluation (estado: PENDING)
3. **Backend** → Encola tarea Celery run_evaluation
4. **Backend** → Devuelve {evaluation_id} al Frontend
5. **Frontend** → Polling GET /api/evaluations/{id}/status (cada 2s)
6. **Celery Worker** → Descarga dataset de MinIO
7. **Celery Worker** → Invoca EvaluationService
8. **EvaluationService** → Ejecuta cada métrica configurada
9. **EvaluationService** → Calcula Quality Score
10. **EvaluationService** → Genera fingerprints y compara con baseline
11. **EvaluationService** → Evalúa Quality Gate
12. **Celery Worker** → Actualiza Evaluation y AnalysisRun en PostgreSQL (estado: COMPLETED)
13. **Frontend** → Detecta estado COMPLETED → Redirige a página de resultados

Diagrama de clases: Motor de métricas

Jerarquía de clases del motor de métricas (backend/services/metrics/):

- **BaseMetric** (abstracta)

- Atributos: `metric_id: str, name: str, threshold: float, weight: float`
- Métodos: `evaluate(df, params) → MetricResult (abstracto), _get_severity(), _mask_pii(), _infer_column_type()`
- **CompletenessMetric** hereda BaseMetric
- **UniquenessMetric** hereda BaseMetric
- **OutliersMetric** hereda BaseMetric
- **SyntacticAccuracyMetric** hereda BaseMetric
- **LogicalConsistencyMetric** hereda BaseMetric
- **ClassBalanceMetric** hereda BaseMetric
- **CurrentnessMetric** hereda BaseMetric
- **DiversityMetric** hereda BaseMetric
- **MetricResult** (dataclass): `metric_id, score, details, issues: List[Issue]`
- **METRIC_REGISTRY**: `Dict[str, Type[BaseMetric]]`

Referencias

- Abedjan, Z., Golab, L., Naumann, F., and Papenbrock, T. (2018). *Data Profiling*. Synthesis Lectures on Data Management. Morgan & Claypool Publishers.
- Alteryx, Inc. (2024). Alteryx: Data analytics and process automation platform. <https://www.alteryx.com/>. Consultado: marzo 2026.
- Anaconda, Inc. (2022). State of data science 2022.
- Bayer, M. (2024). SQLAlchemy: The database toolkit for Python. <https://www.sqlalchemy.org/>.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., and Thomas, D. (2001). Manifesto for agile software development. <https://agilemanifesto.org>. Consultado el 12 de mayo de 2026.
- Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, volume 81 of *Proceedings of Machine Learning Research*, pages 77–91. PMLR.
- Celery Project (2024). Celery: Distributed task queue. <https://docs.celeryq.dev/>.
- Chart.js Contributors (2024). Chart.js: Simple yet flexible JavaScript charting for designers & developers. <https://www.chartjs.org/>.
- CrowdFlower (2016). 2016 data science report.
- DAMA International (2017). *DAMA-DMBOK: Data Management Body of Knowledge*. Technics Publications, 2 edition.
- Dastin, J. (2018). Amazon scraps secret AI recruiting tool that showed bias against women. Reuters. Consultado: 2026.
- Docker, Inc. (2024). Docker: Accelerated container application development. <https://www.docker.com/>.
- DQOps (2024). DQOps — data quality operations center. <https://dqops.com/>. Consultado: marzo 2026.

- Evidently AI (2024). Evidently: Open-source ML monitoring and data quality framework. <https://www.evidentlyai.com/>. Consultado: marzo 2026.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine. Capítulo 5: Representational State Transfer (REST).
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Great Expectations (2024). Great expectations: Always know what to expect from your data. <https://greatexpectations.io/>. Consultado: marzo 2026.
- ISO/IEC (2008). ISO/IEC 25012:2008 — software engineering — software product quality requirements and evaluation (SQuaRE) — data quality model. Standard, International Organization for Standardization.
- ISO/IEC (2015). ISO/IEC 25024:2015 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — measurement of data quality. Standard, International Organization for Standardization.
- ISO/IEC (2024). ISO/IEC 5259 — data quality for analytics and machine learning (ML). Standard, International Organization for Standardization. Partes 1 a 5 publicadas en 2024.
- Kaggle (2022). State of data science and machine learning 2022. <https://www.kaggle.com/kaggle-survey-2022>. Consultado: 2026.
- Kreuzberger, D., Kühn, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.
- Meta Platforms, Inc. (2024). React: A javascript library for building user interfaces. <https://react.dev/>.
- Microsoft (2024). TypeScript: Javascript with syntax for types. <https://www.typescriptlang.org/>.
- MinIO, Inc. (2024). MinIO: High performance object storage. <https://min.io/>.
- MUI (2024). Material UI: React components for faster and easier web development. <https://mui.com/>.
- Ng, A. (2021). A chat with Andrew on MLOps: From model-centric to data-centric AI. DeepLearning.AI, <https://www.youtube.com/watch?v=06-AZXmWHjo>. Consultado: 2026.
- Parlamento Europeo y Consejo de la Unión Europea (2016). Reglamento (UE) 2016/679 del parlamento europeo y del consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (Reglamento General de Protección de Datos). Diario Oficial de la Unión Europea, L 119, pp. 1–88.
- Parlamento Europeo y Consejo de la Unión Europea (2024). Reglamento (UE) 2024/1689 del parlamento europeo y del consejo por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). Diario Oficial de la Unión Europea.

- Qlik Technologies, Inc. (2024). Talend: Data integration and integrity platform. <https://www.talend.com/>. Consultado: marzo 2026.
- Redis Ltd. (2024). Redis: In-memory data structure store. <https://redis.io/>.
- Redman, T. C. (1998). The impact of poor data quality on the typical enterprise. *Communications of the ACM*, 41(2):79–82.
- Redman, T. C. (2001). *Data Quality: The Field Guide*. Digital Press.
- Ronacher, A. (2024). Flask: A python microframework. <https://flask.palletsprojects.com/>.
- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., and Aroyo, L. M. (2021). “everyone wants to do the model work, not the data work”: Data cascades in high-stakes AI. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM.
- Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., and Grafberger, A. (2018). Automating large-scale data quality verification. In *Proceedings of the VLDB Endowment*, volume 11, pages 1781–1794. Apache Deequ.
- Schwaber, K. and Sutherland, J. (2020). The scrum guide: The definitive guide to scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>. Versión noviembre 2020.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.
- Soda Data, Inc. (2024). Soda core: Open-source data quality framework. <https://github.com/sodadata/soda-core>. Consultado: marzo 2026.
- Sommerville, I. (2016). *Software Engineering*. Pearson Education, 10 edition.
- SonarSource (2024). SonarQube: Code quality and security. <https://www.sonarsource.com/products/sonarqube/>.
- The PostgreSQL Global Development Group (2024). PostgreSQL: The world’s most advanced open source relational database. <https://www.postgresql.org/>.
- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- Union.ai and Pandera Contributors (2024). Pandera: A statistical data testing toolkit for Python. <https://pandera.readthedocs.io/>. Consultado: marzo 2026.
- Vercel (2024). Next.js: The react framework for the web. <https://nextjs.org/>.
- Wang, R. Y. and Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4):5–33.
- ydata-ai (2024). ydata-profiling: Extended pandas profiling. <https://github.com/ydataai/ydata-profiling>. Consultado: marzo 2026.

Este documento fue editado y tipografiado con $\text{\LaTeX}$
empleando la clase **gita-tfg** que se puede encontrar en:

<https://github.com/anarubioruiz/gita-tfg>

La clase **gita-tfg** está basada en la clase **esi-tfg**:

<https://github.com/UCLM-ESI/esi-tfg>